IT PROJECT 2 REPORT

CHAPTERS 1–6

Web-Based Preorder and Freshness Management System for Home-Based Entrepreneurs

Christopher Hansen

Student ID: 2402081406

Supervisor

Ms. Nur Elana Binti Mohd Saferinor

2026

ABSTRACT

Home-based food businesses need a way to coordinate advance orders, ingredient dates and customer choices without the overhead of a multi-vendor marketplace. This project implements Dapur Kita, a single-vendor web application built with React, Django and PostgreSQL. Customers can browse a branded menu, use explainable recommendations, maintain a basket and request delivery or collection. Server-side checks coordinate future handover with menu restrictions, accepted recipes, labour, equipment and existing bookings. Owners can confirm or replan work, manage dated stock and review sales and cost evidence.

Ingredient monitoring records printed, manufacture-based or receipt-based deadlines, documented storage guidance, earlier handling limits and holds. Actual food-readiness confirmation consumes accepted ingredients by earliest expiry and records batch evidence. The menu finder supports mixed or repeated-menu alternatives, a whole-order budget and bounded kitchen-capacity checks. FreshCast separately presents local database forecasts, an external Genpact CatBoost demonstration and an illustrative model-free walkthrough. Optional PayPal Sandbox, Sepolia escrow, manual courier records and consented email reminders retain explicit verification boundaries.

Verification at revision ab1a976 exercised 829 application tests across isolated SQLite and PostgreSQL environments, together with release checks and frontend suites. Retained browser evidence covers selected responsive, accessibility and recovery states. The original external-model holdout reproduced 28.23494% WAPE against 34.80762% for its baseline. These results establish tested software behaviour and an external benchmark comparison; they do not demonstrate real kitchen forecast accuracy, food safety, conversion uplift or user acceptance. An objective-linked user-evaluation protocol is provided for collecting the remaining evidence.

TABLE OF CONTENTS

CHAPTER 1....1
1.1 INTRODUCTION....1
1.2 PROBLEM STATEMENTS....2
1.3 OBJECTIVES....3
1.4 SCOPE....3
1.5 REVENUE MODEL....4
1.6 METHODOLOGY....5
1.7 GANTT CHART....5
1.8 SOFTWARE TOOLS....5
1.9 CONCLUSION....6
CHAPTER 2....7
2.1 INTRODUCTION....7
2.2 BACKGROUND STUDY....8
2.3 METHODOLOGY....14
2.4 TECHNOLOGY....16
2.5 IMPLEMENTATION IMPLICATIONS....17
CHAPTER 3....18
3.1 INTRODUCTION....18
3.2 AGILE METHODOLOGY....18
3.3 USE CASE DIAGRAMS....19
3.4 USE CASE SCENARIOS....23
3.5 ACTIVITY DIAGRAMS....33
3.6 SEQUENCE DIAGRAMS....37
3.7 CLASS DIAGRAM....48
3.8 ENTITY–RELATIONSHIP DIAGRAMS....50
3.9 REQUIREMENT TRACEABILITY AND DESIGN LIMITS....58
3.10 CONCLUSION....58
CHAPTER 4....59
4.1 INTRODUCTION....59
4.2 APPLICATION ARCHITECTURE....59
4.3 CUSTOMER INTERFACES....62
4.4 OWNER INTERFACES....70
4.5 KEY CALCULATIONS AND TRANSACTION RULES....80
4.6 FRESHCAST AND FORECASTING IMPLEMENTATION....82
4.7 PAYMENTS AND ASSISTED ORDERING....84
4.8 EMAIL REMINDERS, RECOVERY AND DEPLOYMENT....85
4.9 INTERFACE AND ACCESSIBILITY PRINCIPLES....86

4.10 CONCLUSION....87
CHAPTER 5....88
5.1 INTRODUCTION....88
5.2 TESTING APPROACH AND ENVIRONMENT....88
5.3 DECISION TABLES....89
5.4 FUNCTIONAL AND SECURITY TEST CASES....92
5.5 REGRESSION RESULTS....98
5.6 INTERFACE, RECOVERY AND PERFORMANCE FINDINGS....98
5.7 FORECAST EVALUATION AND DATA LIMITS....101
5.8 VALIDATION AGAINST THE OBJECTIVES....103
5.9 USER ACCEPTANCE AND FIELD-EVALUATION PROTOCOL....104
5.10 DISCUSSION IN RELATION TO THE LITERATURE....106
5.11 CONCLUSION....107
CHAPTER 6....108
6.1 INTRODUCTION....108
6.2 ACHIEVEMENT OF THE PROJECT OBJECTIVES....108
6.3 DEVELOPMENT CHALLENGES AND RESPONSES....110
6.4 ADVANTAGES OF THE IMPLEMENTED SYSTEM....111
6.5 LIMITATIONS....112
6.6 PRIORITISED FUTURE DEVELOPMENT....113
6.7 MAINTENANCE AND HANDOVER....114
6.8 CONCLUSION....115
REFERENCES....63
APPENDIX A — SUPERVISOR CONSULTATION RECORD....119
APPENDIX B — ORIGINALITY / SIMILARITY RECORD....119
APPENDIX C — DECLARATION OF INTEGRITY....120
APPENDIX D — APPLICATION DATA DICTIONARY....121
MenuPhoto — myapp_menuphoto....121
Category — myapp_category....121
Product — myapp_product....121
RawMaterial — myapp_rawmaterial....122
ProductIngredient — myapp_productingredient....123
InventoryItem — myapp_inventoryitem....123
Wallet — myapp_wallet....124
WalletTransaction — myapp_wallettransaction....124
Address — myapp_address....125
CartItem — myapp_cartitem....125
Order — myapp_order....126
OrderItem — myapp_orderitem....126

InventoryLog — myapp_inventorylog....127
Storefront — myapp_storefront....127
Review — myapp_review....129
OrderReminder — myapp_orderreminder....129
KitchenBlock — myapp_kitchenblock....129
OperatingExpense — myapp_operatingexpense....129
WasteRecord — myapp_wasterecord....130
OrderIngredient — myapp_orderingredient....130
IngredientConsumption — myapp_ingredientconsumption....131
PaymentEvent — myapp_paymentevent....131
AuthAttempt — myapp_authattempt....132
OrderAmendment — myapp_orderamendment....132
RecommendationSession — myapp_recommendationsession....132
RecommendationEvent — myapp_recommendationevent....132
APPENDIX E — USER-EVALUATION INSTRUMENT....134
APPENDIX F — IMPLEMENTATION AND EVIDENCE MAP....136
APPENDIX G — SHORT OPERATING GUIDE....138
G.1 Owner setup....138
G.2 Receiving ingredients....138
G.3 Accepting and preparing orders....138
G.4 Actual readiness and handover....138
G.5 Payment and reminders....138
G.6 Reading analytics and FreshCast....138
G.7 Recovery....139

LIST OF FIGURES

Figure 1.1: Original IT Project 1 proposal Gantt chart (retained; not an actual completion log).. 5
Figure 2.1: Original literature map retained from IT Project 1.....7
Figure 3.1: Customer and visitor use cases.....20
Figure 3.2: Owner and integration use cases.....22
Figure 3.3: Preorder checkout with confirmed and unconfirmed paths.....34
Figure 3.4: Inventory evidence, effective deadline and handling hold.....35
Figure 3.5: Calculation and confirmation of an order preparation plan.....36
Figure 3.6: Grouped food-readiness and actual handover activity.....37
Figure 3.7: Explainable discovery and whole-order date guidance.....38
Figure 3.8: Basket mutation, refresh and timed confirmation.....39
Figure 3.9: Authoritative quote and retry-safe placement.....40
Figure 3.10: Server-verified PayPal Sandbox payment.....41
Figure 3.11: Replanning with a signed, expiring preview.....42
Figure 3.12: FEFO deduction and all-or-nothing progress updates.....43
Figure 3.13: Recording batch evidence without overwriting a newer edit.....44
Figure 3.14: Sales and recorded contribution reporting.....45
Figure 3.15: Source-labelled forecasts and read-only scenarios.....46
Figure 3.16: One authenticated worker for owner and opted-in customer emails.....47
Figure 3.17: Sepolia escrow invoice and independent funding verification.....48
Figure 3.18: Core domain classes and service-module responsibilities.....49
Figure 3.19: Commerce, basket, addresses and order-line ERD.....51
Figure 3.20: Current recipes, accepted recipes, stock and consumption ERD.....53
Figure 3.21: Operational settings, amendments, reminders and payments ERD.....55
Figure 3.22: Reviews, recommendation telemetry, menu photos and attempt limits.....57
Figure 4.1: Implemented presentation, API, service and persistence layers.....60
Figure 4.2: Current customer menu with search, categories and compact food cards (local test records).....62
Figure 4.3: Product detail showing price, selling unit and preorder information (local test record).....63
Figure 4.4: Whole-order date finder with three total items and an optional RM100 budget (actual local search).....65
Figure 4.5: Basket quantities, fulfilment choice and authoritative summary (local test records). 66
Figure 4.6: Checkout separates fulfilment, address, time and payment (local test record; no payment submitted).....68
Figure 4.7: Customer order cards, private filters and payment/fulfilment separation (local test records).....69
Figure 4.8: Owner menu list with price, recipe and lifecycle actions (local test catalogue).....70

Figure 4.9: Menu editor with selling details and progressive recipe/preparation settings (local unsaved form).....71
Figure 4.10: Inventory overview, held/dated stock and batch review (local test records).....72
Figure 4.11: Received-date calculation for an unlabelled ingredient; owner-supplied evidence is required (unsaved local form).....74
Figure 4.12: Planner with dated work, review queues and kitchen availability (local test date).75
Figure 4.13: Owner analytics with distinct sales and evidence-based report tabs (local test data). 78
Figure 4.14: Owner Settings and focused configuration sections (local test settings).....79
Figure 4.15: Kitchen FreshCast decision assistant with source and evidence labels (local scenario)..... 82
Figure 4.16: Phone menu with compact navigation, food cards and basket access (390-pixel local viewport).....87
Figure 5.1: Reproduced Genpact rolling one-step WAPE on the original holdout; lower is better. 102

LIST OF TABLES

Table 2.1: Summary of findings on Malaysian online food delivery consumer behaviour.....8
Table 2.2: Research areas, focus and identified gaps retained from the proposal.....11
Table 2.3: Proposal-stage feature comparison; provider features/regions may change.....13
Table 2.4: Agile and traditional comparison, adapted from the original proposal summary (Islam & Ferworn, 2020).....14
Table 2.5: Comparison of development dimensions, adapted from the proposal's Leau et al. (2012) summary.....15
Table 3.1: Agile methodology.....19
Table 3.2: Requirement traceability and design limits.....58
Table 4.1: Application architecture.....61
Table 4.2: Ingredient directory and inventory.....72
Table 4.3: Three data sources and their purposes.....82
Table 5.1: Testing approach and environment.....88
Table 5.2: Checkout and request confirmation.....89
Table 5.3: Batch eligibility and deadlines.....90
Table 5.4: Actual food readiness and grouped progress.....91
Table 5.5: Payment, reminder and budget evidence.....91
Table 5.6: Executed functional cases BT-01–BT-06.....92
Table 5.7: Executed functional cases BT-07–BT-12.....93
Table 5.8: Executed functional cases BT-13–BT-18.....94
Table 5.9: Executed functional cases BT-19–BT-24.....95
Table 5.10: Executed functional cases BT-25–BT-30.....95
Table 5.11: Executed functional cases BT-31–BT-36.....96
Table 5.12: Executed functional cases BT-37–BT-42.....97
Table 5.13: Regression results.....98
Table 5.14: Defects and retested corrections.....99
Table 5.15: External genpact model recovery.....101
Table 5.16: Validation against the objectives.....103
Table 5.17: Tasks linked to objectives.....104
Table 6.1: Limitations.....112

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application programming interface
CSRF	Cross-site request forgery
DRF	Django REST Framework
ERD	Entity–relationship diagram
FEFO	First-expiry, first-out
FK / PK	Foreign key / primary key
FYP	Final-year project
MAE	Mean absolute error
MYR / RM	Malaysian ringgit
ORM	Object-relational mapper
RPC	Remote procedure call
SMTP	Simple Mail Transfer Protocol
UAT	User acceptance testing
UML	Unified Modeling Language
WAPE	Weighted absolute percentage error
WCAG	Web Content Accessibility Guidelines

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The home-based food industry experienced significant growth during and after the COVID-19 pandemic (Pallianysamy et al., 2025). According to Statista (2024), the online food delivery market in Malaysia is forecasted to reach USD 2.77 billion in 2024 and is projected to grow to USD 4.63 billion by 2029.

However, existing food ordering platforms may not adequately address the specific needs of home-based food entrepreneurs, particularly in supporting structured pre-order operations. Some local home-based food sellers might rely on WhatsApp for ordering — either through its structured cart feature or informal chat — or use large food delivery platforms such as GrabFood and FoodPanda. While the big platforms offer scheduled ordering, the proposal-stage comparison recorded advance booking windows of approximately 1–2 days, which may not align with the needs of home-based sellers who prepare food in batches and require confirmed orders further in advance. WhatsApp, meanwhile, has no proper mechanism for date-based preordering in either mode: in its cart feature, any date must be manually written into the product title or description, requiring the seller to read and verify each date themselves rather than having the system check availability or capacity automatically; chat-based arrangements, similarly, rely entirely on the seller manually tracking dates and capacity.

Neither WhatsApp nor large food ordering platforms such as GrabFood and FoodPanda offer any feature for monitoring ingredient freshness, leaving sellers without a structured way to track raw food expiry. This is particularly concerning given that storing large quantities of perishable goods increases the risk of microbial growth (Pallianysamy et al., 2025), a risk further worsened by Malaysia's indoor temperature range of 21°C to 37°C (Gou et al., 2018), which accelerates food spoilage. Real-life food poisoning incidents, such as the illness of 65 primary school students after consuming flat noodle soup in May 2024 ("Sixty-five Sungai Petani primary school pupils suffer food poisoning," 2024) and the deaths caused by spoiled fried noodles and eggs in Selangor, highlight the seriousness of this problem ("Food poisoning turns fatal," 2024).

In addition, GrabFood and FoodPanda place multiple competing sellers within the same marketplace environment, limiting opportunities for sellers to establish distinctive branding and direct customer relationships. WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence. Furthermore, the basic WhatsApp ordering workflow reviewed here does not provide the integrated analytics and menu-recommendation functions proposed for this system. Sellers who need to adopt a different platform for preordering, freshness monitoring, or a dedicated storefront would also benefit from integrated analytics and recommendation features, rather than relying on separate or absent tools. The absence of recommendation mechanisms may contribute to customer decision fatigue, which could negatively influence purchasing decisions and potentially increase the likelihood of purchase abandonment, while limited access to sales analytics restricts sellers' ability to monitor performance and make informed business decisions.

To address these issues, this proposal introduces a web-based food pre-ordering system that allows home-based food sellers to manage personalised storefronts and strengthen their branding efforts. The system also includes raw ingredient freshness monitoring based on storage duration, sales analytics to support better business decisions, and a menu recommendation system to assist customers in selecting suitable menu options and enhance the overall purchasing experience.

In conclusion, the proposed web-based system aims to assist home-based food sellers in managing pre-order operations more efficiently through a dedicated platform tailored to their business needs. The system further incorporates ingredient freshness monitoring to support food safety practices, while providing personalized storefront management, sales analytics, and menu recommendation features to enhance sellers' operational effectiveness and customer experience.

1.2 PROBLEM STATEMENTS

1. Lack of reliable advance preordering: The proposal-stage comparison recorded approximately 1–2-day advance windows for GrabFood and FoodPanda, which may not align with the needs of sellers preparing food in batches further ahead. WhatsApp's cart feature has no structured date field, requiring sellers to manually write and verify dates themselves rather than having the system check availability automatically. Chat-based ordering has no structure at all, leaving sellers to track every order manually.

2. Lack of ingredient freshness monitoring: The consumer ordering interfaces compared in this proposal do not provide the proposed kitchen-level ingredient shelf-life and storage records, limiting sellers' ability to proactively manage food safety.

3. Lack of dedicated single-vendor storefront: GrabFood and FoodPanda place competing sellers in the same multivendor environment, limiting branding opportunities. WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence.

4. Limited business and decision support tailored to small operators: The basic WhatsApp ordering workflow reviewed here does not provide these integrated analytics or menu-recommendation functions. Sellers who need to adopt a different platform for preordering, freshness monitoring, or a dedicated storefront would also benefit from integrated analytics and recommendation features, rather than relying on separate or absent tools.

1.3 OBJECTIVES

1. To design and develop a web application that supports flexible date-based food preordering and scheduling for home-based food entrepreneurs.

2. To implement a food stock freshness monitoring feature based on ingredient shelf life and storage duration.

3. To develop a personalised single-vendor storefront module that allows sellers to strengthen branding without direct competitor distraction.

4. To implement a sales analytics dashboard that supports data-driven business decision-making for home-based food operators.

5. To develop a menu recommendation system that reduces customer decision fatigue and improves purchase conversion rates.

1.4 SCOPE

1.4.1 Customer Scope

- Customer can view the seller's storefront and product listings with category filters
- Customer can select available preorder dates when placing orders
- Customer can view food recommendations and add items to cart

1.4.2 Food Entrepreneur Scope

- Seller can log in as admin and manage product listings
- Seller can manage preorder availability settings
- Seller can monitor ingredient freshness status based on remaining shelf life duration
- Seller can view sales analytics to support business decisions
- Seller can customise their storefront including store name, story, contact information and menu photographs

1.4.3 System Scope

- The system checks future preorder dates and kitchen availability, and distinguishes confirmed bookings from explicit owner-review requests
- The system tracks and displays ingredient freshness status based on remaining shelf life
- The system shows recommended and all product listings filterable by category
- The system displays relevant features based on user role — entrepreneur or customer

1.4.4 IMPLEMENTED SCOPE CLARIFICATION

The implementation remains a single-kitchen web system. It adds collection, documented storage guidance, accepted-order snapshots, multi-day preparation, grouped fulfilment, price/contribution evidence and optional decision/payment/reminder integrations. Requests needing review remain unpaid and do not reserve preparation. Date-based stock monitoring does not measure physical food safety. These refinements support the five original objectives without changing their wording.

1.5 REVENUE MODEL

The proposed solution is designed to help home-based food entrepreneurs manage preordering operations through a structured and fixed-cost platform with record-based ingredient-date monitoring and personalised branding. The proposed commercial model has two revenue streams; automated seller subscription billing is outside the implemented system:

1. Upfront payment A one-time fee is charged for initial system access and setup.

2. Recurring fee Monthly maintenance and hosting fees are charged to sustain system operation.

1.6 METHODOLOGY

This project follows an Agile methodology due to its suitability for small-scale iterative development. Development is organised into iterative cycles where each cycle involves requirement gathering, design, implementation and testing of specific features. The process begins with identifying home-based food entrepreneurs' needs and customer pain points. Each feature — including product preordering, ingredient freshness monitoring, storefront management, sales analytics and menu recommendation — is then designed, implemented and tested iteratively. The system is refined based on feedback and testing outcomes throughout development.

1.7 GANTT CHART

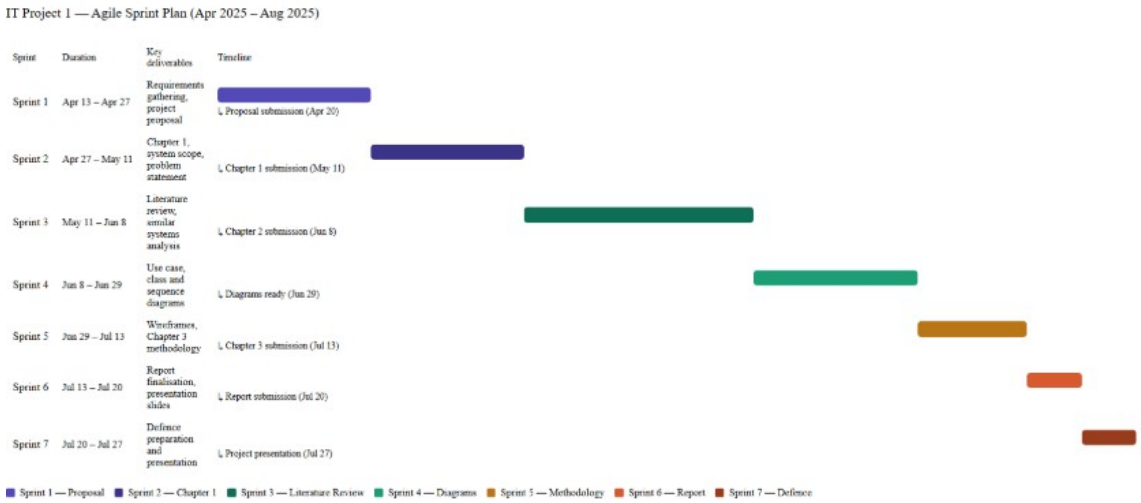


Figure 1.1: Original IT Project 1 proposal Gantt chart (retained; not an actual completion log).

1.8 SOFTWARE TOOLS

1. React — Frontend framework for building interactive user interfaces

2. Django — Backend framework supporting rapid development with built-in ORM and MVT architecture

3. PostgreSQL — Relational database suited for structured interconnected data entities

4. Visual Studio Code — Code editor with extensive Python and JavaScript extension support

5. GitHub — Version control platform for tracking code changes and managing development history

1.9 CONCLUSION

The proposed system aims to help home-based food entrepreneurs manage preorder operations more efficiently through a dedicated fixed-cost web platform tailored to their business needs.

By providing flexible date-based preordering, ingredient freshness monitoring, personalised storefront management, sales analytics and menu recommendation features, the system addresses key operational gaps faced by home-based food entrepreneurs. Operating through a dedicated platform allows sellers to maintain their own branded storefront and build direct customer relationships independent of large multivendor marketplace environments.

The project follows an Agile methodology to support iterative feature development and flexibility throughout the development process. Therefore, the proposed system supports the growth of local home-based food SMEs while contributing to food safety awareness and improved customer purchasing experience.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

This chapter consists of three parts including background study, methodology, and technology.

The background study discusses the evolution of the food ordering industry, digital transformation in Malaysia's food and beverage (F&B) sector, and the research interest landscape related to preorder and freshness management systems. In addition, similar existing systems and current technological trends relevant to the proposed system will also be reviewed and discussed.

The methodology section discusses the comparison between Agile and Traditional software development approaches to determine the most suitable methodology for the proposed system.

Meanwhile, the technology section describes the framework and database technologies considered for the project. A comparison between Django and Express.js will be carried out to determine the most suitable framework for the proposed system. Moreover, a comparison between Firebase and PostgreSQL will also be conducted to identify the most suitable database solution for the proposed system.

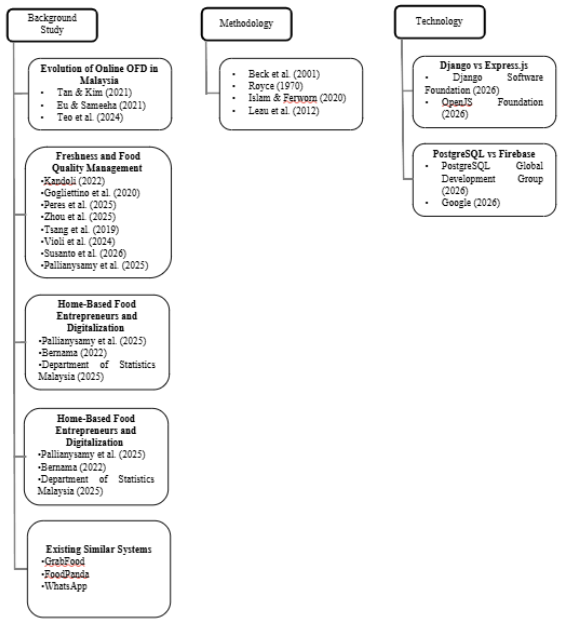


Figure 2.1: Original literature map retained from IT Project 1.

2.2 BACKGROUND STUDY

2.2.1 Evolution of Online Food Ordering Industry in Malaysia

The Online Food Ordering industry boomed in Malaysia in 2017, driven by smartphone adoption and internet accessibility (Tan & Kim, 2021). The COVID-19 pandemic further accelerated this growth as consumers shifted toward online food ordering amid the shutdown of non-essential services and dining restrictions (Eu & Sameeha, 2021). Within this growing market, research has identified food quality and health considerations as significant factors influencing Malaysian consumer behavior in OFD platforms (Teo et al., 2024).

Therefore, understanding consumer expectations regarding food quality and freshness within Malaysia's digital food ecosystem is important in identifying the relevance of the proposed web-based preorder and freshness management system. Table 2.1 presents an analysis of key findings and their implications for the Malaysian OFD landscape.

Table 2.1: Summary of findings on Malaysian online food delivery consumer behaviour.

APA citation	Key finding	Relevance
Tan & Kim (2021)	Food quality, service approach and perceived price were linked to satisfaction.	Supports attention to food quality and service.
Eu & Sameeha (2021)	76.9% perceived limited healthy options; 85.9% expressed willingness to purchase healthy food.	Highlights a gap in perceived choice/quality.
Teo et al. (2024)	Health, quality, service and value influence continuance intention.	Supports integrating quality and service information.

Across these studies, food quality and healthy food availability consistently emerge as critical factors influencing Malaysian consumers' OFD usage and satisfaction. However, existing platforms are perceived as inadequately addressing these concerns. Given that freshness is identified as a core component of food quality (Tan & Kim, 2021), this suggests a gap for more practical freshness management features within digital food ordering systems.

2.2.2 Freshness and Food Quality Management in Digital Food Systems

Food storage is one method used to preserve food for future consumption (Kandoli, 2022). However, certain perishable food products with high nutrient content

remain vulnerable to microbial contamination and pathogen growth if storage conditions are not properly managed (Gogliettino et al., 2020). The presence and growth of these microorganisms influence the freshness status and quality of raw food, thereby requiring proper freshness management before food is served to consumers.

Food quality management has become a critical aspect of the food industry, particularly in ensuring food safety and freshness throughout the supply chain. Recent studies have explored the integration of Industry 4.0 technologies such as sensor systems, blockchain, and artificial intelligence to improve food monitoring, process automation, and traceability mechanisms (Peres et al., 2025). Furthermore, TTIs and FIs were explored in food shelf-life monitoring (Zhou et al., 2025).

However, these technologies are generally implemented within large-scale food supply chains and industrial environments where structured operational processes, dedicated quality control divisions, and larger financial resources are available. Such implementations commonly emphasize end-to-end supply chain monitoring rather than operational-level freshness management for small-scale food services or home-based food businesses.

Another study explored the integration of IoT, fuzzy logic, and zero-order quality degradation models into an Internet of Things-based Shelf Life Management System (Tsang et al., 2019). Nevertheless, the implementation complexity and infrastructure requirements of such systems may reduce their suitability for home-based food microbusinesses.

Similarly, Violi et al. (2024) proposed a mathematical inventory-routing model for perishable products. The model assists warehouses in determining delivery priorities to minimize spoilage while addressing stochastic demand using historical sales data and Conditional Value-at- Risk (CVaR) risk analysis. However, the computational complexity and integration requirements of such approaches may limit their applicability in small-scale academic projects and home-based food business operations. Furthermore, these systems may require IoT infrastructure to monitor physical inventory movement, potentially increasing implementation costs.

Susanto et al. (2026) also proposed a Quality Management System framework for food SMEs consisting of Quality Planning, Quality Assurance, and Quality Control and Improvement stages. However, the implementation scope of such systems may exceed the operational requirements of small-scale home-based food businesses.

Moreover, Pallianysamy et al. (2025) highlighted the need for monitoring and inspection solutions for home-based food operations from a food safety perspective. Nevertheless, implementing advanced monitoring technologies such as cameras, sensors, or IoT devices may introduce additional infrastructure and maintenance costs that are less suitable for lightweight small-scale systems.

Therefore, a research gap exists in the development of lightweight and cost-effective freshness management approaches that can be integrated into smaller-scale digital food systems such as online preorder and home-based food ordering platforms.

2.2.3 Home-Based Food Entrepreneurs and Digitalization

Home-based food businesses in Malaysia represent a growing segment particularly following COVID-19 (Pallianysamy et al., 2025). This growth coincides with significant expansion in Malaysia's digital food market — ecommerce transactions increased by 21.8 percent from 2020 to RM1.1 trillion in 2021 (Bernama, 2022), with revenue for the first nine months of 2025 reaching RM937.5 billion (Department of Statistics Malaysia, 2025).

However, as identified in Section 2.2.5, existing platforms present several limitations for home-based food operators. GrabFood and FoodPanda were recorded in the proposal comparison as offering approximately 1–2-day advance windows, which is insufficient for sellers who prepare food in batches and require confirmed orders further ahead of time. WhatsApp's Cart feature has no structured date field, requiring sellers to manually write and verify dates themselves rather than having the system check availability automatically, while informal chat-based ordering has no structure at all, making it difficult for sellers to reliably track orders and avoid missing them amid casual conversation. None of these platforms provide ingredient freshness monitoring features.

Furthermore, existing platforms lack dedicated personalised storefronts — GrabFood and FoodPanda offer limited personalisation within competitive multivendor environments, while WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence. While dedicated single-vendor ordering systems exist for large established food brands such as McDonald's, these systems are developed for large-scale commercial operations and are not accessible to informal home-based food micro-entrepreneurs as ready-to-use platforms.

Home-based entrepreneurs whose operational needs — flexible preordering, freshness monitoring, or a dedicated single-vendor storefront — are not met by GrabFood, FoodPanda, or WhatsApp would therefore need to adopt a different platform altogether; once on such a platform, they would also benefit from integrated sales analytics and recommendation features rather than having to rely on separate or absent tools. While GrabFood and FoodPanda do provide sales analytics through vendor dashboards (Table 2.3), and WhatsApp offers none at all, neither pathway currently combines these analytics and recommendation capabilities with the preorder, freshness, and storefront features that the proposed system addresses together as a single integrated solution.

2.2.4 Research Interest & Academic Relevance Landscape

Tan & Kim (2021) examined consumer satisfaction in Malaysian OFD, focusing on factors such as food quality and pricing. However, this study focuses on consumer satisfaction rather than operational tools for food businesses.

In addition, research on healthy food awareness in OFD platforms indicates that Malaysian consumers value healthier food options and food quality visibility (Eu & Sameeha, 2021). However, these studies mainly focus on healthy food availability perception and food choice behavior, rather than real-time freshness monitoring or freshness-status tracking within online food ordering systems.

Moreover, recent studies exploring digital technologies in food monitoring suggest increasing adoption of Artificial Intelligence (AI), Internet of Things (IoT), and smart sensors for food tracking and waste reduction (Zhou et al., 2025). However, these solutions are often complex and costly, making them less suitable for small-scale food businesses. Therefore, there is a need for a more lightweight and cost-effective approach to freshness management, such as a system based on shelf-life tracking and inventory entry time. Table 2.2 presents an analysis of existing research areas and their identified gaps relevant to the proposed system.

Table 2.2: Research areas, focus and identified gaps retained from the proposal.

Research area	Study focus	Gap relevant to this project	Citation
Consumer satisfaction	Quality, price and service	Operational preorder/freshness tools	Tan & Kim (2021)
Healthy food availability	Perceptions of variety and price	Practical ingredient-date visibility	Eu & Sameeha (2021)

Research area	Study focus	Gap relevant to this project	Citation
Digital food monitoring	AI/IoT and larger supply chains	Lightweight tools for small operators	Zhou et al. (2025)

Overall, the existing literature tends to focus either on consumer behavior, healthy food availability perception, or advanced technological solutions, with limited attention given to affordable, practical freshness management systems tailored for home-based food entrepreneurs. This identifies a clear research gap that the proposed web-based preorder and freshness management system aims to address.

2.2.5 Existing Similar Systems

The following platform descriptions and Table 2.3 preserve the proposal-stage comparison. Ordering windows and commercial features can vary by region, account and date; they were not revalidated as current provider promises for this implementation review. The comparison is not a universal claim that no other software supports these functions.

This section compares existing systems to identify features currently available and limitations that the proposed system aims to address.

2.2.5.1 GrabFood

GrabFood is the most popular food ordering platform among university students (Lok et al., 2024). It was founded in 2018 which provided online food delivery solutions in Southeast Asia (Grab, 2018). The platform primarily focuses on registered restaurant vendors and large-scale food businesses and was recorded in the proposal comparison as allowing advance preordering up to two days. Moreover it is a multivendor platform, therefore, they provide less freedom in personalized content. However, publicly available platform features indicate limited support for freshness-status monitoring and inventory- oriented analytics.

2.2.5.2 FoodPanda

FoodPanda was launched in 2012 before expanding across Southeast Asia (Foodpanda, 2012). It allows advance preordering up to 1 day. Same with GrabFood, the platform primarily focuses on registered restaurant vendors and large-scale food businesses. It allows limited preordering and does not provide personalized storefront. It also does not offer food freshness monitoring features, and provides limited sales

analytics through its vendor dashboard. Recommendation capabilities are relatively limited and primarily focused on vendor visibility rather than personalized freshness-oriented recommendations.

2.2.5.3 WhatsApp

WhatsApp may be used as an informal preorder communication platform among small- scale food businesses and microentrepreneurs in Southeast Asia. However, the preorder process is largely unstructured and dependent on manual chat management, which may increase the risk of missed orders and communication inefficiencies. In addition, WhatsApp does not provide dedicated features such as freshness monitoring, analytics dashboards, or personalized preorder management.

This section reviews several existing platforms related to online food ordering and preorder management to identify their functionalities, limitations, and relevance to the proposed system. The comparison focuses on preorder flexibility, personalization, freshness monitoring, sales analytics, and recommendation capabilities. Through this comparison, the limitations of current systems can be identified to justify the need for the proposed system.

Table 2.3: Proposal-stage feature comparison; provider features/regions may change.

Feature	GrabFood	foodpanda	WhatsApp	McDonald's	Proposed system
Advance ordering	Up to 2 days	Up to 1 day	Manual date in conversation	Limited scheduling	Flexible seller rules
Single-vendor storefront	Marketplace	Marketplace	Basic catalogue/chat	Brand-owned	Dedicated kitchen
Ingredient-date monitoring	Not in compared interface	Not in compared interface	Manual records	Not in compared interface	Shelf-life records
Sales analytics	Vendor analytics	Vendor analytics	Not in basic ordering flow	Not evaluated	Integrated reports
Recommendations	Platform-based	Category-based	Not in compared flow	Not evaluated	Explainable ranking

Note. McDonald's is included as a reference example of an existing single-vendor dedicated ordering system. It is not a practical alternative for home-based food entrepreneurs, as such systems are developed for large-scale commercial operations and are not accessible as ready-to-use platforms for informal micro-entrepreneurs.

2.3 METHODOLOGY

Agile methodology is an iterative software development approach where developers can always revisit earlier stages of the Software Development Life Cycle after feedback. According to Beck et al. (2001), the Agile Manifesto emphasizes four core values: individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. Individuals and interactions over processes and tools means the methodology capitalizes more on human resources rather than advanced technologies or tools. Working software over comprehensive documentation means the methodology makes use of early prototypes for testing purposes rather than planning everything from start through documentation. Customer collaboration over contract negotiation means the method prefers continuous customer feedback rather than a fixed negotiation contract at the beginning. Responding to change over following a plan means the methodology prioritises adapting to new information and feedback rather than rigidly adhering to an initial plan.

A traditional model is a software development process where each stage is rigidly planned from the beginning. According to Royce (1970) it consists of system requirements, software requirements, analysis, program design, coding, testing, operation. He, however, said that the model is so simple that it's very risky for larger and more costly software projects.

Agile methodology is an incremental software development process while traditional methodology is a more formal or plan-driven software development approach (Islam & Ferworn, 2020). The comparisons between the two are described in the table below.

Table 2.4: Agile and traditional comparison, adapted from the original proposal summary (Islam & Ferworn, 2020).

Dimension	Agile (proposal summary)	Traditional (proposal summary)
Approach	Adaptive	Predictive
Success emphasis	Business value	Conformance to plan
Typical project context	Small / evolving	Larger / stable
Management emphasis	Decentralised	Centralised
Change	Adaptability	Controlled changes

Dimension	Agile (proposal summary)	Traditional (proposal summary)
Culture	Collaboration	Formal coordination
Documentation	Selected supporting records	Extensive records
Focus	People and iteration	Process and sequence
Cycles	Numerous	Limited
Domain	Exploratory	Predictable
Team	Small / adaptable	Larger / structured
Upfront planning	Incremental	Comprehensive
Return	Incremental delivery	Later completion

This table summarises the proposal's comparison rather than establishing universal rules about documentation, team size or engineering skill. The present project uses iterative development with written requirements, diagrams and automated tests in every relevant increment.

Agile methodology is selected for this project due to its suitability for small-scale projects, support for iterative feature development, and flexibility to accommodate requirement changes throughout the development process. These characteristics align with the nature of this academic project, where features may be refined or extended based on feedback and testing outcomes. Leau et al. (2012) provide a comparative analysis between Agile and traditional methodologies regarding rework costs, testing protocols, and project scalability, as outlined in Table 2.5.

Table 2.5: Comparison of development dimensions, adapted from the proposal's Leau et al. (2012) summary.

Dimension	Agile	Traditional
Requirements	Iteratively refined	Defined before implementation
Rework	Earlier feedback can reduce later rework	Late changes can be expensive
Direction	Changeable within iterations	More fixed baseline
Testing	Each iteration	Historically later stages
Customer involvement	Frequent	Often concentrated at milestones
Developer skills	Interpersonal and business knowledge	Technical and coordination skills still required

Dimension	Agile	Traditional
Project scale in proposal	Low to medium	Larger plan-driven context

This table summarises the proposal’s comparison rather than establishing universal rules about documentation, team size or engineering skill. The present project uses iterative development with written requirements, diagrams and automated tests in every relevant increment.

2.4 TECHNOLOGY

2.4.1 Django vs Express.js

According to the Django Software Foundation (2026), Django is a high-level Python web framework designed to support the rapid development of backend code. The framework is widely adopted for its Model-View-Template (MVT) architecture, which separates the application's core data, logic, and user interface. Within this architecture, a Model is defined as a Python class object that maps directly to a database table, providing an object-relational interface to database queries (Django Software Foundation, 2026). The View serves as a function or class that accepts web requests and returns the corresponding web responses. Finally, the Django architecture incorporates Templates, which consist of HTML code enhanced with template tags to dynamically display the data retrieved from the database to the end user (Django Software Foundation, 2026).

According to the OpenJS Foundation (2026), Express.js is a minimalist web framework designed to provide a robust set of features for web and application development. The framework relies primarily on two core functionalities: routing and middleware. Routing defines how application endpoints or URIs respond to specific incoming client requests, while middleware functionality allows developers to access and manipulate the request (req) and response (res) objects during the application's cycle (OpenJS Foundation, 2026).

Django was selected over Express.js due to prior development experience with the framework, which reduces learning overhead in a time-constrained academic project. Additionally, Django's MVT architecture enforces a clear project structure that supports code maintainability and organisation throughout development.

2.4.2 PostgreSQL vs Firebase

According to PostgreSQL Global Development Group (2026), PostgreSQL is an open- source object-relational database that has been developed for more than 35 years, making it a reputable database. It has many features that help developers in application building, administrators that protect data integrity, and support datasets of varying sizes. Furthermore, many features that are supported by SQL standard are also supported by PostgreSQL making it compliant with the SQL standard.

The Google (2026) comparison concerns Cloud Firestore, Firebase's document-oriented database, rather than every service under the Firebase platform. Meanwhile, it stores data in documents where each document contains key-value pairs. A document object is a type of JSON object that supports extra data types and has document size limits. Collection is the container that holds instances of the same data structure.

PostgreSQL is selected as its relational data model better suits structured data with interconnected entities, improving readability and maintainability compared to Firebase's document-based storage which may result in deeply nested data structures that are harder to navigate and maintain. Furthermore, Firebase's NoSQL structure is not natively supported by Django's ORM, which would require bypassing Django's built-in database management tools and increasing implementation complexity.

2.5 IMPLEMENTATION IMPLICATIONS

The literature rationale remains the same: a lightweight integrated tool is relevant to a home kitchen whose workflow is not represented by a short-notice marketplace checkout. The final implementation uses React for presentation and Django/DRF for its server API, not Django business-page templates. Storage evidence and holds translate the quality discussion into practical records; they do not implement the sensor/physical monitoring discussed in the reviewed research. Explainable recommendations and baseline forecasting support decisions, while their behavioural/business effects require evaluation. Chapters 3–6 distinguish implemented features from those unmeasured effects.

CHAPTER 3

METHODOLOGY AND SYSTEM DESIGN

3.1 INTRODUCTION

This chapter retains the project's iterative Agile approach and updates the design to the implemented Dapur Kita system at revision ab1a976 (29 October 2026). The five objectives in Section 1.3 remain unchanged. The diagrams distinguish customer and owner actions from external payment, email and forecasting services. A use case describes an observable user goal; an activity diagram describes decisions in a workflow; a sequence diagram identifies time-ordered component messages; and class and entity–relationship diagrams describe domain responsibilities and persistent relationships (Sommerville, 2016).

The implementation uses Django's Product model for the menu entity. RawMaterial identifies a measured ingredient, ProductIngredient is the current recipe junction, and InventoryItem identifies an individual received batch. OrderIngredient and preparation snapshots preserve what was accepted for an order. SalesReport is not a database model: reports are calculated views of recorded transactions. The design therefore avoids the earlier diagram's implication that a separate SalesReport entity stores all analytics.

3.2 AGILE METHODOLOGY

Development was organised into repeated requirements, design, implementation, verification and refinement cycles. This is an individual, Agile-inspired project rather than a claim that a fully staffed Scrum team conducted formal ceremonies. The approach follows the Agile emphasis on working software and responsiveness to change described in Chapter 2 (Beck et al., 2001). Feedback raised during development was converted into bounded changes and checked before release.

The implementation history includes refinement of automatic scheduling, separation of unconfirmed requests from accepted plans, expiry evidence, current and historical cost calculations, recommendation budgets, payment verification and responsive layouts. These changes illustrate iterative development: an early solution was tested against a concrete scenario, its limitation was identified, and its behaviour was revised while preserving accepted order history.

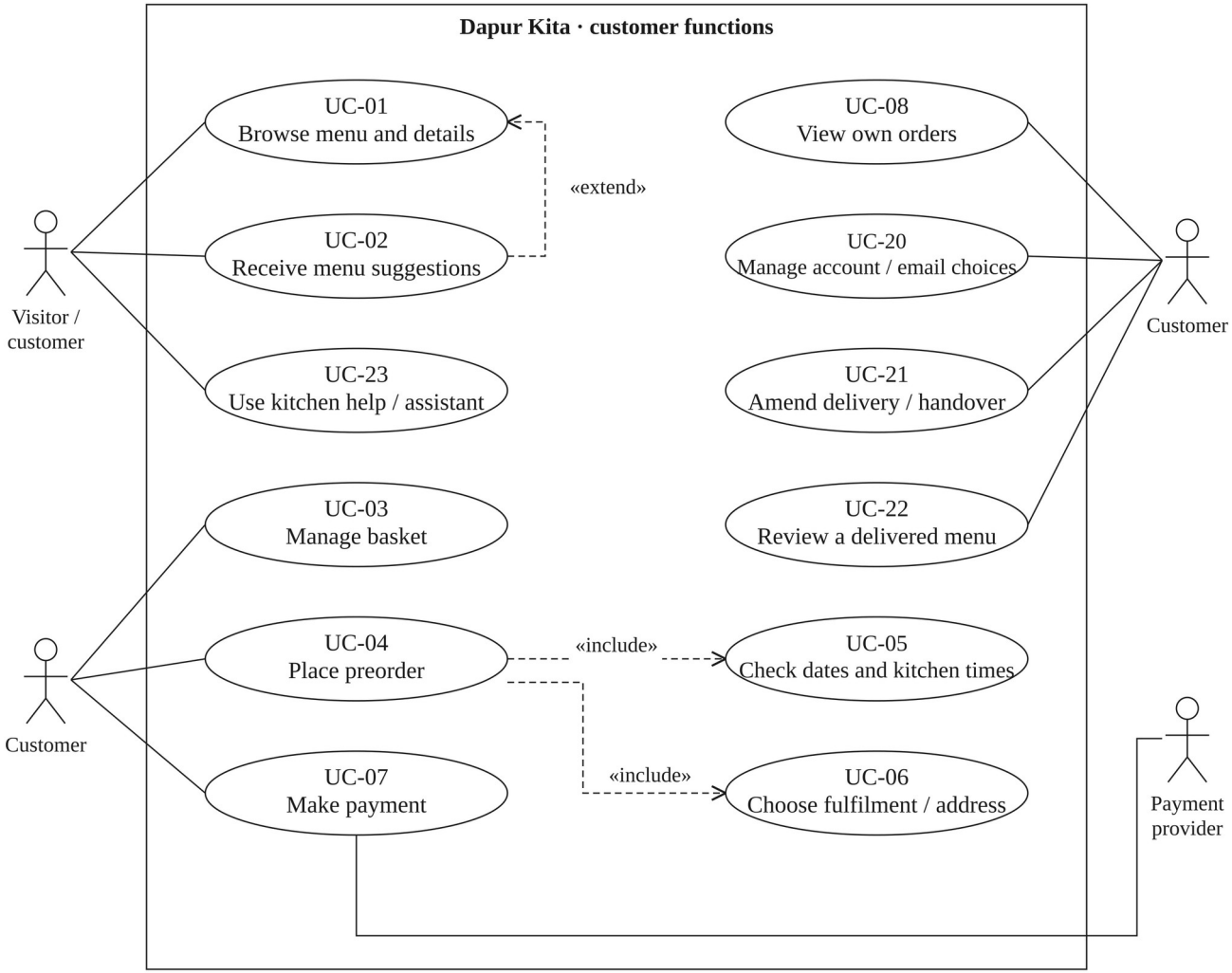
Table 3.1: Agile methodology.

Development increment	Implemented output	Verification evidence
Foundation and storefront	Customer accounts, categories, branded catalogue, basket and saved addresses	Authentication, permissions, cart and address tests
Preorder and operational planning	Accepted recipe/settings snapshots, resource scheduling, closures and replanning	Planning, fulfilment, multiday and concurrent-booking tests
Freshness and traceability	Evidence-based batch dates, holds, FEFO, consumption and waste records	Freshness, inventory audit and PostgreSQL stock-locking checks
Decision support	Sales/cost reports, recommendations, whole-order finder and FreshCast	Financial, recommendation, source separation and model API checks
External boundaries	PayPal Sandbox, order-bound Sepolia escrow and opt-in SMTP reminders	Verification, retry, ownership, worker and mocked-provider checks
Usability refinement	Compact food cards, daily handover queues, timed feedback and bounded history	Responsive Chromium checks and frontend regression suites

The Gantt chart in Chapter 1 remains the original proposal plan. This increment table describes implemented work and does not manufacture consultation dates or attendance. Supervisor consultations must be documented using the institution’s actual logbook.

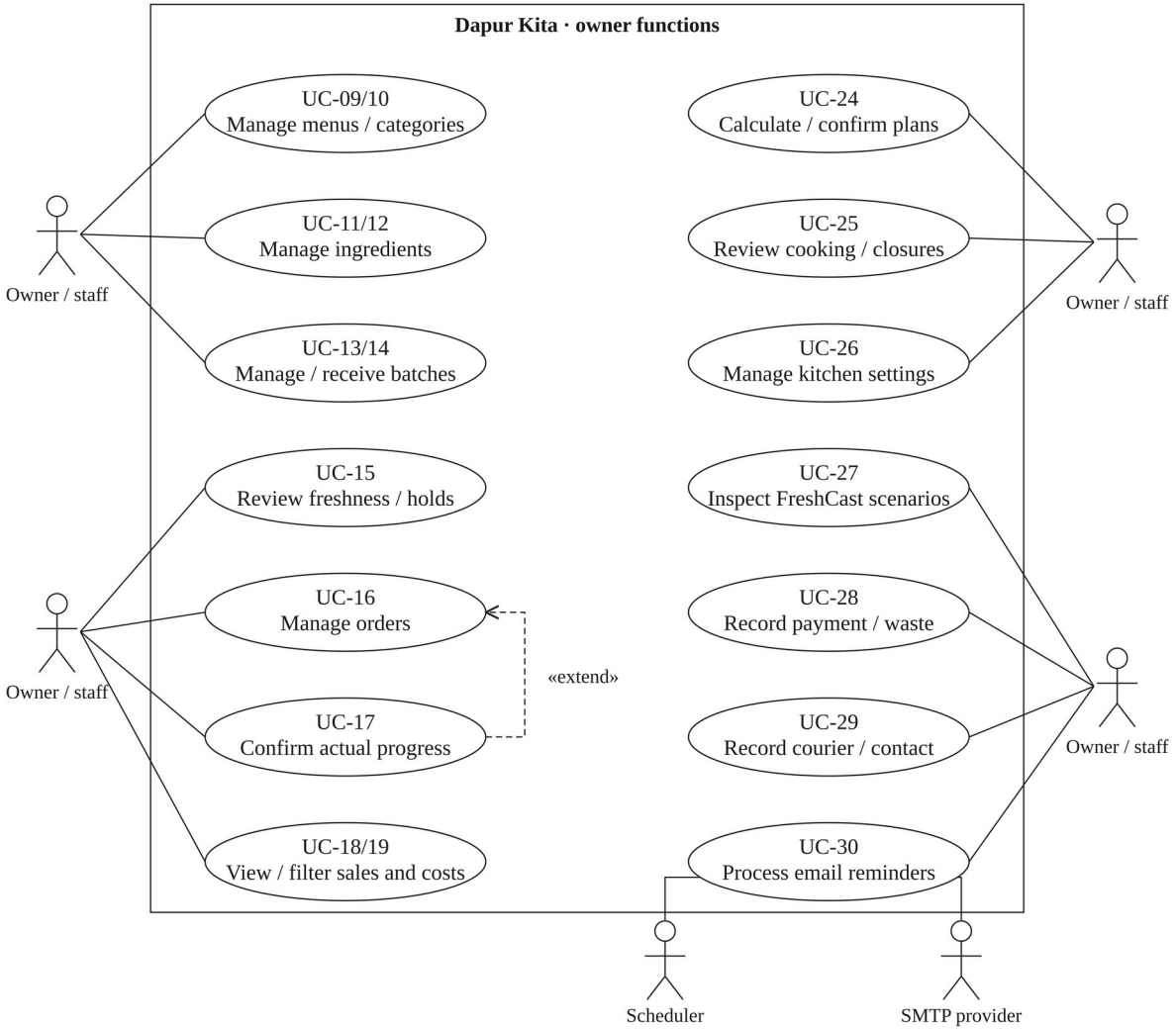
3.3 USE CASE DIAGRAMS

The two views below share one system boundary and divide it by user role for readability. Visitors can browse and request general suggestions without becoming staff. Customers authenticate to maintain a persistent basket, place orders and access their own records. The kitchen owner uses staff-authorised APIs to manage business data. Payment providers and the email scheduler are supporting external systems; the courier is coordinated outside the application.



Repeated customer / owner symbols identify the same role; they are repeated to keep associations readable.

Figure 3.1: Customer and visitor use cases.



Repeated customer / owner symbols identify the same role; they are repeated to keep associations readable.

Figure 3.2: Owner and integration use cases.

The include relationship means checkout always checks fulfilment and the requested date/time. Payment is a separate goal because an unconfirmed review request is saved unpaid and chooses payment only after a workable preparation plan is accepted. Recommendations are optional assistance to browsing. No courier API actor is drawn because the current application does not subscribe to verified courier events.

3.4 USE CASE SCENARIOS

The original UC-01 to UC-19 identifiers are retained so that earlier analysis can still be traced to the implementation. Their conditions and outcomes are corrected where the system has evolved. UC-20 to UC-30 cover the supporting features added during implementation. A saved request, a confirmed booking, a payment and an inventory deduction are different outcomes.

3.4.1 BROWSE MENU

Use case	Specification
Identifier / actor	UC-01 · Visitor / customer
Precondition	Public storefront is available.
Main flow	Open Menu; search or choose a category; inspect a product’s unit, price and ordering rules.
Alternative / exception	No active match shows a useful empty state; shipping-enquiry offers lead to contact instead of checkout.
Postcondition	Catalogue view only; no booking or payment.

3.4.2 GET RECOMMENDATION

Use case	Specification
Identifier / actor	UC-02 · Visitor / customer
Precondition	Active candidate menus and recipes exist; history is optional.
Main flow	View immediate cards or enter a date, total items, mix/same choice and optional budget; review up to three alternatives and reasons.
Alternative / exception	No history uses menu/popularity evidence; missing budget removes the price limit; infeasible or unmatched options explain recovery.
Postcondition	RecommendationSession records context; nothing is reserved.

3.4.3 ADD TO CART

Use case	Specification
Identifier / actor	UC-03 · Authenticated customer
Precondition	Selected local-order product is accepting orders.
Main flow	Choose Add or change quantity; server validates the owned line and quantity; refresh basket and show timed confirmation.
Alternative / exception	Read failure after a confirmed write explains it was saved; invalid/enquiry/archived additions are rejected.
Postcondition	Durable owned basket changes; no kitchen or stock reservation.

3.4.4 CHECKOUT

Use case	Specification
Identifier / actor	UC-04 · Authenticated customer
Precondition	Basket can be read; required fulfilment/contact choices are supplied.
Main flow	Choose handover time; review server quote; select an available payment method; place using a unique checkout request ID.
Alternative / exception	Stale state is rechecked; eligible explicit review requests remain deferred/unpaid; invalid sale rules cannot be bypassed.
Postcondition	One saved order with accepted snapshots or no change on rejected placement.

3.4.5 SELECT PREORDER DATES

Use case	Specification
Identifier / actor	UC-05 · Customer
Precondition	Basket has locally orderable products.
Main flow	Choose future date; inspect common weekdays/cutoffs and feasible whole-basket times; search a later day if needed.
Alternative / exception	No fit distinguishes notice, quantity or resource limits; a preview does not hold the time.
Postcondition	Selected request date/time passed to checkout for revalidation.

3.4.6 SELECT ADDRESS AND FULFILMENT

Use case	Specification
Identifier / actor	UC-06 · Authenticated customer
Precondition	Owner has at least one configured service method.
Main flow	Choose enabled delivery or collection; use an owned saved address for delivery or review public pickup location/hours.
Alternative / exception	Foreign addresses and disabled/incomplete methods are rejected; collection requires no delivery address or fee.
Postcondition	Accepted fulfilment/address copied into the order at placement.

3.4.7 MAKE PAYMENT

Use case	Specification
Identifier / actor	UC-07 · Customer; supporting provider / owner
Precondition	Saved order has confirmed preparation and an eligible payment choice.
Main flow	Choose configured manual transfer, PayPal or escrow; use the bound instructions/provider flow; await verified receipt.
Alternative / exception	Cash is collection-only; provider timeout stays recoverable; test funds stay disclosed; unconfirmed requests cannot be charged.
Postcondition	Payment ledger/status reflects evidence independently of kitchen progress.

3.4.8 VIEW ORDER

Use case	Specification
Identifier / actor	UC-08 · Authenticated customer
Precondition	Own orders may exist.
Main flow	Open Current/History/All; read at most ten rows; load older records; inspect handover, progress and payment.
Alternative / exception	No orders shows empty state; failed refresh hides stale actions; other users' rows/counts are excluded.
Postcondition	Read-only private history unless a separate authorised action is chosen.

3.4.9 MANAGE MENU

Use case	Specification
Identifier / actor	UC-09 · Owner / staff
Precondition	Staff session and current product version/conditions.
Main flow	Search/filter menus; edit identity, category, price, photo, recipe, selling and preparation rules; save.
Alternative / exception	Pause/archive stops new ordering; deletion follows history/commitment guards; validation remains in the form.
Postcondition	Current catalogue changes; old accepted order snapshots are retained.

3.4.10 CREATE MENU

Use case	Specification
Identifier / actor	UC-10 · Owner / staff
Precondition	Valid category, price and required menu data are entered.
Main flow	Create product; enter unit, optional content/size, recipe and preparation settings; upload or link an image; save.
Alternative / exception	Missing/invalid fields return associated errors; image content must pass upload validation.
Postcondition	A new menu exists; active ordering still depends on its saved rules.

3.4.11 MANAGE RAW MATERIAL

Use case	Specification
Identifier / actor	UC-11 · Owner / staff
Precondition	Ingredient directory exists.
Main flow	Review unit/cost and optional storage guidance; edit documented references without changing historical batches.
Alternative / exception	Referenced materials are protected; invalid/ambiguous guidance is rejected.
Postcondition	Directory updated; accepted ingredient and batch evidence remain traceable.

3.4.12 CREATE RAW MATERIAL

Use case	Specification
Identifier / actor	UC-12 · Owner / staff
Precondition	A distinct name and supported native unit are available.
Main flow	Enter ingredient name/unit, optional estimated unit cost and optional condition-specific references; save.
Alternative / exception	Duplicate name, invalid unit or invalid guidance is rejected; references are not compulsory.
Postcondition	Material is available for recipes and batches.

3.4.13 MANAGE INVENTORY

Use case	Specification
Identifier / actor	UC-13 · Owner / staff
Precondition	Received batches exist.
Main flow	Search/filter lots; inspect date/hold/cost evidence; edit with reason and current version or use the supported disposal/deletion action.
Alternative / exception	Stale version or protected history prevents unsafe overwrite/deletion; quantity cannot become negative.
Postcondition	Validated stock/audit change or unchanged data on failure.

3.4.14 CREATE INVENTORY

Use case	Specification
Identifier / actor	UC-14 · Owner / staff
Precondition	Material and actual storage are known.
Main flow	Enter quantity, physical location and receipt date; choose printed/manufacture/receipt evidence; review applicable limits; save.
Alternative / exception	Unlabelled food uses custom or explicitly copied documented guidance; incomplete dates/durations or inconsistent evidence are rejected.
Postcondition	Dated batch and audit recorded without inventing safe shelf life.

3.4.15 CHECK FRESHNESS STATUS

Use case	Specification
Identifier / actor	UC-15 · Owner / staff; limited public summary
Precondition	Batch has recorded evidence.
Main flow	Compare effective deadline, handling hold and Malaysia date; inspect remaining-window information and unknowns.
Alternative / exception	Unknown evidence is labelled; held/expired lots are excluded from cooking regardless of a public summary.
Postcondition	Monitoring view; no physical-safety certification or automatic disposal.

3.4.16 MANAGE ORDERS

Use case	Specification
Identifier / actor	UC-16 · Owner / staff
Precondition	Staff-authorised order list.
Main flow	Open dated/confirmation/financial queues; inspect accepted items, plan, customer and payment evidence.
Alternative / exception	Missing/stale plans and past-due events go to follow-up; history is not silently renumbered.
Postcondition	Order is reviewed; separate actions change its state.

3.4.17 CHANGE ORDER STATUS

Use case	Specification
Identifier / actor	UC-17 · Owner / staff
Precondition	Order transition and actual event are valid; selected versions are current.
Main flow	Optionally record preparation started; confirm ready; confirm actual dispatch or delivered/collected; group up to twenty eligible records if useful.
Alternative / exception	Ready needs a confirmed plan and sufficient stock; failure rolls back group; collection cannot be dispatched.
Postcondition	Actual stage saved; ready deducts once; payment unchanged.

3.4.18 VIEW SALES REPORT

Use case	Specification
Identifier / actor	UC-18 · Owner / staff
Precondition	Recorded sales/cost evidence may exist.
Main flow	Open Sales; choose daily/menu/financial/contribution views; inspect definitions, gaps and optional exports.
Alternative / exception	Missing consumption/cost produces unavailable contribution, not zero or fabricated profit.
Postcondition	Read-only report, except explicit separate expense/waste actions.

3.4.19 FILTER RESULTS

Use case	Specification
Identifier / actor	UC-19 · Owner / staff
Precondition	Supported report/date filters are selected.
Main flow	Apply ordered dates/category/view; server validates bounds; inspect scoped totals; download supported CSV if needed.
Alternative / exception	Oversized or invalid export range is rejected; report and export scopes are stated.
Postcondition	Filtered evidence; source transactions unchanged.

3.4.20 MANAGE ACCOUNT AND EMAIL

Use case	Specification
Identifier / actor	UC-20 · Customer
Precondition	Own authenticated account; recovery can begin publicly.
Main flow	Manage profile/contact or request a verified email change; choose per-order email updates.
Alternative / exception	Current password/confirmation and rate limits apply; stale recipient consent is not transferred silently.
Postcondition	Own account/preferences updated through guarded paths.

3.4.21 AMEND DELIVERY OR HANDOVER

Use case	Specification
Identifier / actor	UC-21 · Customer / owner
Precondition	Order is eligible and not constrained by started work or a fixed escrow invoice.
Main flow	Review a new future time/window and reason; preview current plan effects; confirm before expiry.
Alternative / exception	Changed version/availability rejects preview; unscheduled requests stay unconfirmed unless a plan is accepted.
Postcondition	OrderAmendment and current arrangement saved; no invented courier event.

3.4.22 REVIEW DELIVERED MENU

Use case	Specification
Identifier / actor	UC-22 · Authenticated customer
Precondition	An eligible delivered order contains this menu; no existing review for user/product.
Main flow	Choose bounded rating and comment; submit; display recorded feedback.
Alternative / exception	Ineligible purchase, duplicate review or invalid input is rejected.
Postcondition	One genuine account review recorded, not synthetic endorsement.

3.4.23 USE KITCHEN HELP AND ORDERING ASSISTANT

Use case	Specification
Identifier / actor	UC-23 · Visitor / authenticated customer for basket actions
Precondition	Approved help topics are available.
Main flow	Ask a topic question or provide bounded order preferences; review grounded answers/proposals; explicitly confirm basket changes.
Alternative / exception	Optional AI failure uses local recovery; assistant cannot directly mark paid or query private orders arbitrarily.
Postcondition	Help answer or authorised basket proposal/action.

3.4.24 CALCULATE AND CONFIRM PREPARATION

Use case	Specification
Identifier / actor	UC-24 · Owner / staff
Precondition	Eligible pending order and a real future handover arrangement.
Main flow	Calculate from accepted settings; inspect steps/per-day work; explicitly adopt current terms if appropriate; confirm signed preview.
Alternative / exception	Stale preview, invalid time or capacity rejects; advanced manual arrangement requires separate review and reason.
Postcondition	Reserved preparation plan/amendment; stock unchanged.

3.4.25 MANAGE SHARED COOKING AND CLOSURES

Use case	Specification
Identifier / actor	UC-25 · Owner / staff
Precondition	Compatible future single-menu orders or a valid closure interval.
Main flow	Preview 2–10 compatible shared orders; confirm common tasks; separately record kitchen closures/breaks.
Alternative / exception	Incompatible/started orders cannot combine; changing a member can require separating all original members first.
Postcondition	Shared reservation counted once; order payments and ingredient use remain separate.

3.4.26 MANAGE STOREFRONT AND FULFILMENT

Use case	Specification
Identifier / actor	UC-26 · Owner / staff
Precondition	Current settings loaded.
Main flow	Edit branding, story, contact, cooking/handover hours, service methods, location and fee policy; save.
Alternative / exception	Failed save retains draft; incomplete pickup setup is not offered; accepted old fees/plans are not rewritten.
Postcondition	Current defaults changed for future operations.

3.4.27 REVIEW FRESHCAST AND SCENARIOS

Use case	Specification
Identifier / actor	UC-27 · Owner / staff
Precondition	Selected source has usable data or an explicit unavailable state.
Main flow	Choose kitchen/Genpact/illustrative source; trace dated requirements, stock and risk; compare a hypothetical action/purchase.
Alternative / exception	Data gaps/exclusions remain visible; no silent model/sample fallback and no unsupported ID mapping.
Postcondition	Read-only decision evidence and optional signed review snapshot.

3.4.28 RECORD FINANCIAL EVIDENCE OR WASTE

Use case	Specification
Identifier / actor	UC-28 · Owner / staff
Precondition	Actual external payment/refund or disposal evidence exists.
Main flow	Record authorised receipt/refund or waste/expense with request ID, amount/quantity and reason; inspect resulting ledger.
Alternative / exception	Provider-bound payment shortcuts, duplicate requests and stock overdraw are rejected or replayed safely.
Postcondition	Evidence/audit saved; waste changes stock, financial record does not imply delivery.

3.4.29 RECORD COURIER REFERENCE AND CONTACT

Use case	Specification
Identifier / actor	UC-29 · Owner / staff
Precondition	Non-cancelled delivery order and reviewed public reference/link.
Main flow	Book in external courier service; save carrier/reference/contact/link; use recipient/WhatsApp tools; confirm actual handover.
Alternative / exception	No verified courier API event exists; stale edits, collection tracking and invalid links are rejected.
Postcondition	Kitchen-entered tracking visible; no automatic courier booking or ETA.

3.4.30 PROCESS ORDER EMAIL REMINDERS

Use case	Specification
Identifier / actor	UC-30 · Authenticated scheduler; owner diagnostics
Precondition	Configured SMTP, worker secret and eligible event; customer consent where required.
Main flow	Invoke one shared worker; select and lock due events; attempt up to three messages; save accepted-send receipts and run totals.
Alternative / exception	Invalid authentication rejected; failed sends can retry; closed/unconfirmed/mismatched-email records are excluded.
Postcondition	Recorded attempts/receipts; no promise of inbox receipt or exact scheduler timing.

3.5 ACTIVITY DIAGRAMS

3.5.1 PREORDER CHECKOUT AND REQUEST REVIEW

The checkout activity includes both the automatically scheduled path and the explicit kitchen-review path. A future request can be recorded without a feasible automatic plan, but it must not be displayed as a reserved cooking slot or successful payment. Sale dates, cutoffs, future-date bounds and fulfilment rules are still validated. A customer must not be repeatedly asked to guess unavailable times: the optional basket preview and checkout search present feasible alternatives when available.

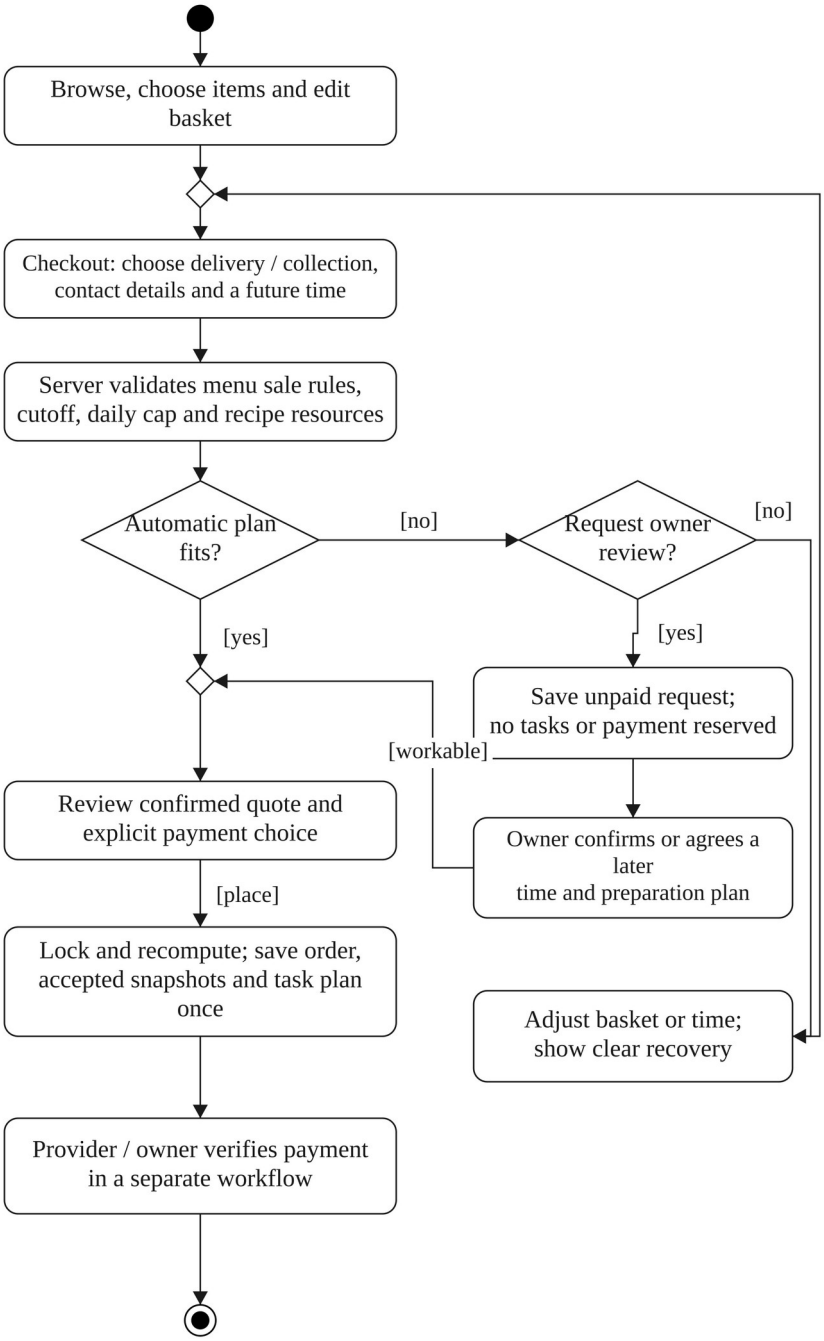


Figure 3.3: Preorder checkout with confirmed and unconfirmed paths.

3.5.2 BATCH DATES AND FRESHNESS

The owner chooses the evidence supporting the deadline rather than having the system invent a universal shelf life. Receipt date records when the batch reached the kitchen. Manufacture date records production; it is not the printed use-by date. A printed best-before date is a quality reference, while a use-by/expiry date is treated as a deadline. The application conservatively treats every recorded effective deadline as an allocation limit. Actual conditions and earlier opened/thawed limits can shorten it.

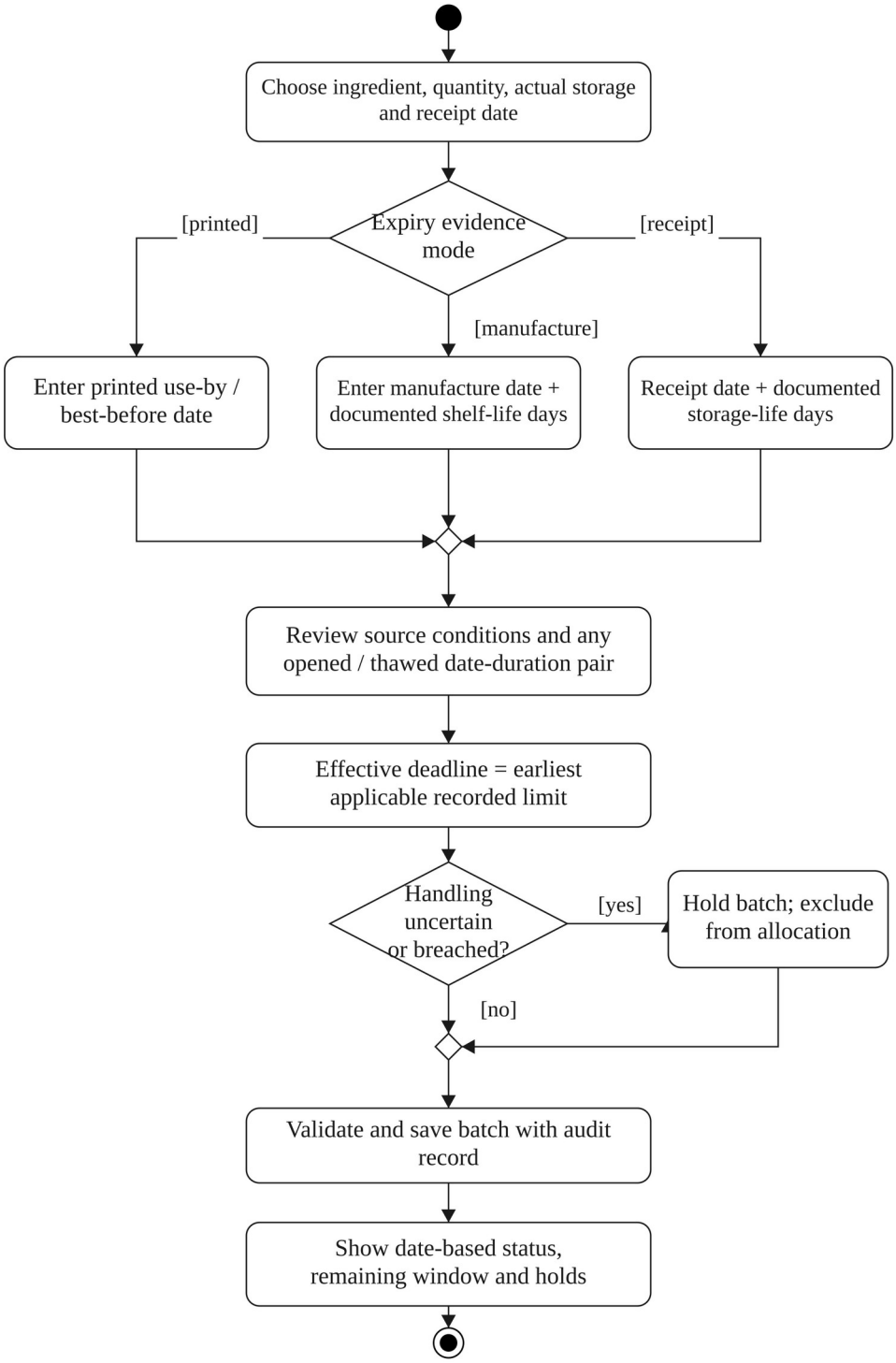


Figure 3.4: Inventory evidence, effective deadline and handling hold.

3.5.3 PREPARATION PLANNING AND REPLANNING

The scheduler calculates a start and end; the owner need not guess an automatic end time. Earlier preparation is permitted only within the accepted preparation span and documented step waits, pauses and finishing allowances. Review precedes confirmation. A changed order, closure or booking invalidates a stale preview rather than silently moving someone else’s work.

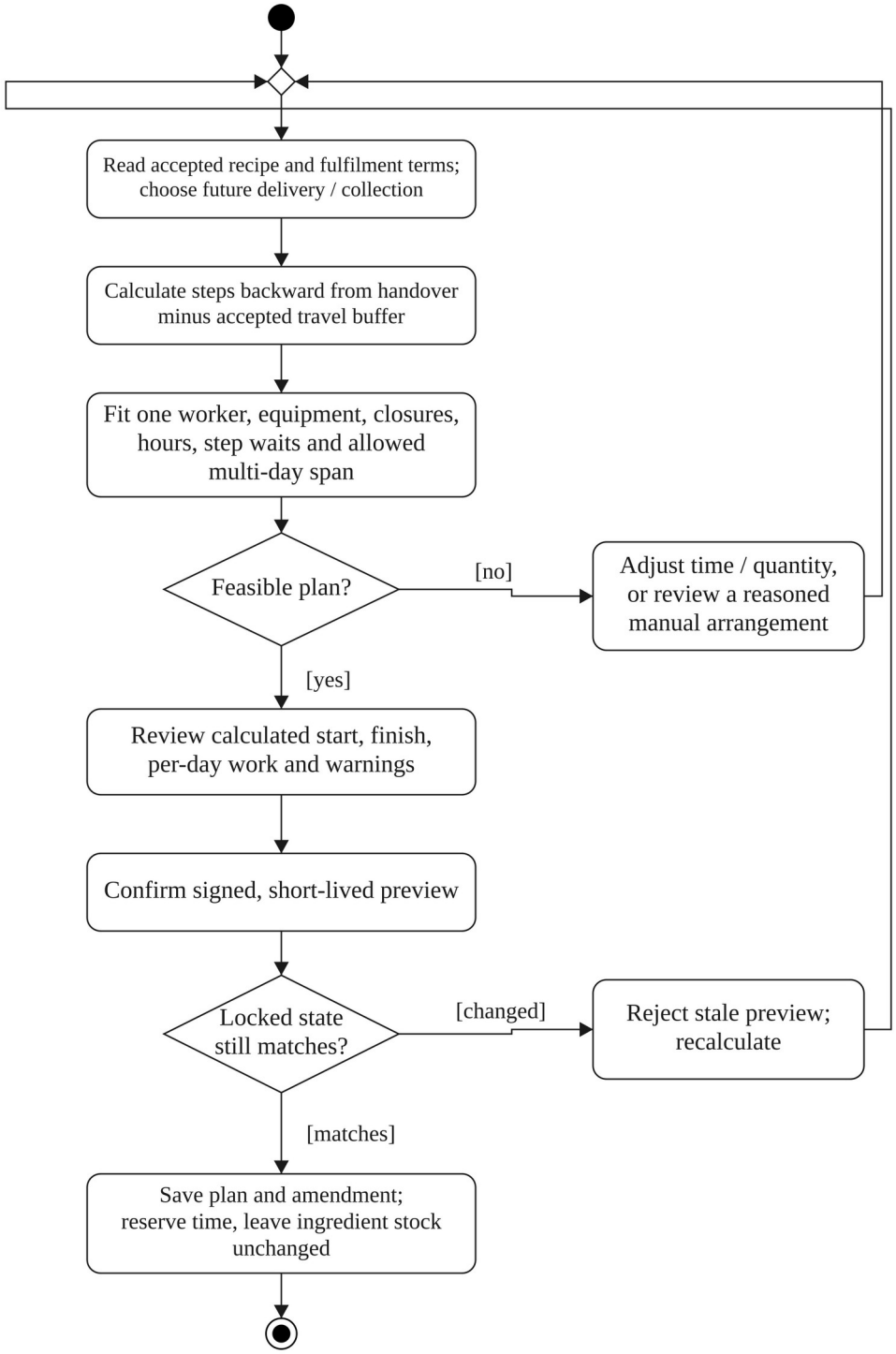


Figure 3.5: Calculation and confirmation of an order preparation plan.

3.5.4 ACTUAL PROGRESS AND HANDOVER

Preparing and out-for-delivery are optional intermediate updates. The owner confirms actual food readiness before collection or delivery. A grouped operation validates every selected order before applying any change, so a failed member cannot leave a partially updated group. Payment is recorded independently from physical handover.

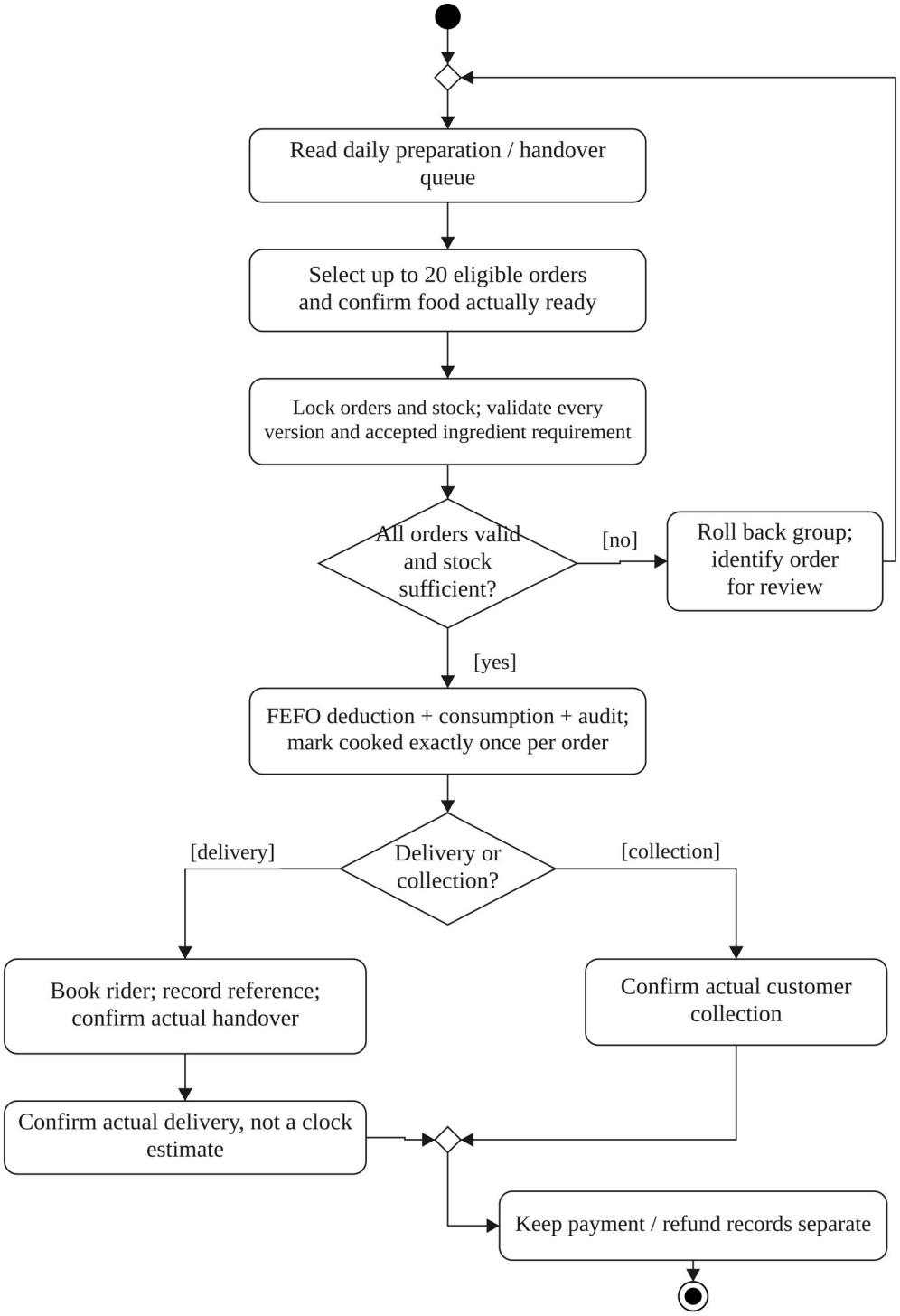


Figure 3.6: Grouped food-readiness and actual handover activity.

3.6 SEQUENCE DIAGRAMS

The earlier sequence diagrams are consolidated into related interactions to avoid duplicating identical list/create operations. Browse and recommendation correspond to UC-01/02; basket to UC-03; quote and placement to UC-04/05/06; payment to UC-07; paginated order reading to UC-08; protected menu/ingredient CRUD to UC-09–14; evidence and FEFO to UC-13–17; reporting to UC-18/19; and the remaining

interactions to UC-20–30. Each diagram is an implementation-level collaboration, not a claim that the browser can directly update database or payment state.

3.6.1 MENU DISCOVERY AND GUIDED RECOMMENDATION

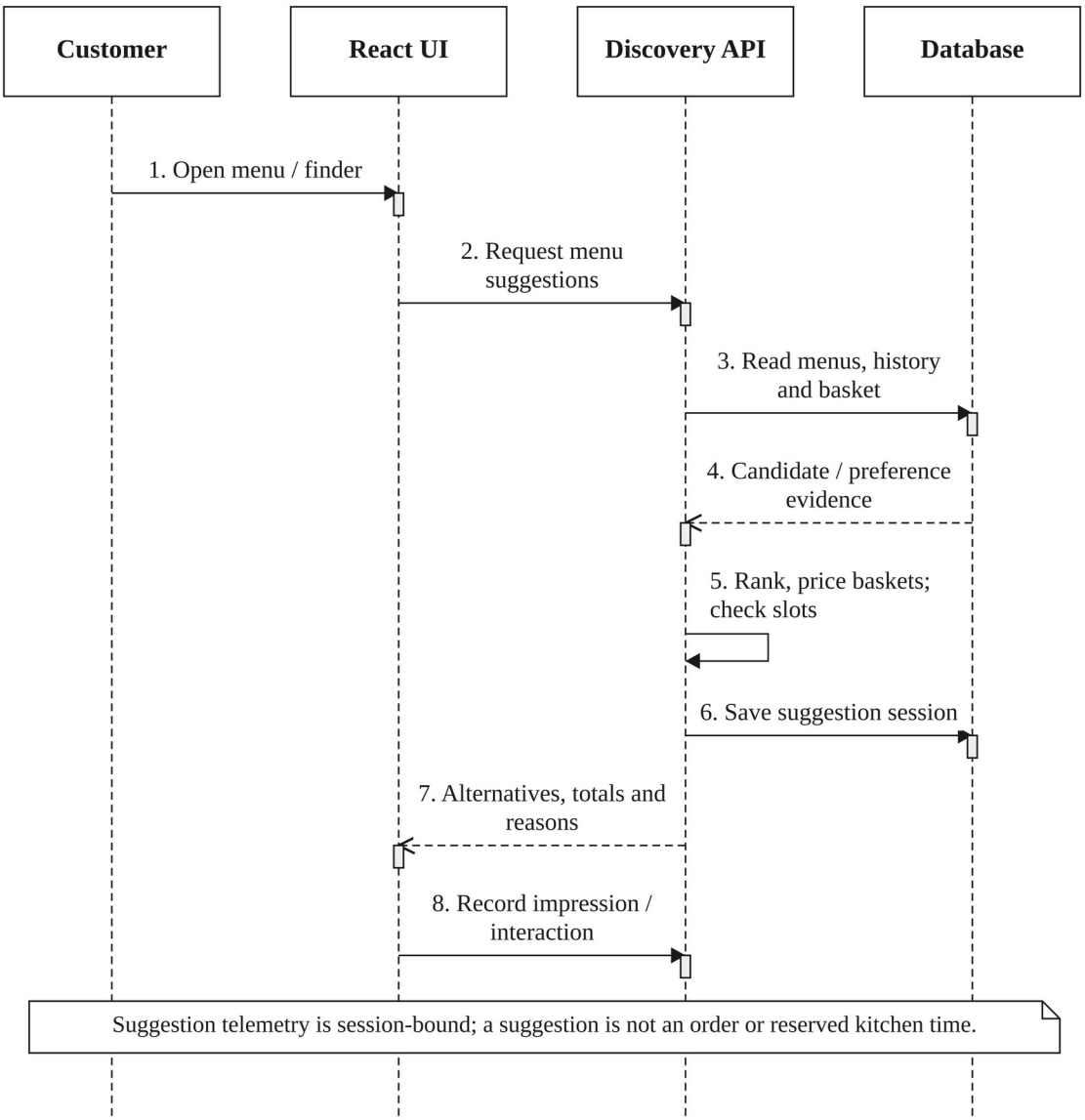


Figure 3.7: Explainable discovery and whole-order date guidance.

The server ranks eligible menu candidates and checks complete candidate baskets. A RecommendationSession binds displayed products and context to later events. Guest discovery is allowed; applying a durable basket change requires the authenticated customer and a fresh, matching search context. The final three suggestions are alternatives, not three orders automatically combined.

3.6.2 BASKET MUTATION AND FEEDBACK

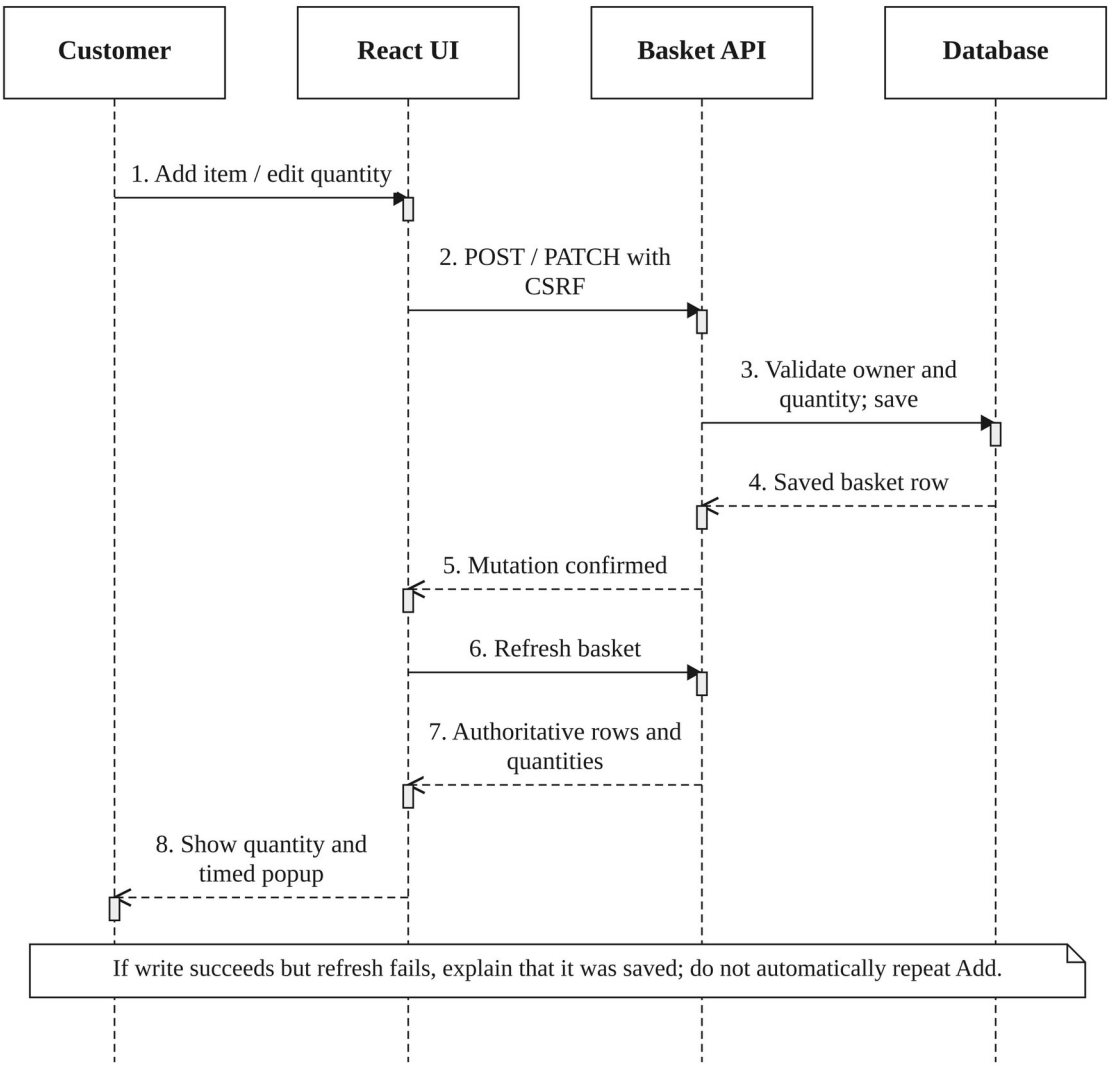


Figure 3.8: Basket mutation, refresh and timed confirmation.

A confirmed write followed by a failed read must not be described as a failed addition. The interface explains that the item was saved and offers a basket refresh, avoiding an unintended duplicate addition. Repeated quantity changes receive distinct notifications. Stale responses from a previous account session are discarded.

3.6.3 QUOTE AND ORDER PLACEMENT

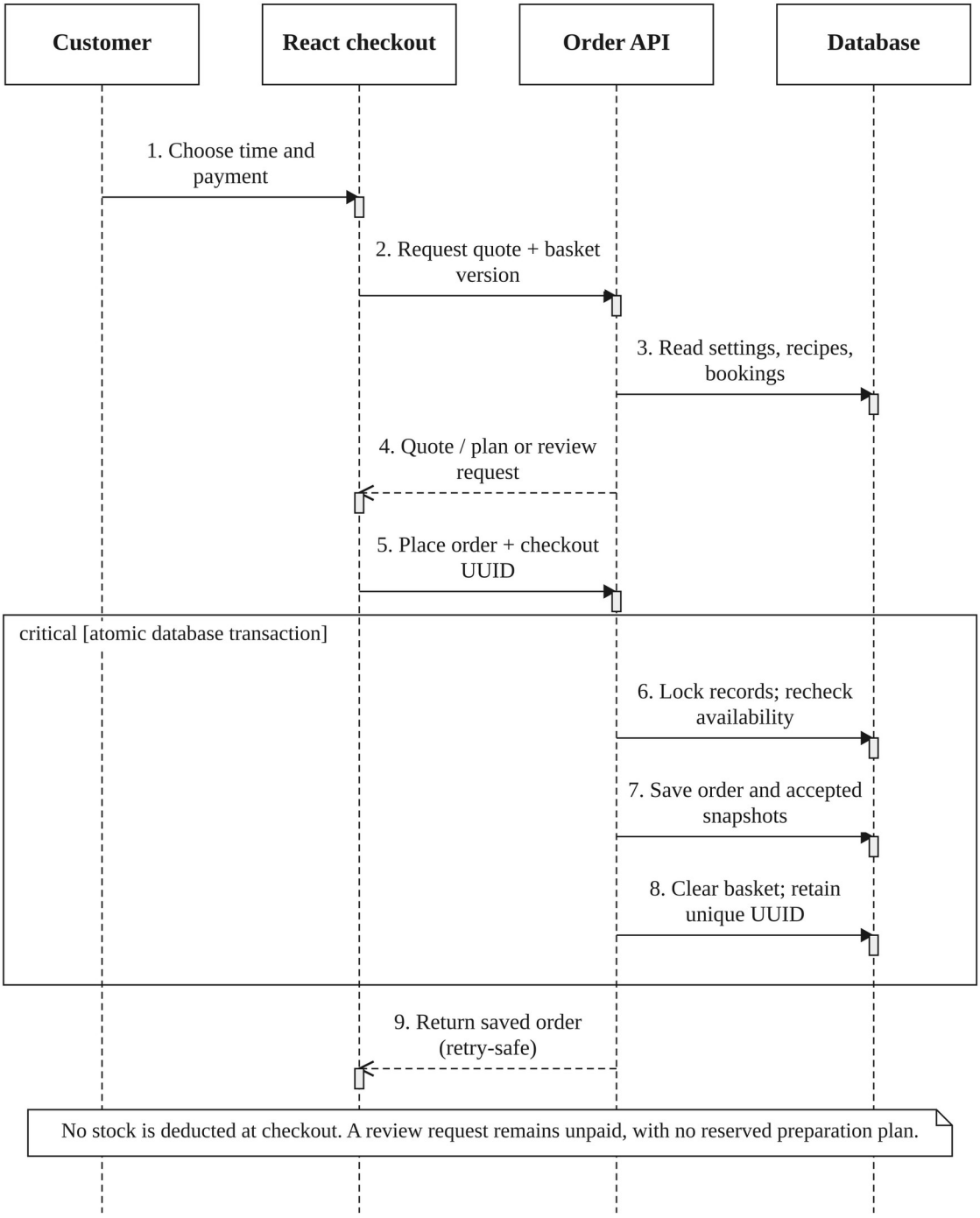


Figure 3.9: Authoritative quote and retry-safe placement.

Checkout prices, discounts and delivery fees are recomputed on the server. A unique checkout request identifier supports recovery after a lost response. The saved order retains line prices, names, accepted ingredient quantities, preparation settings, delivery-address details and fulfilment terms. The basket is not itself a reservation, and placement does not deduct ingredient stock.

3.6.4 PAYPAL PAYMENT VERIFICATION

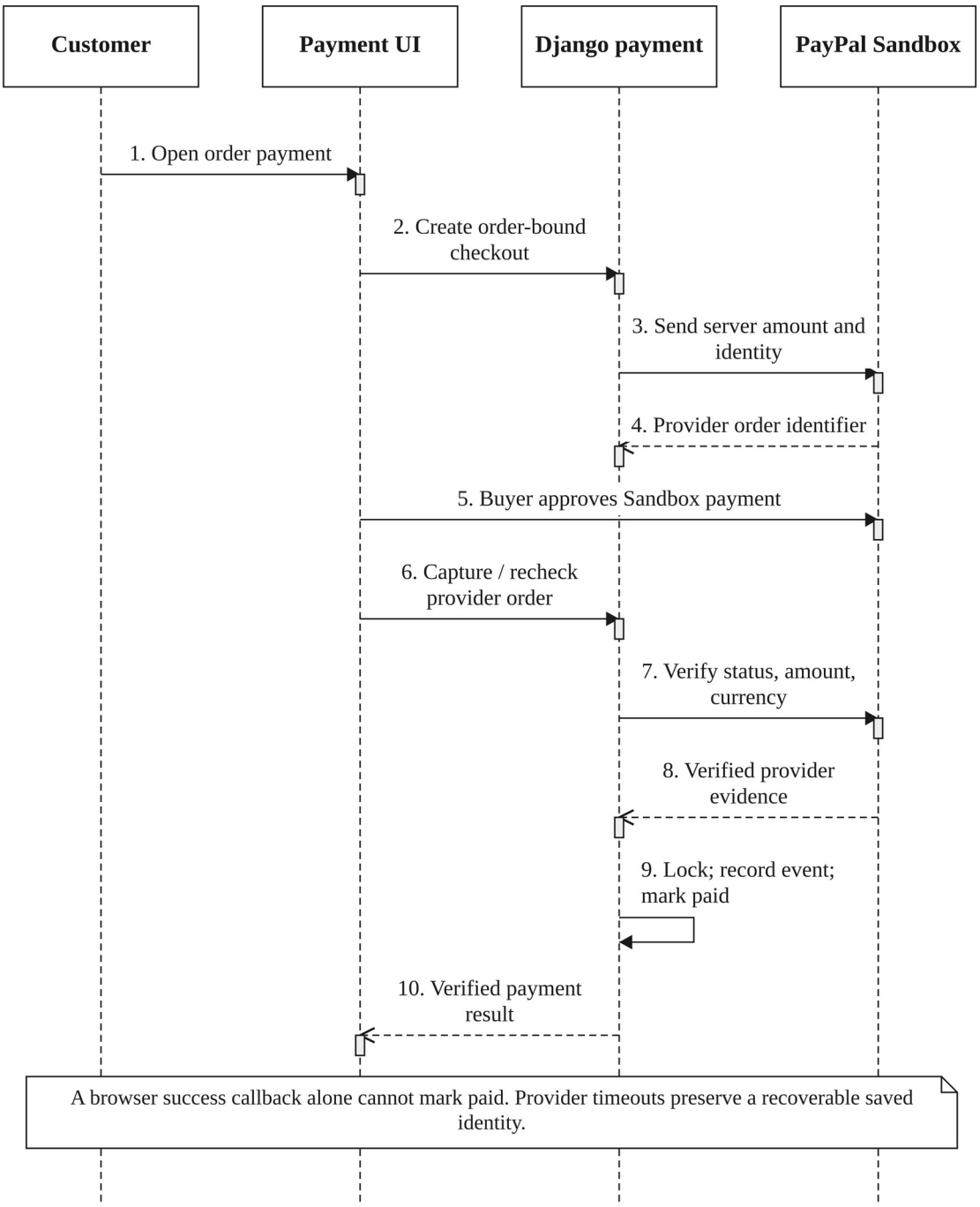


Figure 3.10: Server-verified PayPal Sandbox payment.

Provider confirmation must match the order and expected MYR amount. The displayed payment state comes from server verification and a durable ledger event, not a client-supplied paid flag. The main interface may say PayPal, but the test-funds disclosure remains necessary. The application does not represent Sandbox capture as real customer revenue.

3.6.5 PREPARATION PREVIEW AND CONFIRMATION

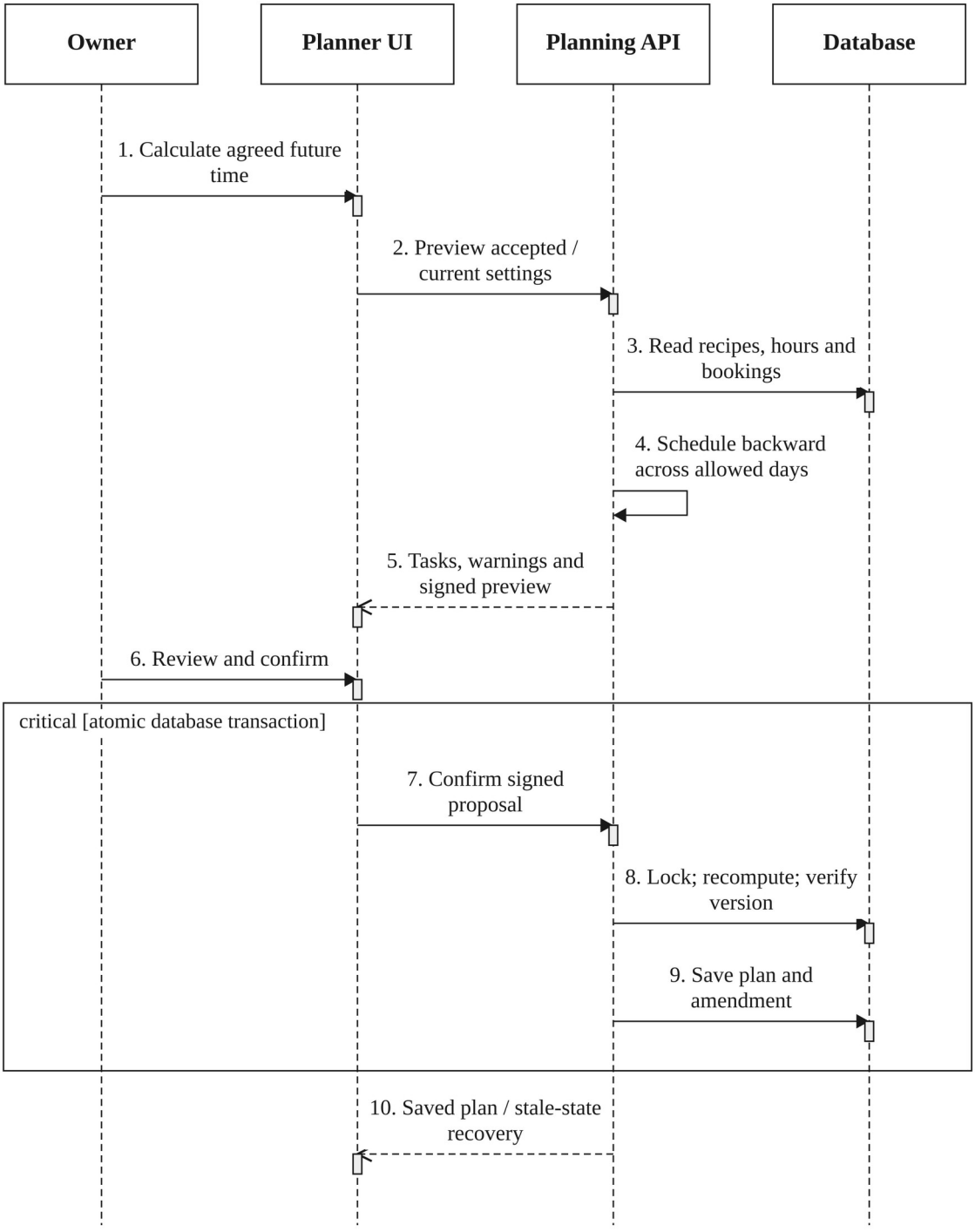


Figure 3.11: Replanning with a signed, expiring preview.

The same-delivery recalculation can use accepted settings; adopting current recipes/settings is explicit. When delivery is changed, the server rechecks the new arrangement. A preview is not a saved reservation. The owner confirms a proposal within its ten-minute validity period, and current locked state must still match.

3.6.6 INGREDIENT DEDUCTION AND GROUPED PROGRESS

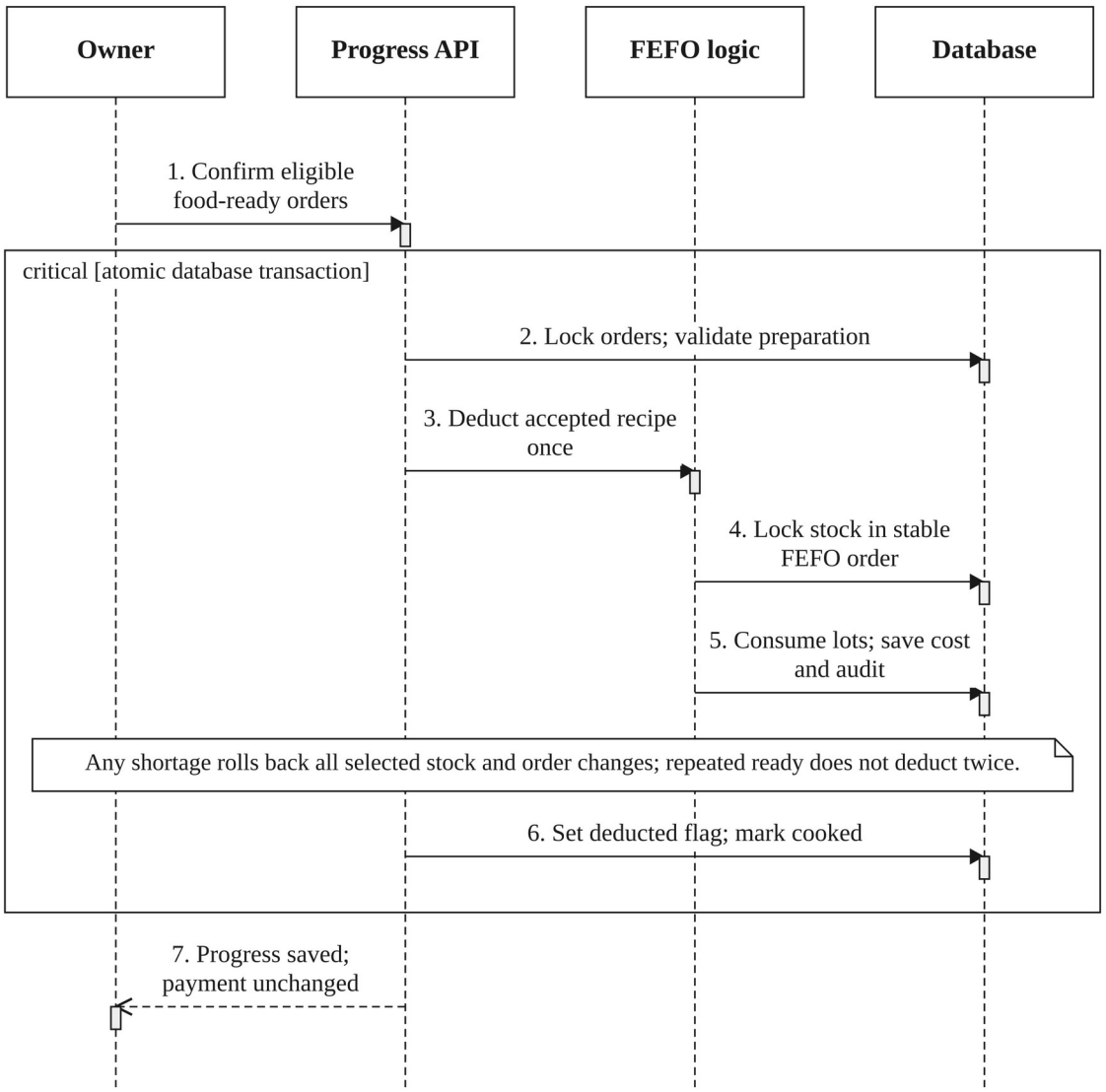


Figure 3.12: FEFO deduction and all-or-nothing progress updates.

The operation uses accepted OrderIngredient quantities. It locks the union of relevant stock rows in a consistent material/expiry/receipt/ID order and consumes eligible batches by earliest expiry. IngredientConsumption records carry batch cost and identity evidence; InventoryLog records quantity changes. Any shortage aborts the transaction. Repeating food-ready after inventory_deducted is true does not consume the ingredients again.

3.6.7 INVENTORY EVIDENCE AND VERSION CONFLICTS

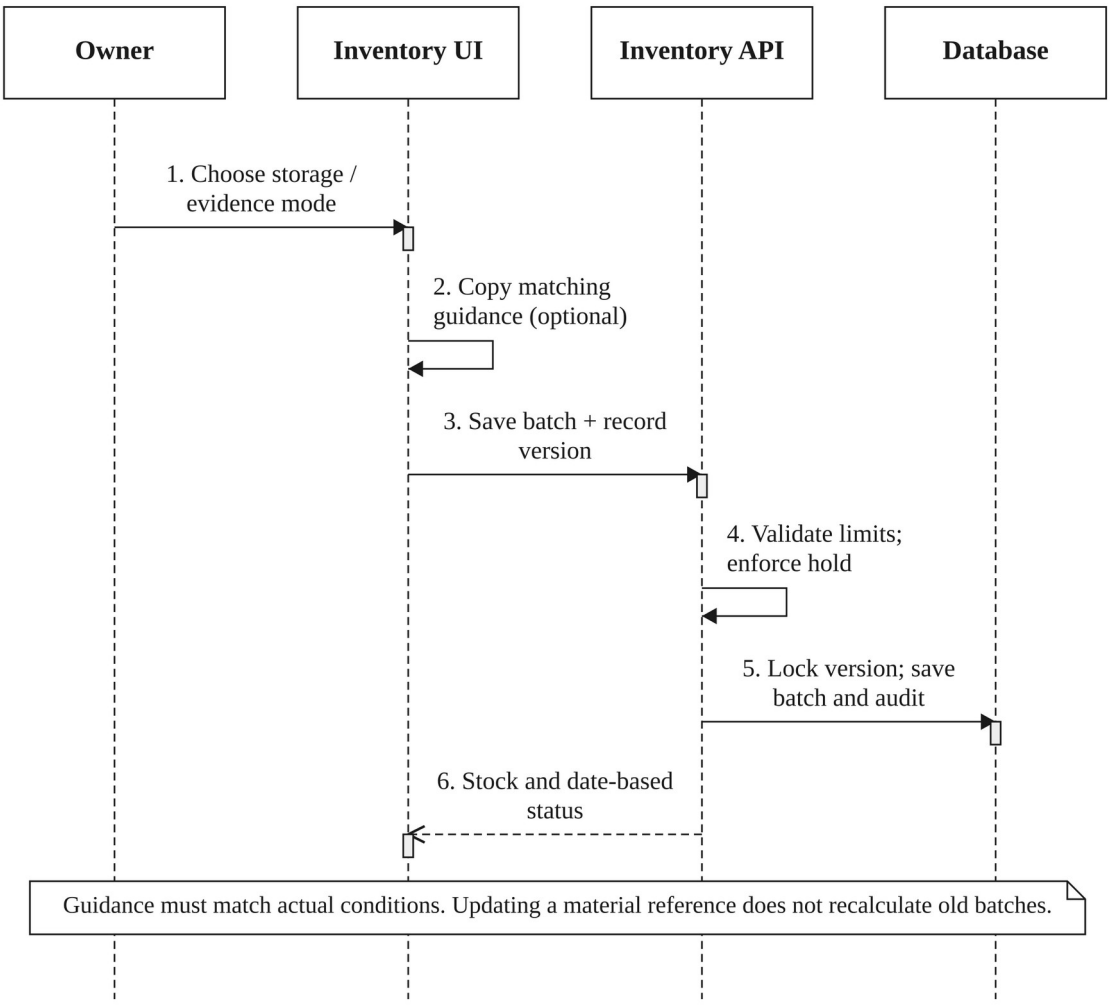


Figure 3.13: Recording batch evidence without overwriting a newer edit.

An optimistic edit version prevents a previously opened form from restoring stock that has since been consumed or adjusted. Matching material guidance is copied into the unsaved form only after owner selection. Its subsequent amendment does not retrospectively change older batches.

3.6.8 SALES AND FINANCIAL EVIDENCE

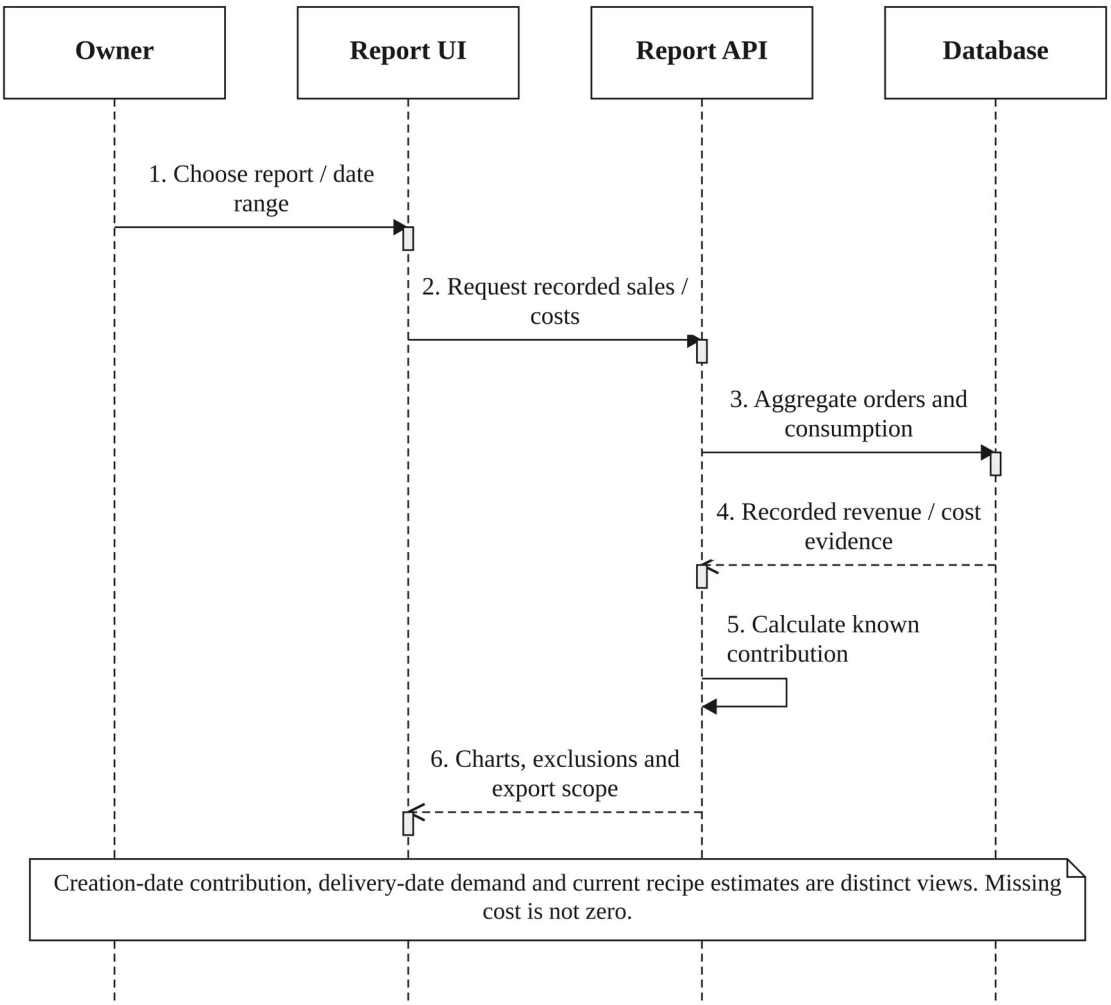


Figure 3.14: Sales and recorded contribution reporting.

The report response includes definitions, unavailable values and exclusions. A price scenario estimates costs using current recipes, while recorded contribution needs actual consumption and accepted packaging evidence. CSV export checks its date range and size and escapes spreadsheet-formula text.

3.6.9 FRESHCAST SOURCE SEPARATION

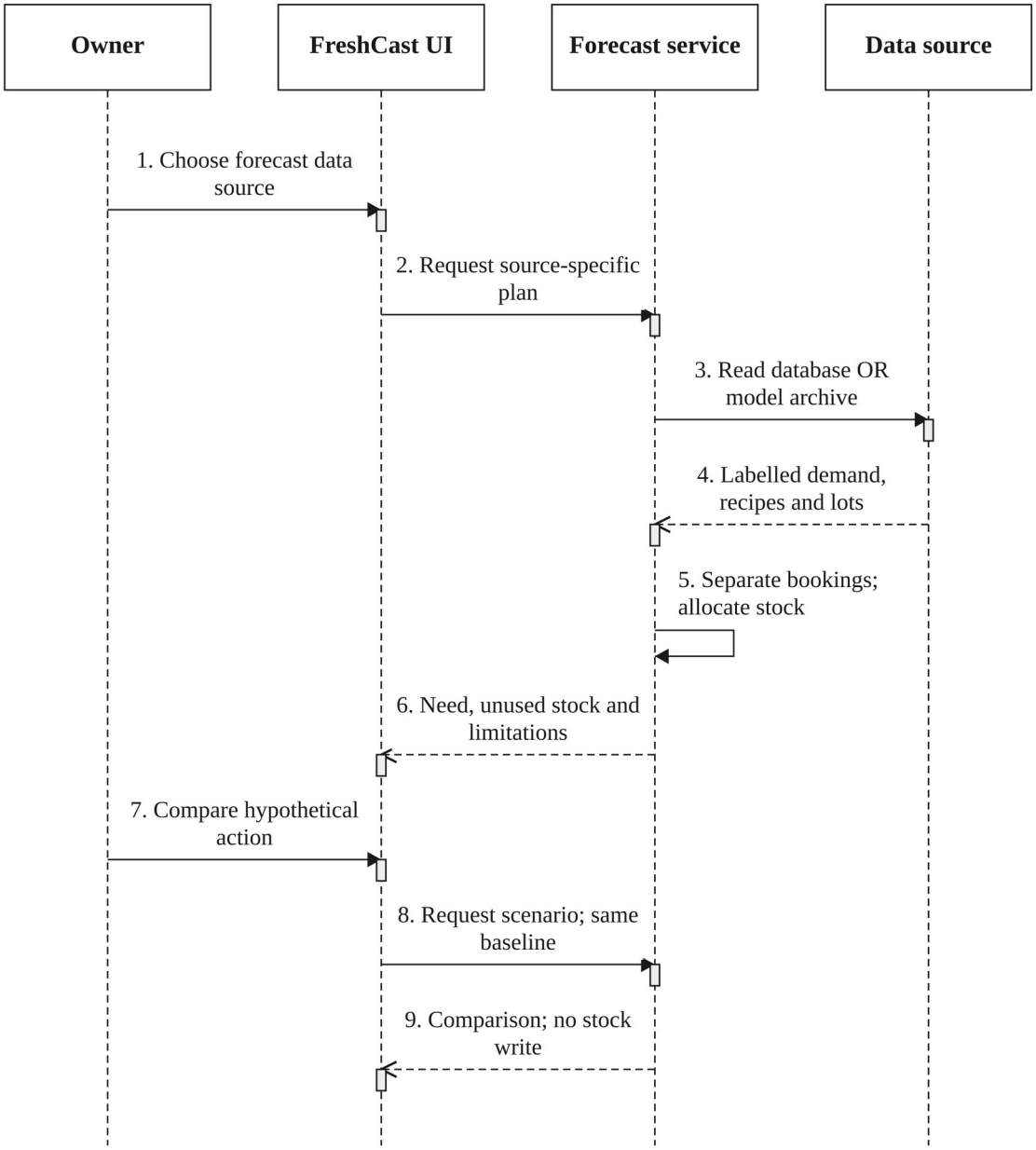


Figure 3.15: Source-labelled forecasts and read-only scenarios.

The database kitchen path, external Genpact path and fixed illustrative walkthrough must not be silently interchanged. Genpact meal IDs are not local product IDs. The kitchen decision view adjusts speculative demand for known commitments before allocating remaining eligible stock. Scenario comparisons do not purchase, dispose of or deduct stock.

3.6.10 EMAIL REMINDERS

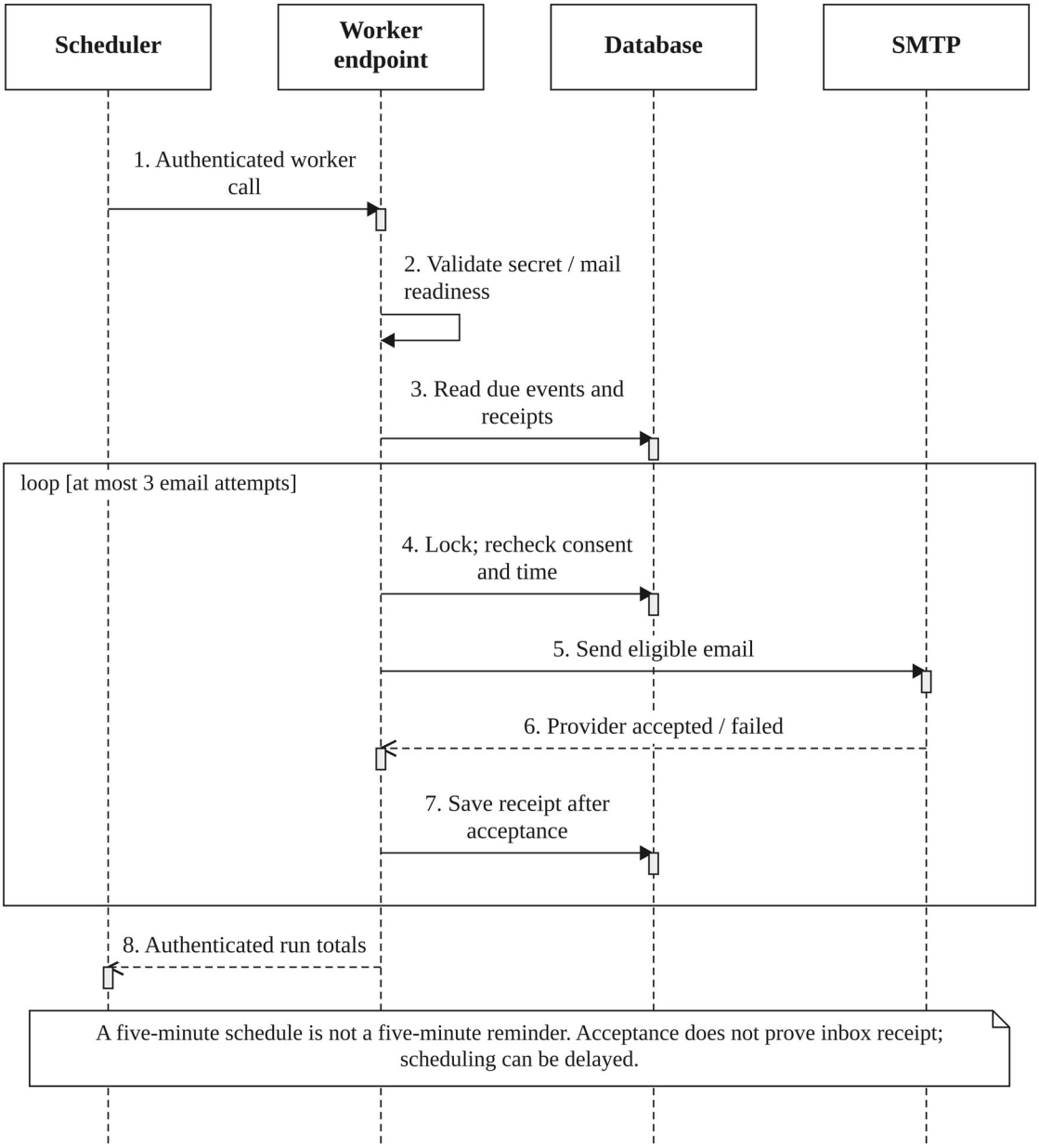


Figure 3.16: One authenticated worker for owner and opted-in customer emails.

Customer advance reminders require a valid account email, per-order consent and confirmed preparation. Separate choices control dispatch and agreed-time updates. Event receipts suppress routine duplicates, while provider acceptance is distinct from inbox delivery. Phone numbers support contact and WhatsApp drafts; they are not SMTP recipients or an automatic WhatsApp subscription.

3.6.11 ORDER-BOUND ESCROW

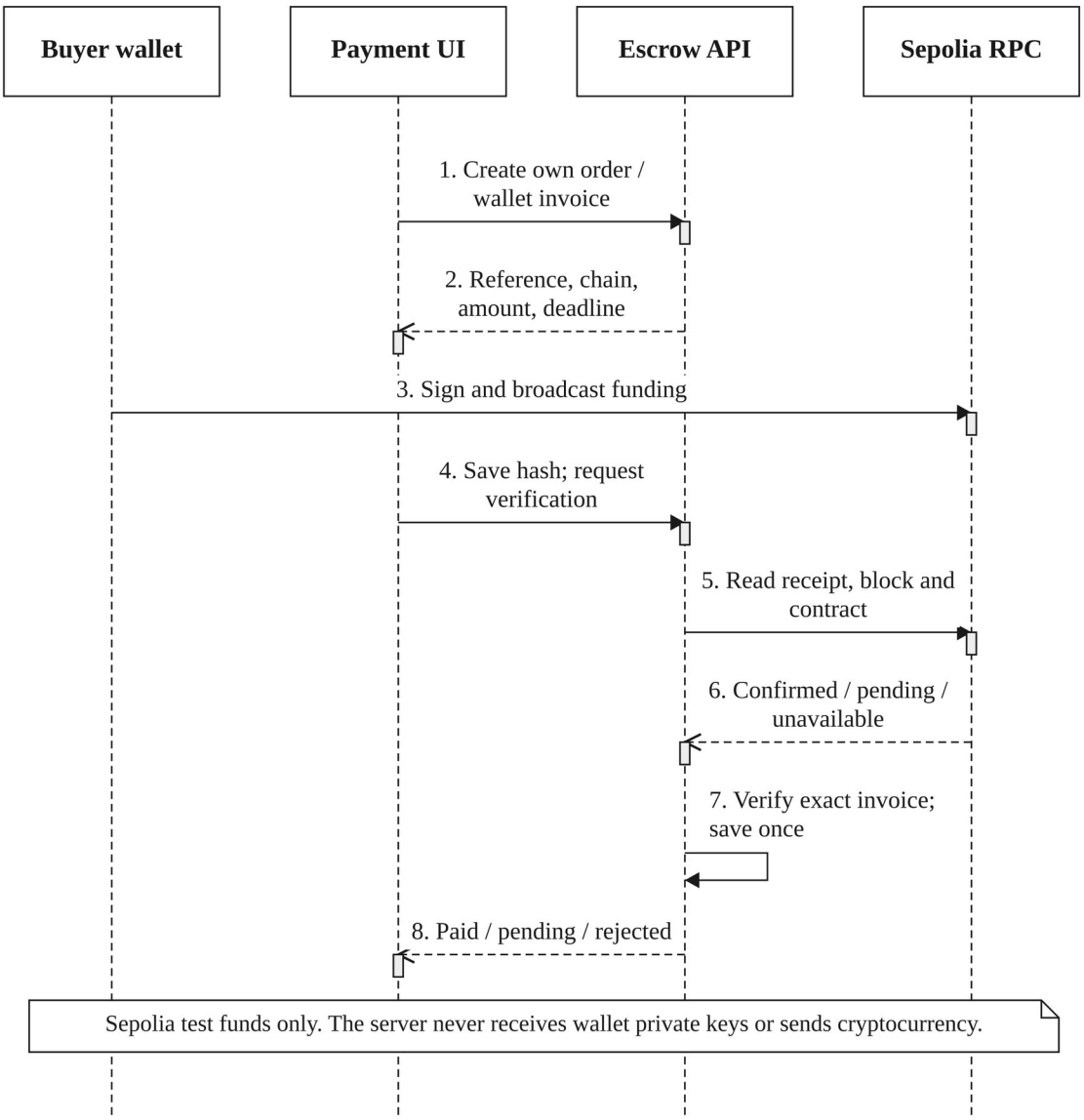
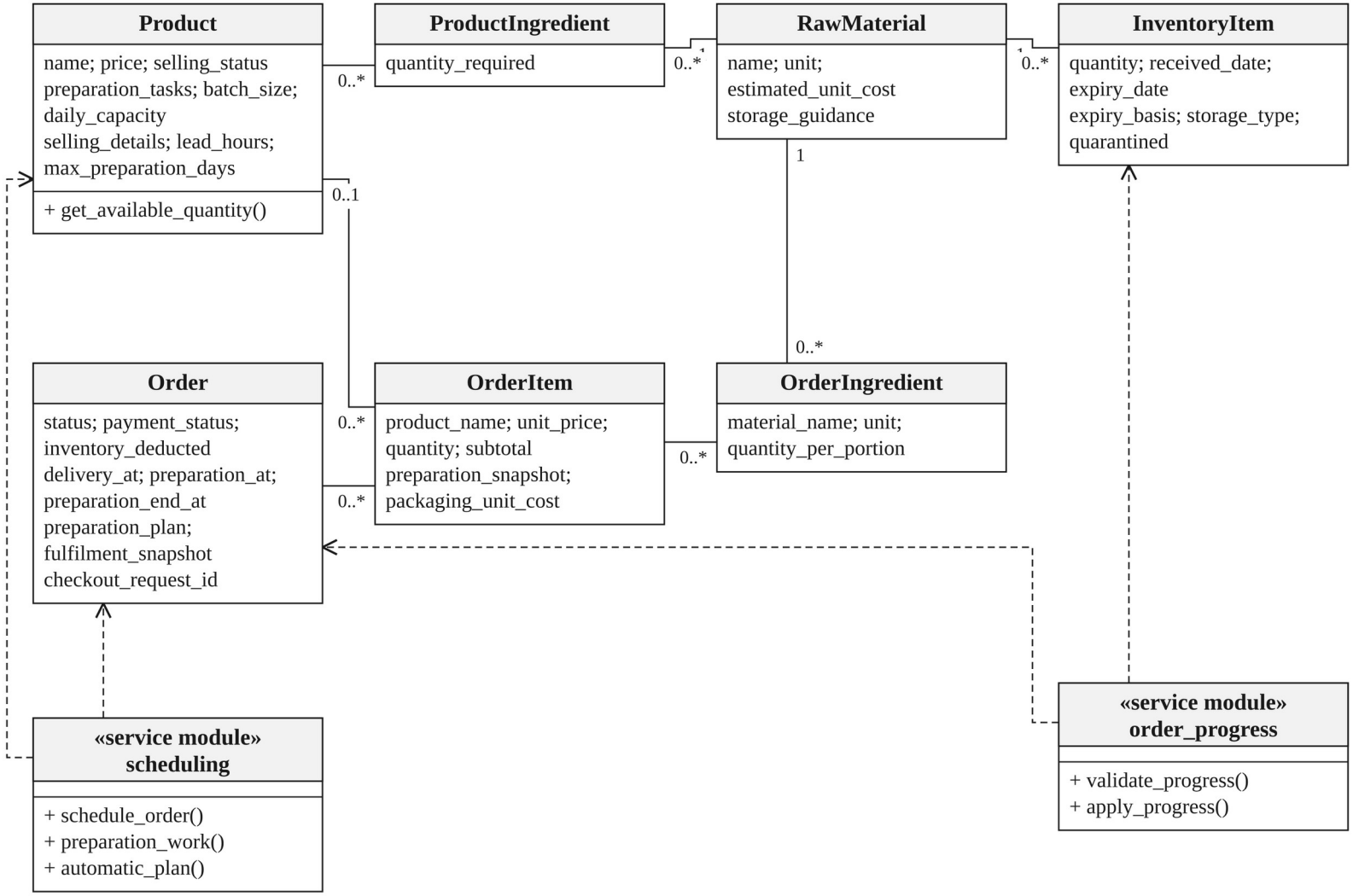


Figure 3.17: Sepolia escrow invoice and independent funding verification.

The invoice fixes chain, contract, participants, value, reference and dispatch deadline. The browser wallet signs the transaction. Django stores the submitted hash for recovery and independently verifies canonical receipt, transaction, event, runtime code and confirmed contract state. An unpaid or pending transaction cannot be promoted to paid by a browser message. Escrow state does not autonomously mark food prepared or delivered.

3.7 CLASS DIAGRAM



Solid lines: model associations with multiplicities. Dashed arrows: service dependencies (not database tables).

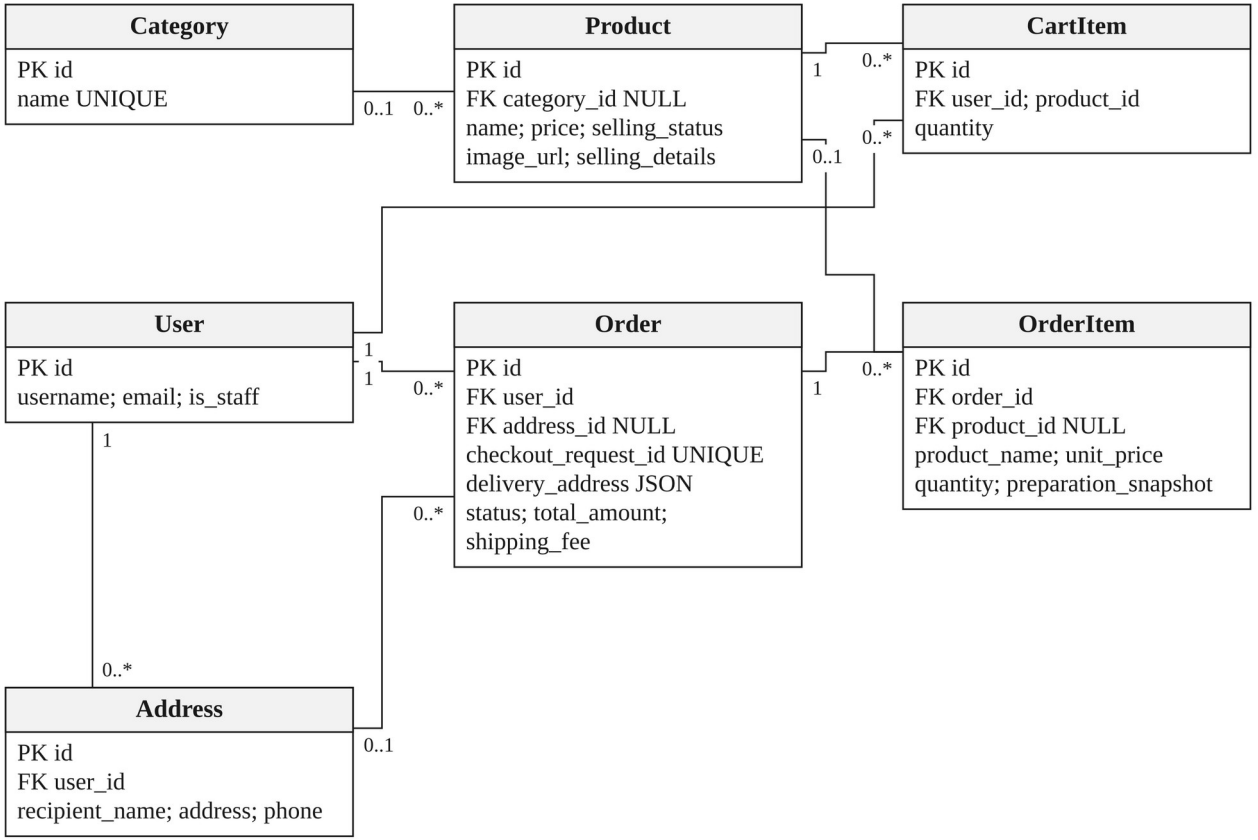
Figure 3.18: Core domain classes and service-module responsibilities.

The diagram uses actual model names. Services are explicitly stereotyped because scheduling and order_progress are Python modules rather than persistent entity tables. User-facing menu wording maps to Product. The full field inventory in Appendix D complements this selected domain view; the entity diagrams below describe foreign-key relationships rather than every method in the codebase.

3.8 ENTITY–RELATIONSHIP DIAGRAMS

The database design is divided into four views so that cardinalities and audit relationships remain readable. PK denotes a primary key; FK denotes a foreign key. A nullable child FK means the child can reference zero or one parent. A parent can have zero or many children unless the annotation says one-to-one. JSON snapshots intentionally preserve accepted terms and are not additional relational entities. Django’s built-in authentication, session and migration tables are outside these application-domain views.

3.8.1 COMMERCE AND ORDER HISTORY

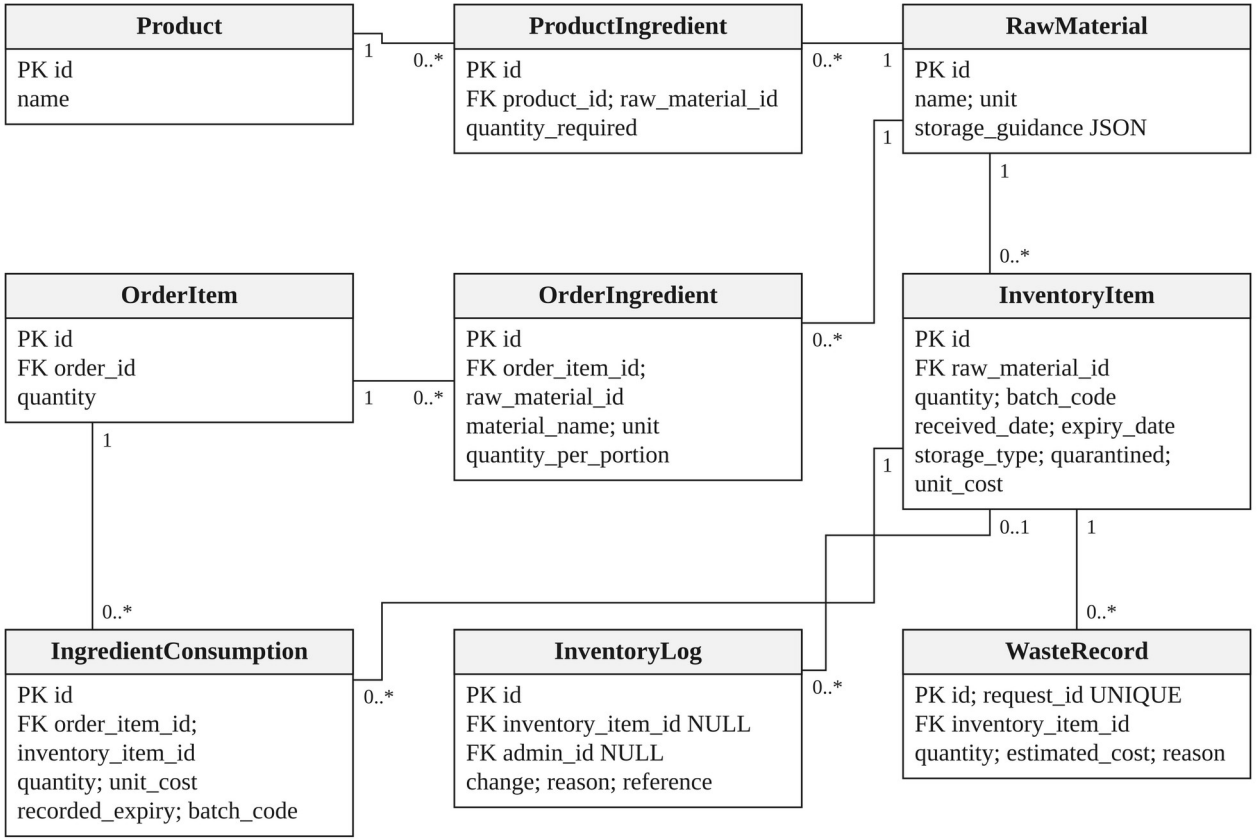


delivery_address preserves accepted address.
JSON snapshots are columns, not separate entities.

Figure 3.19: Commerce, basket, addresses and order-line ERD.

Deleting an address does not erase the order’s accepted address snapshot. Product references in old OrderItem rows can be null while the accepted product name and price remain available. The current menu lifecycle protects relevant commitments and supports archival; a diagram must not imply that all history should be renumbered or deleted when a menu is retired.

3.8.2 RECIPES, DATED BATCHES AND ACTUAL CONSUMPTION

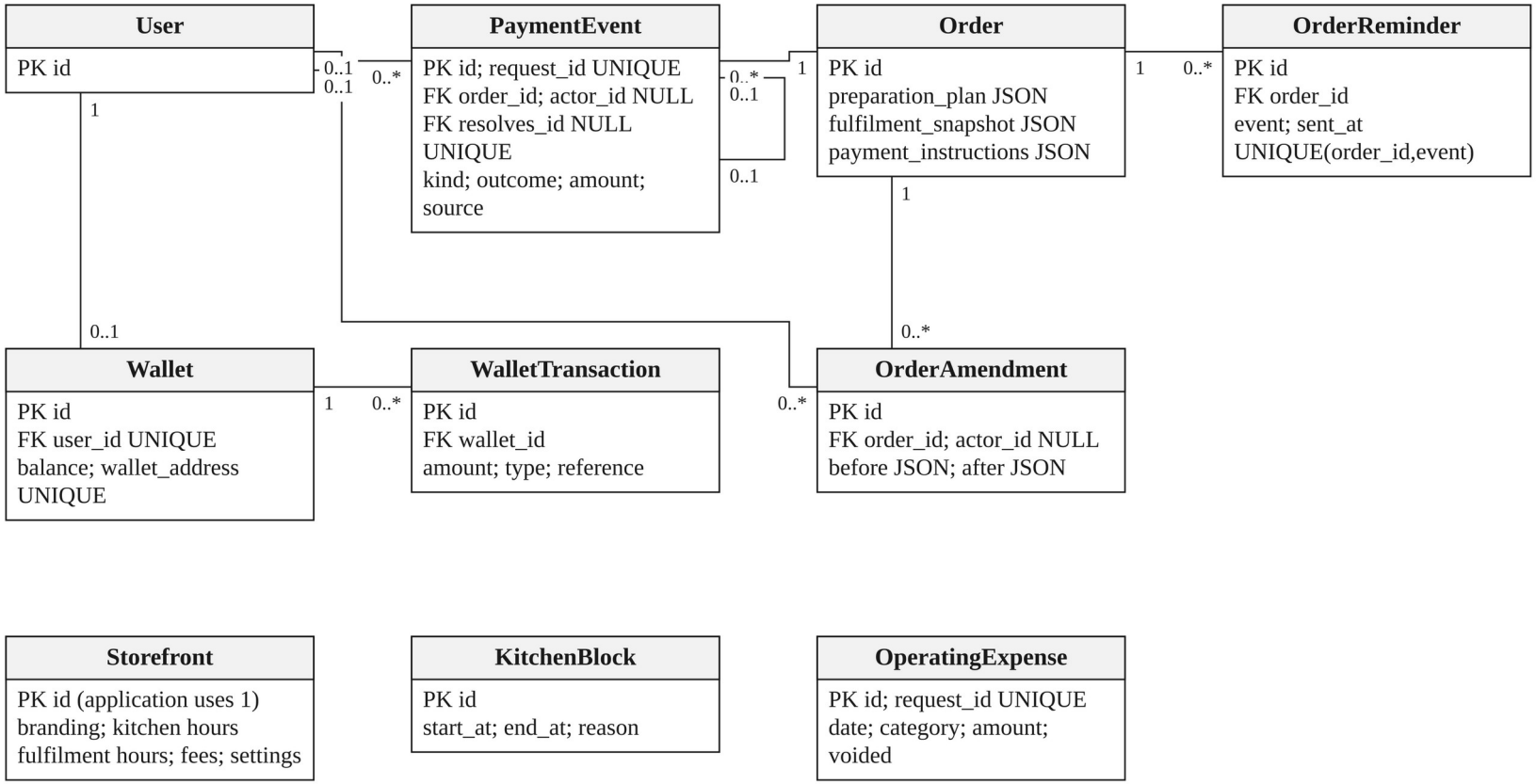


PK = primary key; FK = foreign key. Selected fields are shown; the full dictionary appears in Appendix D.

Figure 3.20: Current recipes, accepted recipes, stock and consumption ERD.

ProductIngredient describes the current recipe; OrderIngredient is the accepted recipe for one order item. IngredientConsumption links that item to the batches actually used and preserves unit-cost/date evidence. This supports tracing a batch into orders and calculating contribution only when its inputs are complete. InventoryLog and WasteRecord serve different audit purposes and must not be treated as interchangeable deductions.

3.8.3 OPERATIONS AND PAYMENT RECORDS

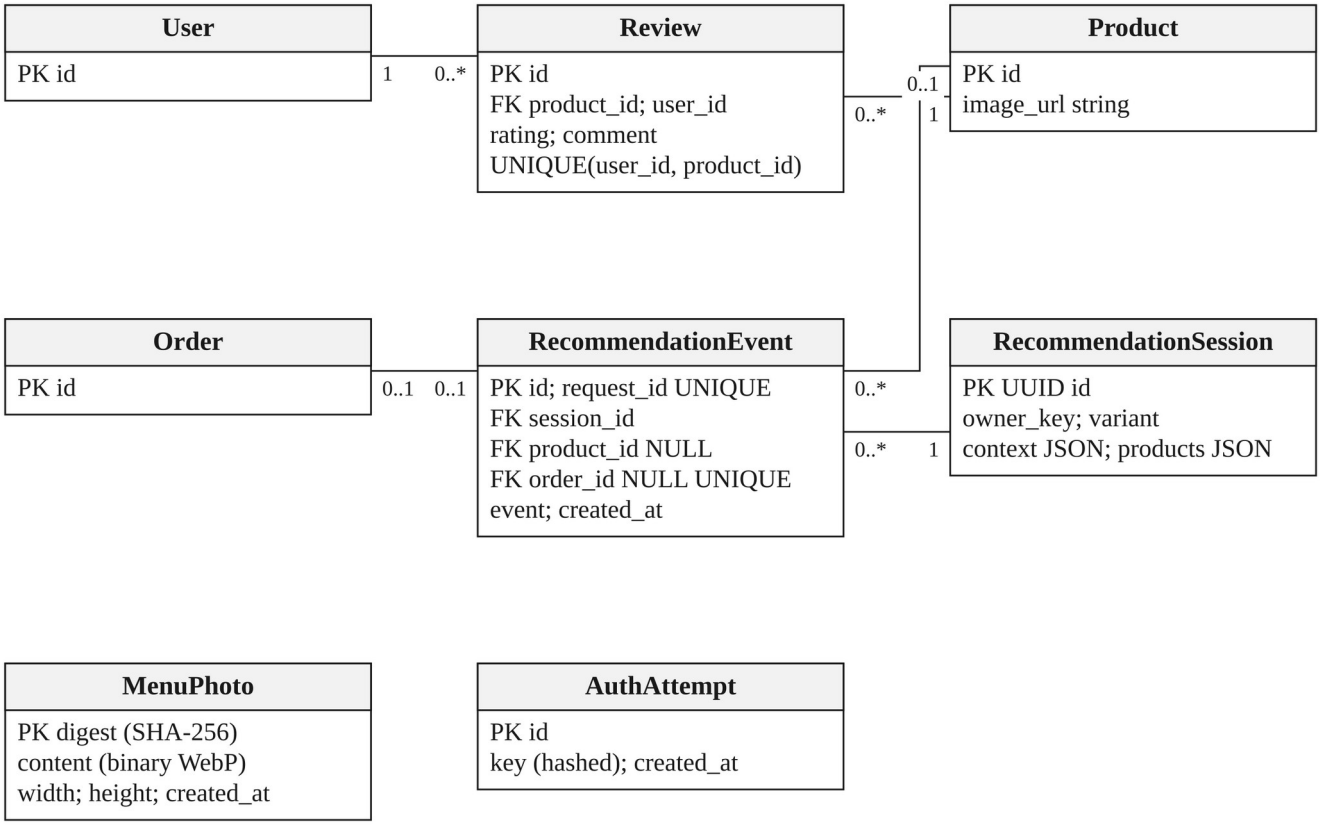


User 1 — 0..1 Wallet (one-to-one).
PaymentEvent resolves 0..1 earlier PaymentEvent (self one-to-one).
Storefront, KitchenBlock and OperatingExpense have no Order FK.

Figure 3.21: Operational settings, amendments, reminders and payments ERD.

OrderReminder has one receipt per order/event key. PaymentEvent records receipt/refund evidence with a unique request ID; its optional resolves relation links a resolution to an earlier event. Storefront is treated as a single business configuration by the application, not a multi-tenant marketplace. KitchenBlock is a global time interval; its effects are calculated without an Order FK. OperatingExpense is an independent dated expense record.

3.8.4 ENGAGEMENT AND SUPPORTING RECORDS



Order 0..1 — 0..1 RecommendationEvent (one-to-one).
Product.image_url may reference MenuPhoto URL: no database FK.
RecommendationSession.owner_key is hashed: no User FK.

Figure 3.22: Reviews, recommendation telemetry, menu photos and attempt limits.

MenuPhoto is content-addressed by a digest and stores sanitised WebP bytes. Product.image_url is a string that can point to its public route or an external image; it is not a MenuPhoto FK. RecommendationSession uses a hashed owner key instead of a User FK. AuthAttempt stores hashed rate-limit identities, not passwords. These distinctions prevent the conceptual ERD from inventing relationships that the schema does not enforce.

3.9 REQUIREMENT TRACEABILITY AND DESIGN LIMITS

Table 3.2: Requirement traceability and design limits.

Objective	Principal user goals	Persistent evidence	Implementation boundary
1. Flexible date-based preordering	UC-03–08, UC-21, UC-24/25	Order, accepted snapshots, amendments and shared task reservations	Bounded deterministic planner; a request can remain unconfirmed
2. Shelf-life/storage monitoring	UC-11–15, UC-17/28	Material guidance, InventoryItem, logs, consumption and waste	Date-based planning; no sensor or microbial assessment
3. Single-vendor branding	UC-01, UC-09/10, UC-26	Storefront, Category, Product and MenuPhoto	One kitchen; no marketplace competitor feed
4. Data-driven analytics	UC-18/19, UC-27/28	Order, financial events, consumption and dated demand sources	Missing cost remains unknown; forecasts are estimates
5. Menu recommendations	UC-02/03/04	RecommendationSession and RecommendationEvent	Explainable ranking and bounded candidates; impact requires user evaluation

3.10 CONCLUSION

The updated design connects each objective to an implemented workflow while separating future capacity, ingredient availability, payment evidence and actual fulfilment. It also recognises the owner’s workload: several eligible orders can share a progress confirmation, compatible cooking can be explicitly combined, and intermediate updates are optional. Chapter 4 explains how these responsibilities appear in the current interfaces and code.

CHAPTER 4

INTERFACE AND SYSTEM IMPLEMENTATION

4.1 INTRODUCTION

This chapter explains the implemented Dapur Kita website from customer and kitchen-owner perspectives. It also describes the architecture, calculations and integration boundaries behind the screens. The evidence baseline is Git revision ab1a976, reviewed on 29 October 2026. All screenshots in this chapter were captured from that application using isolated local test records; they are not photographs of a production business or evidence of real sales. Personal contact, payment-provider and production database credentials were not used for the captures.

The implementation extends the proposal without replacing its five objectives. Customers can browse first, receive optional recommendations, maintain a basket and choose a workable handover arrangement at checkout. Owners manage menus, preparation, stock, financial records and fulfilment from the same single-kitchen workspace. Additional functions—FreshCast, email reminders, assisted ordering and test payment integrations—support these core workflows rather than establishing new objectives.

4.2 APPLICATION ARCHITECTURE

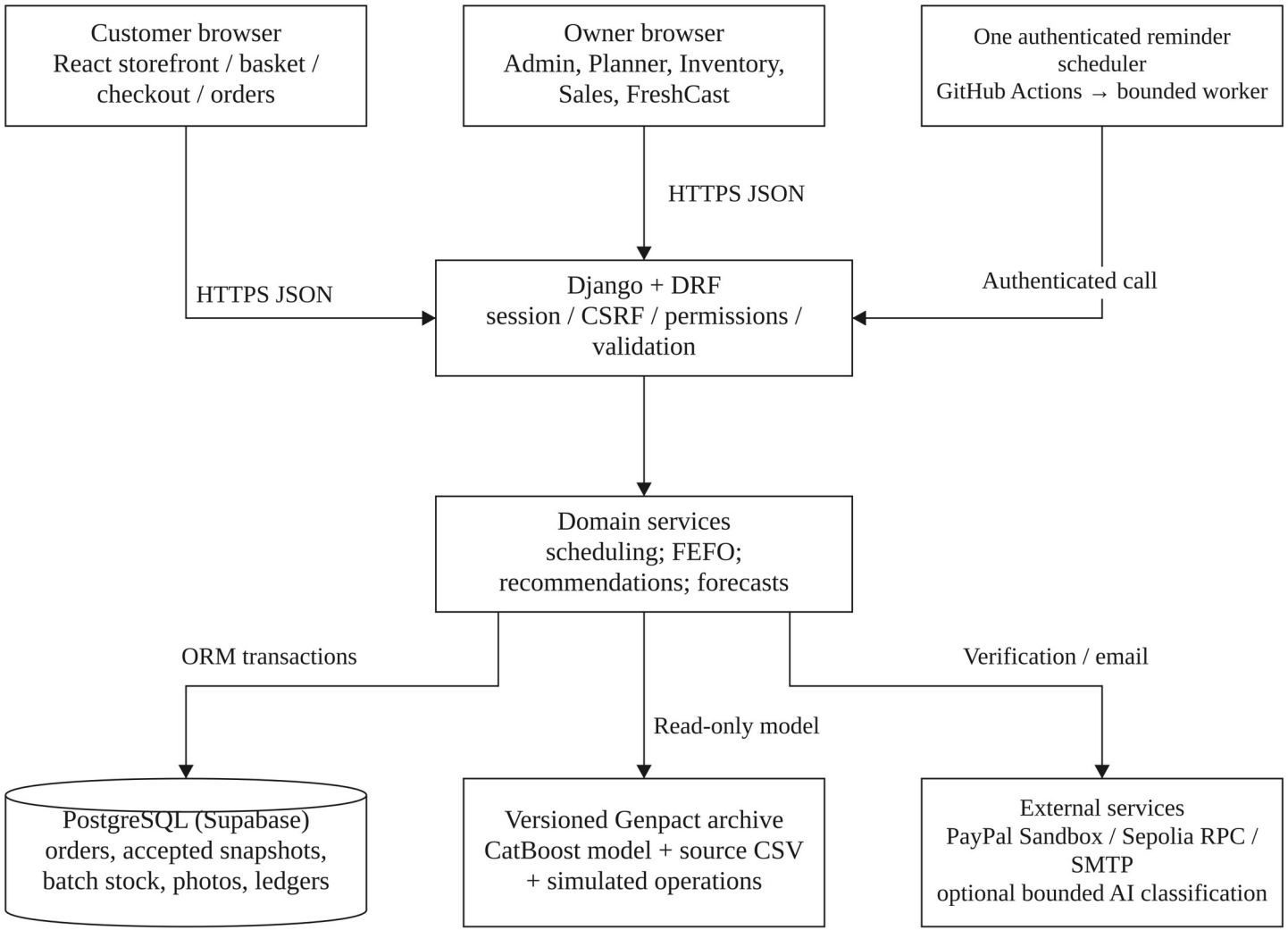


Figure 4.1: Implemented presentation, API, service and persistence layers.

The React frontend provides routed customer and owner pages, shared Context providers and reusable components. Vite produces the static application bundle. Django and Django REST Framework expose JSON endpoints and enforce authentication, CSRF, permissions, business validation and database transactions. PostgreSQL hosted by Supabase stores the application's relational records. Vercel hosts the deployed application; it is not the persistent database. The configured database schema is django_app in the owner's Supabase installation. The source review does not independently certify that installation's current access policies.

This architecture uses Django models and views, but the interactive interface is React rather than a collection of Django-rendered business templates. PostgreSQL is accessed through Django's ORM; the customer browser is not given the database connection string. An external scheduler invokes a narrow authenticated email worker. PayPal, Sepolia JSON-RPC and optional AI classification are separate configured services. The Genpact model/data bundle is provisioned as a versioned read-only runtime asset; ordinary kitchen operations do not need to train or load it.

Table 4.1: Application architecture.

Layer	Main implementation	Responsibility
Presentation	frontend/src/App.jsx; pages, components and Context providers	Navigation, forms, responsive views, loading/recovery and display state
API	backend/myapp/urls.py; views; serializers.py	Access checks, strict inputs, bounded responses and authoritative operations
Domain services	services/scheduling.py, work_sessions.py, order_progress.py and recommendation.py	Preparation constraints, actual progress and explainable ranking
Stock and finance	InventoryItem, OrderIngredient, IngredientConsumption, PaymentEvent and report services	Accepted requirements, dated stock, audit evidence and financial calculations
Decision support	kitchen_decisions.py, kitchen_forecast.py and predictive_ai	Distinct local and external forecast paths; read-only scenarios
Integrations and operations	PayPal/escrow services, reminders.py, GitHub worker workflow and build.py	Independent provider verification, bounded email execution and release checks

The application uses decimal values for money and measured quantities at the transactional boundary. Frontend labels format values for the reader but do not determine server prices or inventory deductions. Public menu data is separated from private customer and staff records. Optional interfaces are loaded as needed; a missing provider configuration does not silently turn a test mechanism into a real payment service.

4.3 CUSTOMER INTERFACES

4.3.1 STOREFRONT, MENU AND PRODUCT DETAILS

The public storefront presents one kitchen's name, story, contact route, food categories and menu photographs. It does not surround a seller's products with competitors' listings. Customers can search the menu and switch categories before using assistance. Product details show the selling unit, price, notice requirements, permitted dates or weekdays, daily capacity, sizes/fixed contents where configured and a route for allergen questions. A box or pack is one selling unit; the number of pieces inside it is not automatically the number of diners.

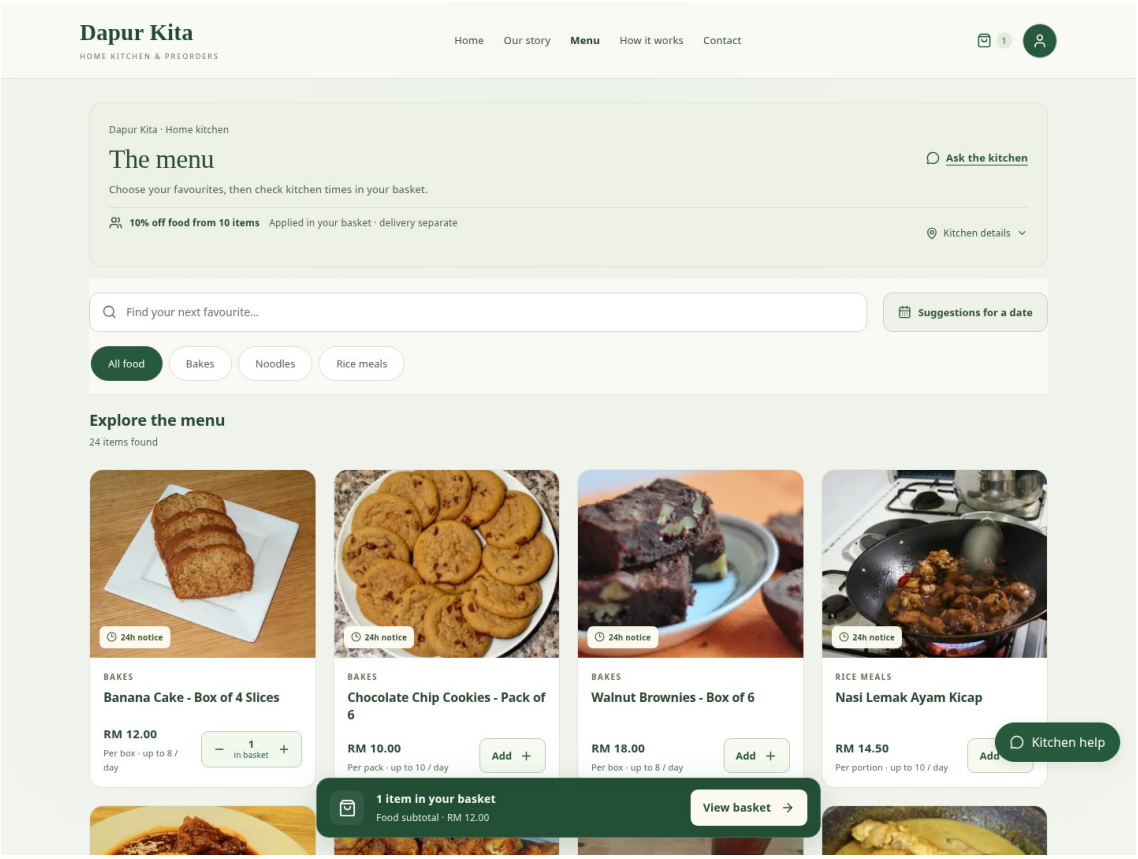


Figure 4.2: Current customer menu with search, categories and compact food cards (local test records).

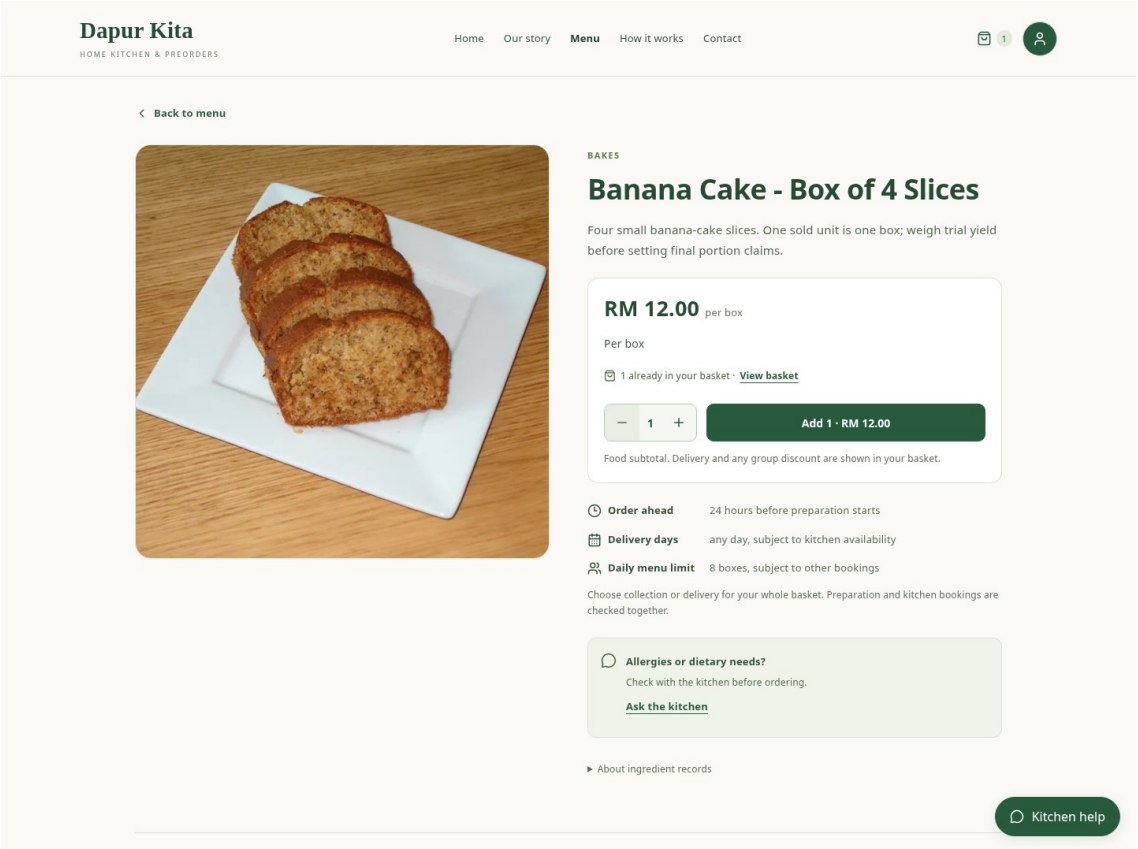


Figure 4.3: Product detail showing price, selling unit and preorder information (local test record).

The initial Add action changes into a minus/quantity/plus control once that menu is in the basket. The accompanying in basket caption makes the number's meaning explicit. Increasing, reducing or removing a line updates the same basket rather than producing a new order. A fixed basket summary links to the next stage without obscuring the food grid on tested phone widths.

Archived products are retained for relevant order history, while new ordering uses active selling status. Paused and archived menus are not silently treated as available recommendations. Menu categories are owner-managed. A seasonal product can use a shipping-enquiry route: it opens a contextual conversation rather than accepting a local-delivery payment or promising a parcel arrival date.

4.3.2 ACCOUNTS, ADDRESSES AND CONTACT PREFERENCES

The current customer experience supports email-based account creation and email-or-username sign-in, retaining legacy username accounts. Password validation, recovery and attempt limits are enforced by the backend. An email-change workflow requires current-account proof and confirmation; an order's reminder consent is not silently transferred to a different address. Saved addresses belong to the signed-in

customer. The order stores the accepted address, so later address edits do not rewrite that delivery record.

Phone and preferred-contact details support a specific order's handover. Optional instructions can be disclosed progressively rather than turning every contact preference into a required checkout task. An email address is required for SMTP reminders; a phone number alone cannot receive email. Opening a WhatsApp draft is a deliberate external contact action, not an automated WhatsApp Business notification. Customer account access does not grant the owner's APIs.

4.3.3 TWO COMPLEMENTARY RECOMMENDATION SURFACES

Recommended for you provides immediate discovery without requiring a form. It ranks eligible active menus using paid, non-cancelled history and recent popularity. Customers without useful personal evidence receive an honest menu/popularity basis rather than invented preferences. The current routed ranking awards 25 points for a preferred category, up to 20 for earlier purchased quantities and up to 15 for recent popularity. Category history uses 180 days, menu history 365 days and global popularity 30 days. These are explainable heuristics; no training dataset or personalised neural model is implied.

The optional Menu suggestions for your date finder asks for a date, total item count, mixed/same-menu preference, optional whole-order budget and category preference. A date is required; leaving the budget blank means no price limit. Budget preference is disabled until a budget exists. The date is within the next 90 days and the item count is 1–100. Suggestions are alternative baskets. A target of three means three total items in a selected option, not three of every displayed dish.

Allow a mix keeps existing basket items and distributes only the remaining target quantity. One menu only searches independent repeated-menu alternatives; replacing an existing basket requires explicit confirmation. This avoids recommending only the already-present menu and prevents an unexpected destructive replacement. Up to three options are returned after pricing and shared capacity checks. The search is bounded to a seeded pool of up to 12 menus and at most 30 complete-basket capacity checks. It is useful decision support, not exhaustive combinatorial optimisation.

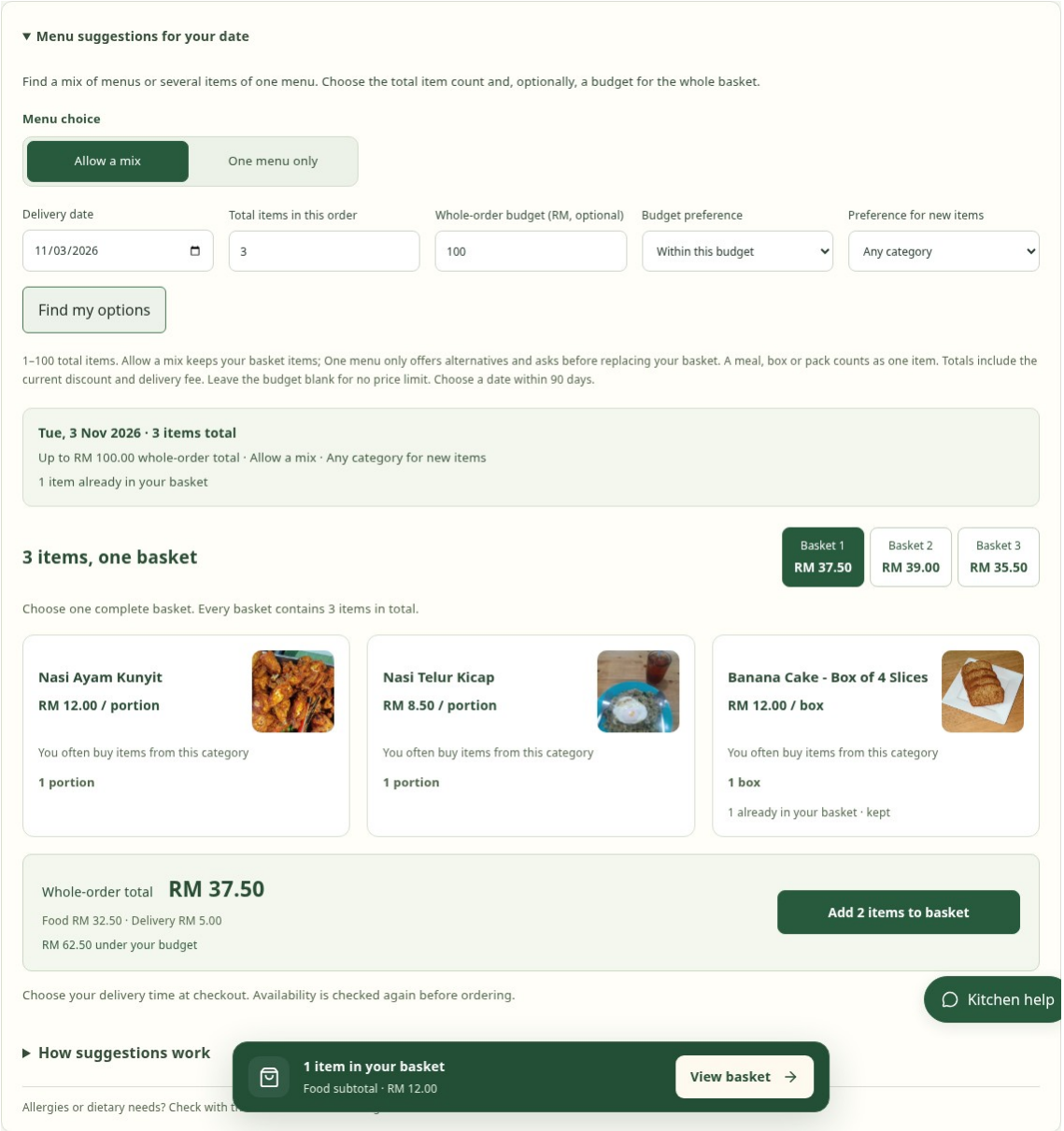


Figure 4.4: Whole-order date finder with three total items and an optional RM100 budget (actual local search).

The budget comparison uses the complete candidate order total: food subtotal minus the current applicable bulk discount plus the current delivery fee. Existing basket items are included in mixed mode. Within includes equality; Above means strictly greater than the entered budget. Comparison occurs before the three-result limit. If all feasible candidates cost less than RM100, Above RM100 correctly has no results; it does not mean every menu omitted from the top three should appear. A missing category is distinguished from a scheduling failure.

For example, three RM12 boxes have a food line total of RM36, not RM36 each. If the configured delivery fee is RM5 and no discount applies, that candidate's whole-order total is RM41. A mixed example of one RM12 box, one RM10 pack and one RM18 box has food subtotal RM40 and total RM45 under the same fee. Both fit

RM100. These are arithmetic illustrations, not recorded orders or a price guarantee; checkout rechecks current accepted terms.

Adding a dated suggestion applies the basket and preserves the chosen day. It does not reserve or force an hourly handover slot. The customer chooses or reviews the final time at checkout, avoiding repeated time-selection controls across discovery, basket and checkout. Recommendation events record rendered exposure, selection and additions where applicable; telemetry failure must not prevent shopping. Objective 5's intended decision-fatigue and conversion effects still require actual evaluation, described in Chapter 5.

4.3.4 BASKET AND FULFILMENT CHOICE

The basket groups food lines, quantities, the current summary and Receive your preorder. Delivery and self-collection appear only when the owner has enabled and completed their configuration. A collection address alone does not prove that collection is offered. Collection requires enabled service, a public location and valid collection hours. Delivery uses saved address details and configured area choices. A descriptive service-area sentence is separate from the short list of selectable area names.

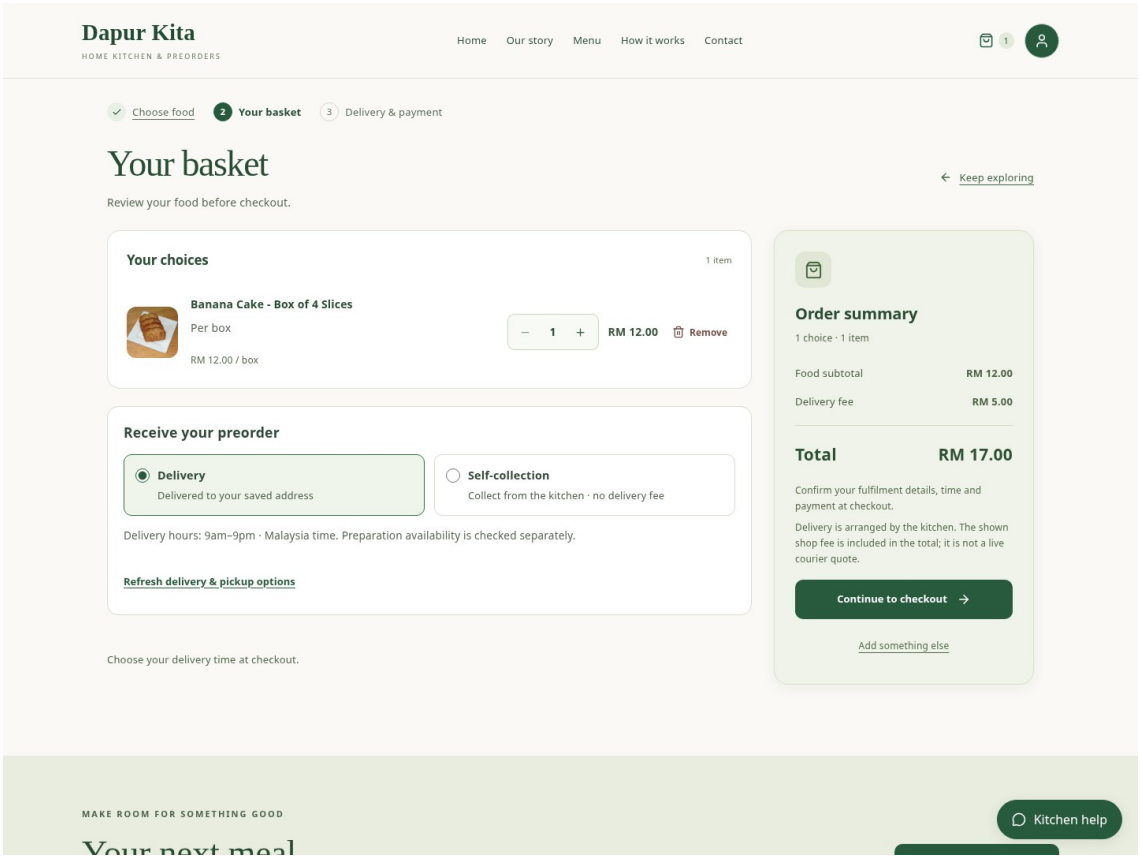


Figure 4.5: Basket quantities, fulfilment choice and authoritative summary (local test records).

Proceeding to checkout is appropriate because the basket is primarily a review step; required address, time and payment validation belongs to the checkout that creates the order. A basket failure hides the onward action until its contents can be read. An optional availability preview can help customers earlier, but it is not a second mandatory checkout form. Delivery and collection choices persist into the final quote and are revalidated there.

The owner's delivery fee and optional free-delivery threshold are editable settings. They are not a live GrabExpress fare or a prediction learned from delivery history. The customer accepts the displayed fee at placement, and the order retains it. A higher later courier charge is a business arrangement to resolve explicitly; the system does not automatically charge the customer twice. Collection has no delivery fee or travel buffer. The current customer wording groups legacy express and standard delivery as Delivery while retaining accepted historical terms.

4.3.5 CHECKOUT, FEASIBLE TIMES AND REVIEW REQUESTS

Checkout presents delivery/collection, address where required, a time or supported arrival window, optional contact/handover preferences, reminder consent, payment options and the order summary. Availability depends on the entire basket. The server checks menu restrictions, order cutoffs, notice before preparation, kitchen resources, existing bookings, closures, daily selling-unit limits and handover hours. A quotation is advisory until placement rechecks under locks.

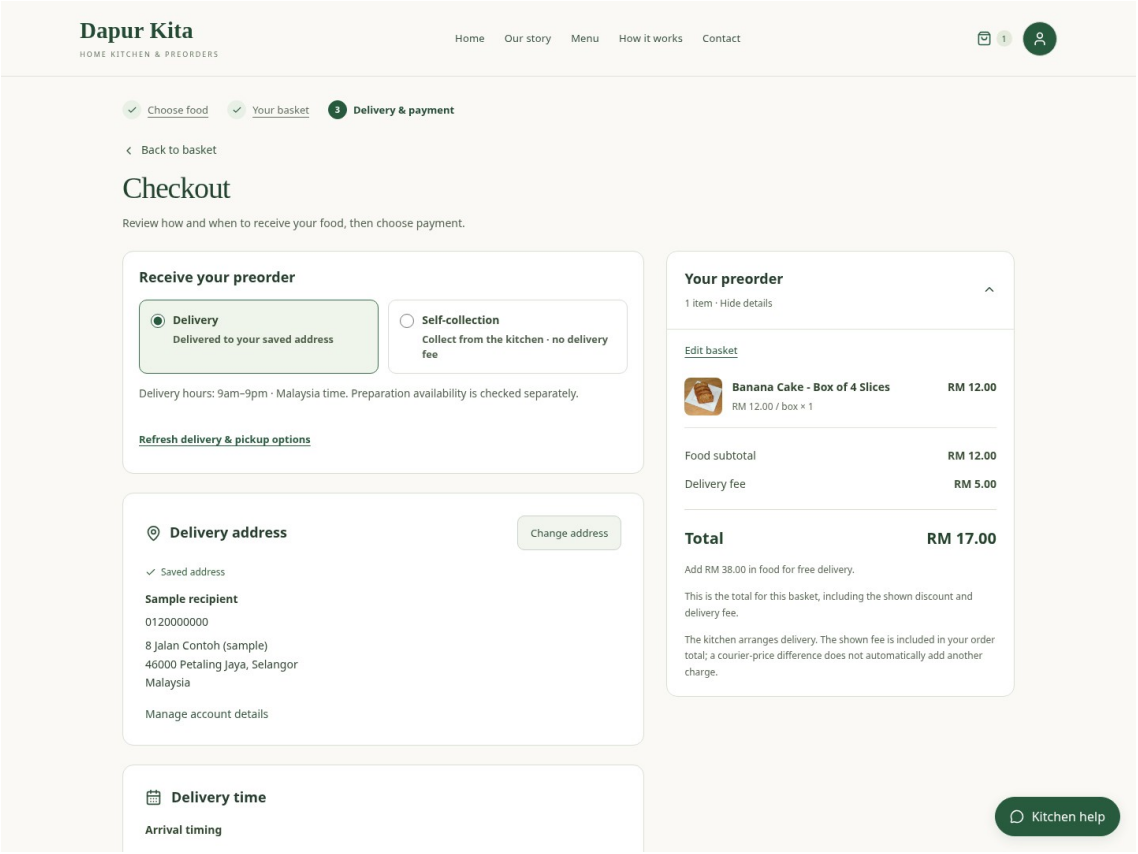


Figure 4.6: Checkout separates fulfilment, address, time and payment (local test record; no payment submitted).

The next-available-day search reduces trial and error when a preferred day has no automatic time. A rejection explains the relevant rule rather than using one generic unavailable message. Current ingredient stock and kitchen time are separate: the owner may procure ingredients for a future booking, but the owner must resolve a shortage before actual cooking can be recorded.

A customer may explicitly submit an eligible request needing kitchen review. The saved request is unpaid, uses a deferred payment choice and reserves no preparation tasks. It is displayed as awaiting confirmation rather than a guaranteed booking. Once the owner confirms a workable plan, the customer chooses an available payment method from that saved order. Repeating a valid payment-choice request returns the same selection; the selection itself does not charge funds or change the price.

Cash is offered only as Cash at collection. New delivery orders cannot select cash on delivery, including by forging a hidden field. Manual bank/DuitNow payment appears when configured and is verified externally by the owner. PayPal is a Sandbox integration with a visible test-funds note; optional delivery escrow uses Sepolia. These separate mechanisms do not make every submitted preorder paid.

4.3.6 MY ORDERS, AMENDMENTS AND REVIEWS

My orders separates Current, History and All with private counts. The API returns at most ten records per page and a descending-ID cursor. Load more appends older records; refresh replaces the current page. New insertions do not shift the older cursor, and an interrupted append can retry without duplicating rows. A failed refresh hides potentially stale actions. The interface distinguishes the accepted delivery time/window, kitchen stage, payment status, courier reference and reminder preferences.

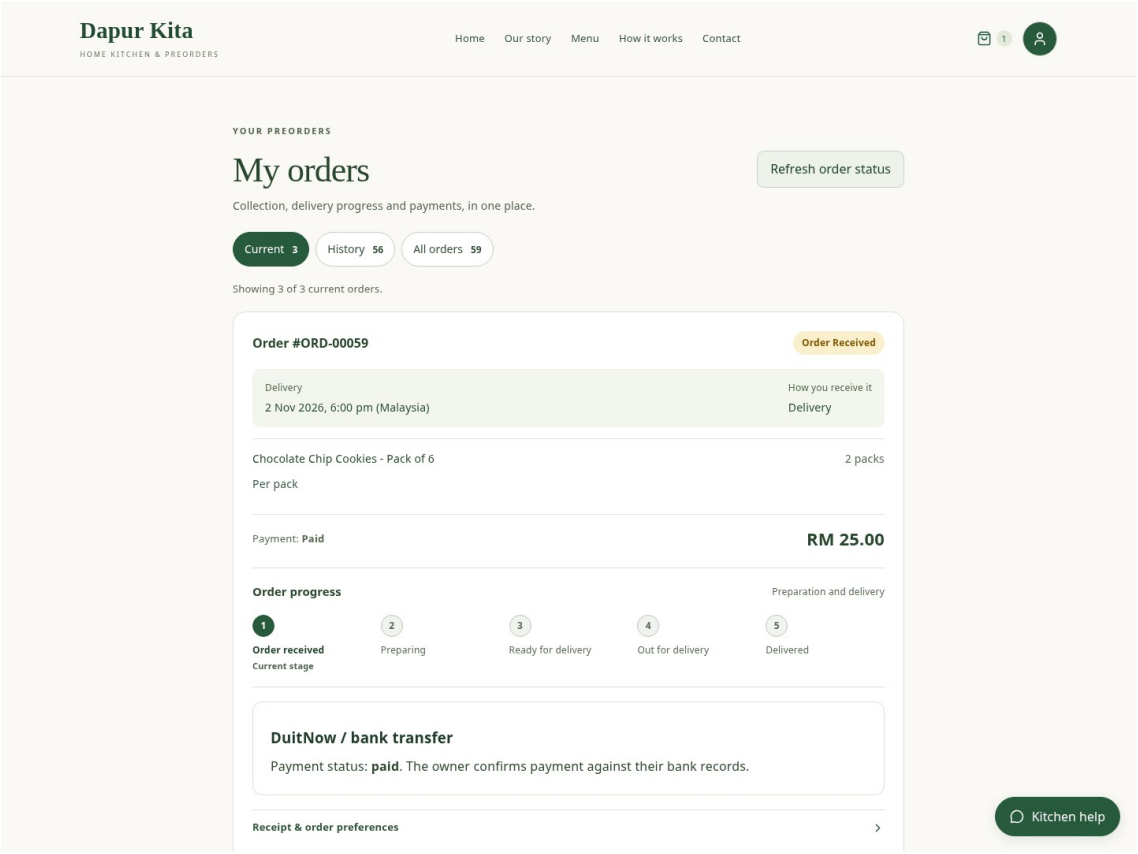


Figure 4.7: Customer order cards, private filters and payment/fulfilment separation (local test records).

Changes to delivery use a preview-and-confirm workflow with current eligibility checks. Started, consumed, refunded or invoice-locked arrangements can require owner resolution instead of silently changing a reservation. When an unscheduled request changes time, it remains unconfirmed unless the operation also confirms a valid plan. The existing order is amended; a date edit is not a new payment or fabricated completion.

Customers can leave a bounded rating/comment only when an eligible delivered order contains the menu. Reviews are not invented to make the storefront appear active. Courier information is kitchen-entered; customers can open its saved public tracking

link and contact the kitchen. A scheduled minute is an agreed arrangement, not a verified live courier ETA.

4.4 OWNER INTERFACES

4.4.1 MENUS, CATEGORIES AND PHOTO UPLOADS

Menu management combines status, price, recipe, preparation settings and selling details. The owner can create and edit categories, create a menu, pause ordering and archive a retired offer. Deletion is subject to current commitment/history safeguards; archival preserves records needed for old orders. Required price/category errors are associated with the form instead of being shown as duplicate, unstructured messages.

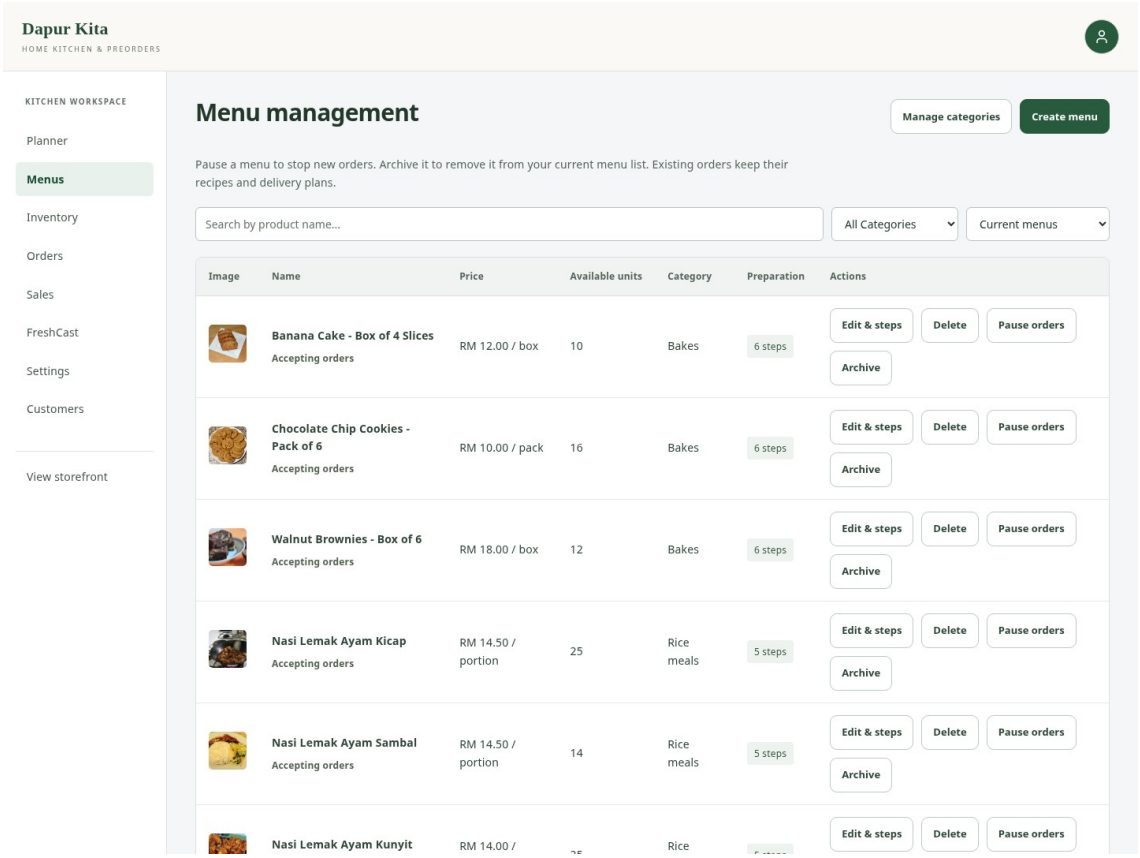


Figure 4.8: Owner menu list with price, recipe and lifecycle actions (local test catalogue).

A selling unit can be a meal, box, pack or other documented unit. Optional serving guidance is descriptive. Fixed box contents are shown without assuming flavour substitutions. Sizes can be separate products with independent prices and recipes, connected by a size group. Sale dates, weekdays and a cutoff can be set per menu. The cutoff does not replace lead time: 24 hours of notice means 24 hours before preparation starts, plus cooking and any travel buffer before handover.

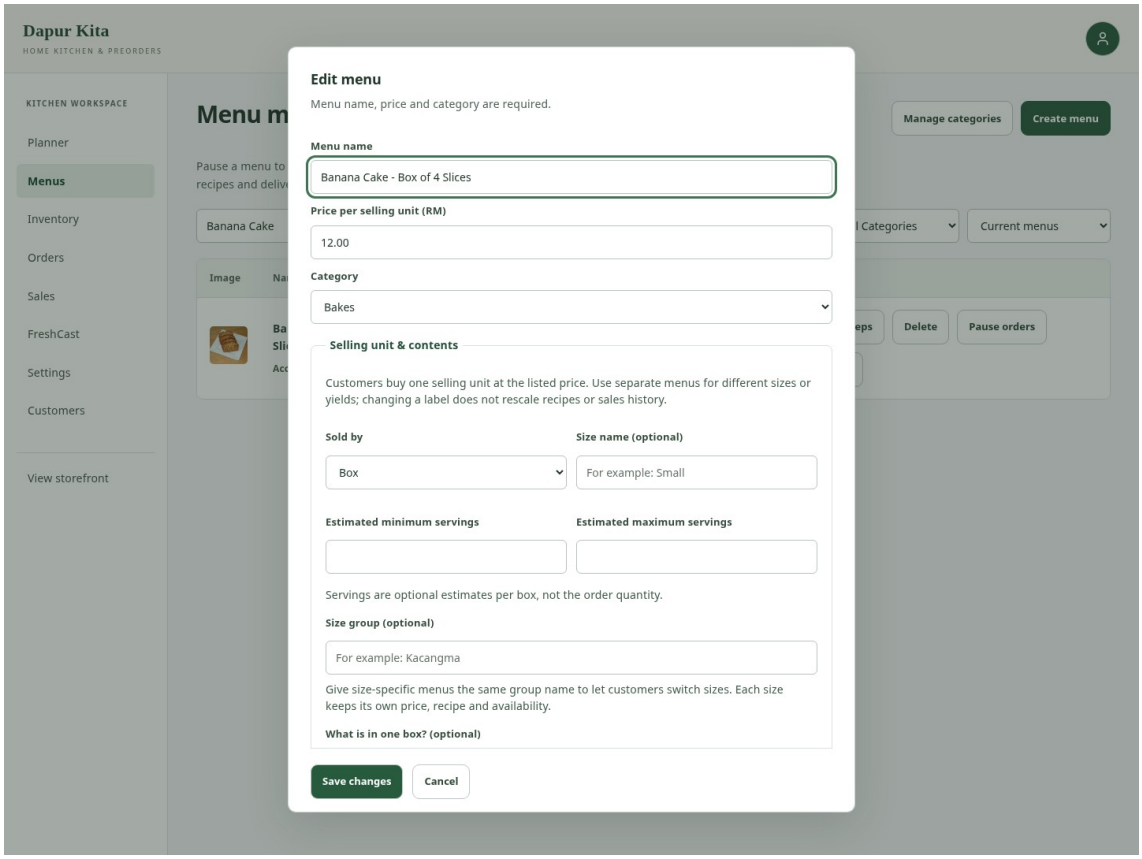


Figure 4.9: Menu editor with selling details and progressive recipe/preparation settings (local unsaved form).

Photos support public URLs and local file upload. The upload endpoint requires staff permission and CSRF. It decodes still JPEG, PNG or WebP input up to 4 MiB, rejects malformed/animated/oversized images, applies orientation, removes metadata and re-encodes at most 1600-pixel WebP within 512 KiB. Sanitised content is deduplicated by SHA-256 and stored in PostgreSQL through MenuPhoto. This persists across Vercel deployments without relying on its temporary filesystem. Uploading is public menu-media publication, not private file storage. An upload cancelled before menu save can remain unreferenced; storage maintenance is a separate concern.

4.4.2 INGREDIENT DIRECTORY AND INVENTORY

The ingredient directory records each raw material’s native unit and optional estimated unit cost. Reusable storage guidance can contain documented storage type, days from receipt, source and matching conditions. It is optional rather than a compulsory prerequisite to receiving stock. The owner can enter batch-specific documented guidance when no suitable material reference exists. A chilled reference is not offered as a frozen reference, and editing guidance does not revise previously recorded batch evidence automatically.

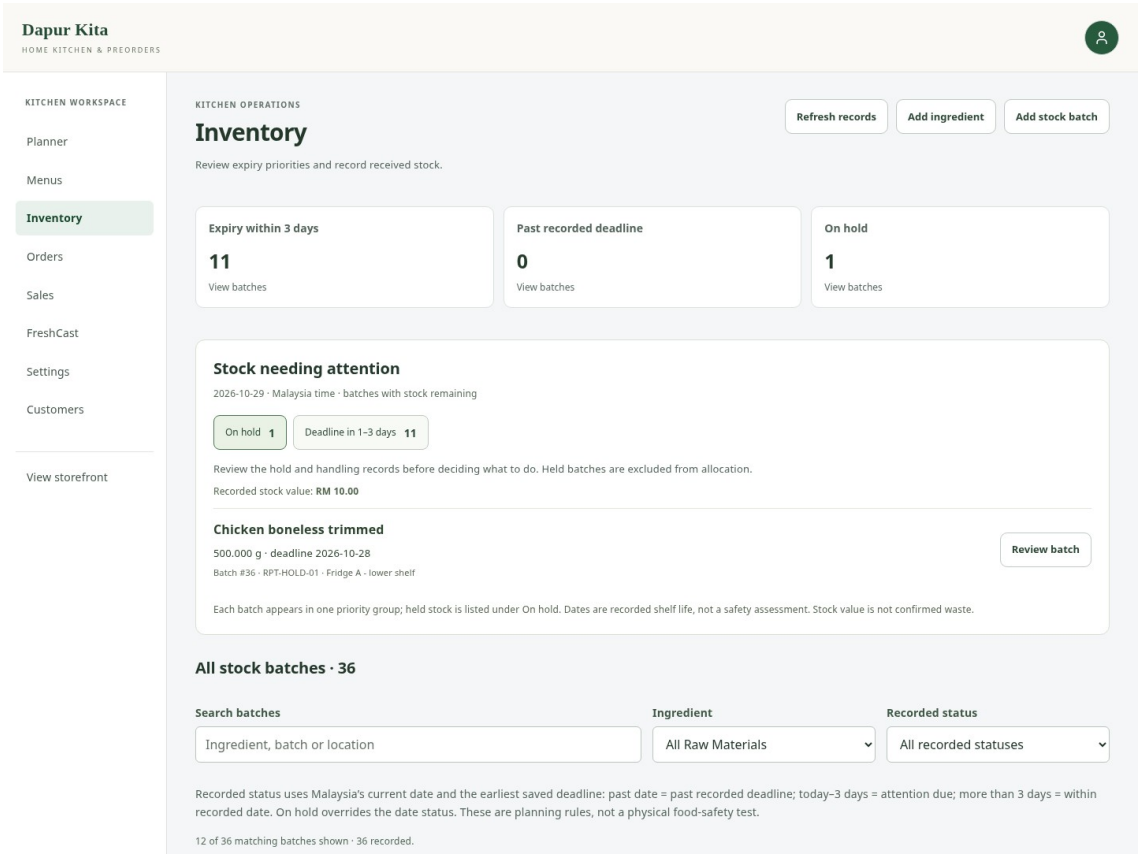


Figure 4.10: Inventory overview, held/dated stock and batch review (local test records).

The batch form supports three expiry modes. Printed expiry/best-before records the supplier's date and its meaning. Manufacture date plus documented shelf life uses the production date as the calculation start. Received date plus documented storage life supports suitable unlabelled ingredients, including wet-market purchases, where evidence matches the ingredient's actual condition. Purchase source is contextual evidence; selecting Wet market does not magically provide a safe duration. The FoodSafety.gov cold-food storage chart is a reference to read, not an automatically verified supplier label (FoodSafety.gov, n.d.).

Table 4.2: Ingredient directory and inventory.

Recorded field	Meaning	Effect on the batch
Received date	When the kitchen received this batch	Receipt-based start and future-receipt eligibility
Manufacture date	When the ingredient/product was produced	Start for manufacture-plus-life mode
Printed deadline	Supplier use-by/expiry or quality best-before date	A recorded allocation deadline; not the manufacturing date
Documented shelf/storage days	Applicable whole-calendar-day duration with source/conditions	Calculates a planning deadline in the chosen mode

Recorded field	Meaning	Effect on the batch
Opened/thawed pair	Relevant date and documented permitted days	Can impose an earlier deadline
Handling history and hold	Evidence of uncertainty or breach	Uncertain/breached stock is held and excluded
Storage location	Physical kitchen location such as Fridge A, top shelf	Helps locate the batch; does not prove temperature compliance

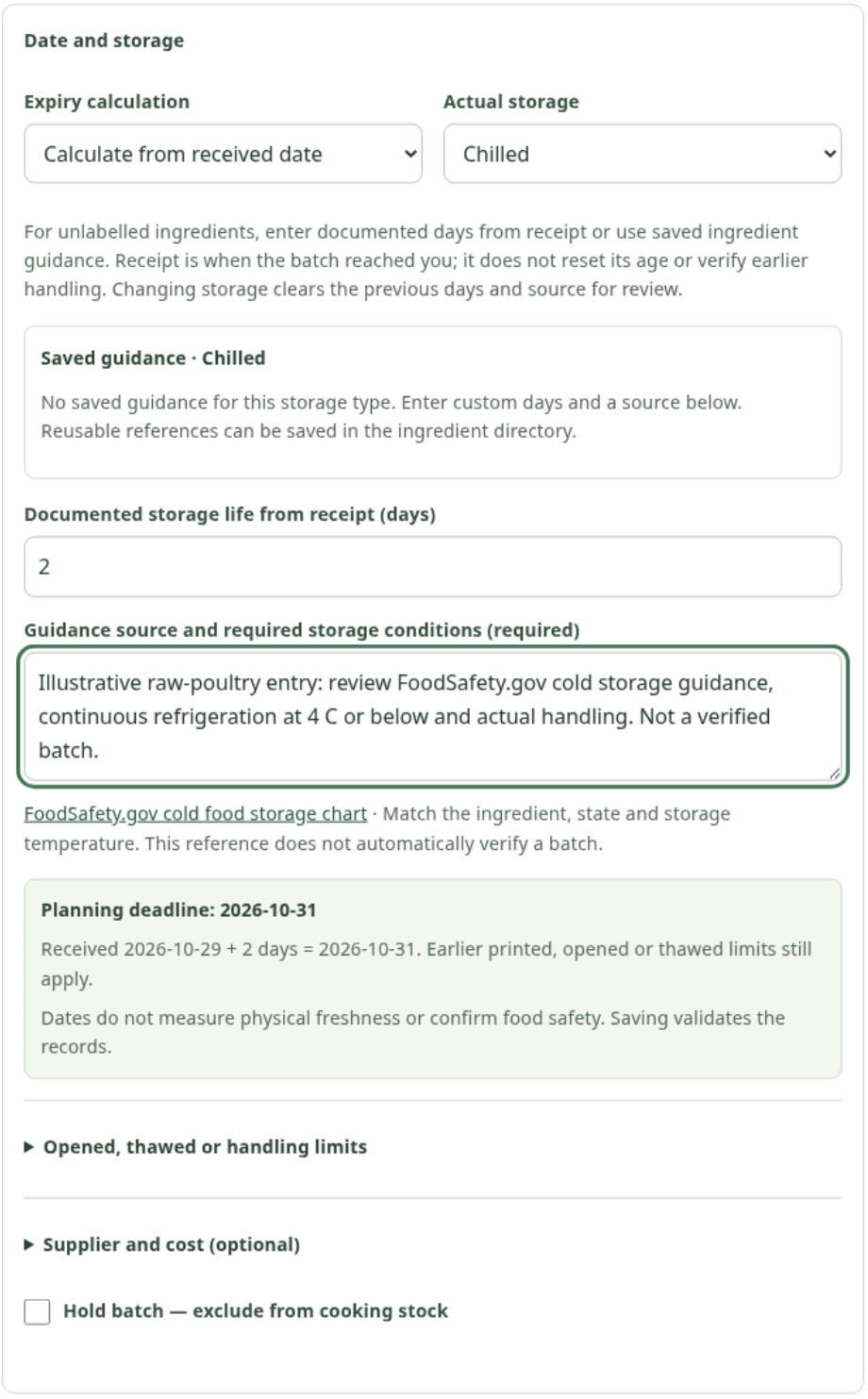


Figure 4.11: Received-date calculation for an unlabelled ingredient; owner-supplied evidence is required (unsaved local form).

Quantity edits require a reason and the current record version. Receipts, adjustments, holds, deletion and waste maintain the relevant audit evidence. Batch IDs identify deliveries, not menu IDs. Historical fictional stock remains test evidence even when its storage-location text is changed to a plausible shelf. A realistic location must not be treated as a verified physical stock count.

4.4.3 PLANNER, MULTI-DAY WORK AND CLOSURES

Planner separates accepted scheduled work from delivery requests, orders needing plan review and past-due follow-up. A pending order can already have a reserved plan; pending does not mean absent from the planner. Moving it to processing is optional progress recording, not a compulsory acceptance step for an automatically scheduled booking. An unconfirmed request has no reserved tasks and must be reviewed explicitly.

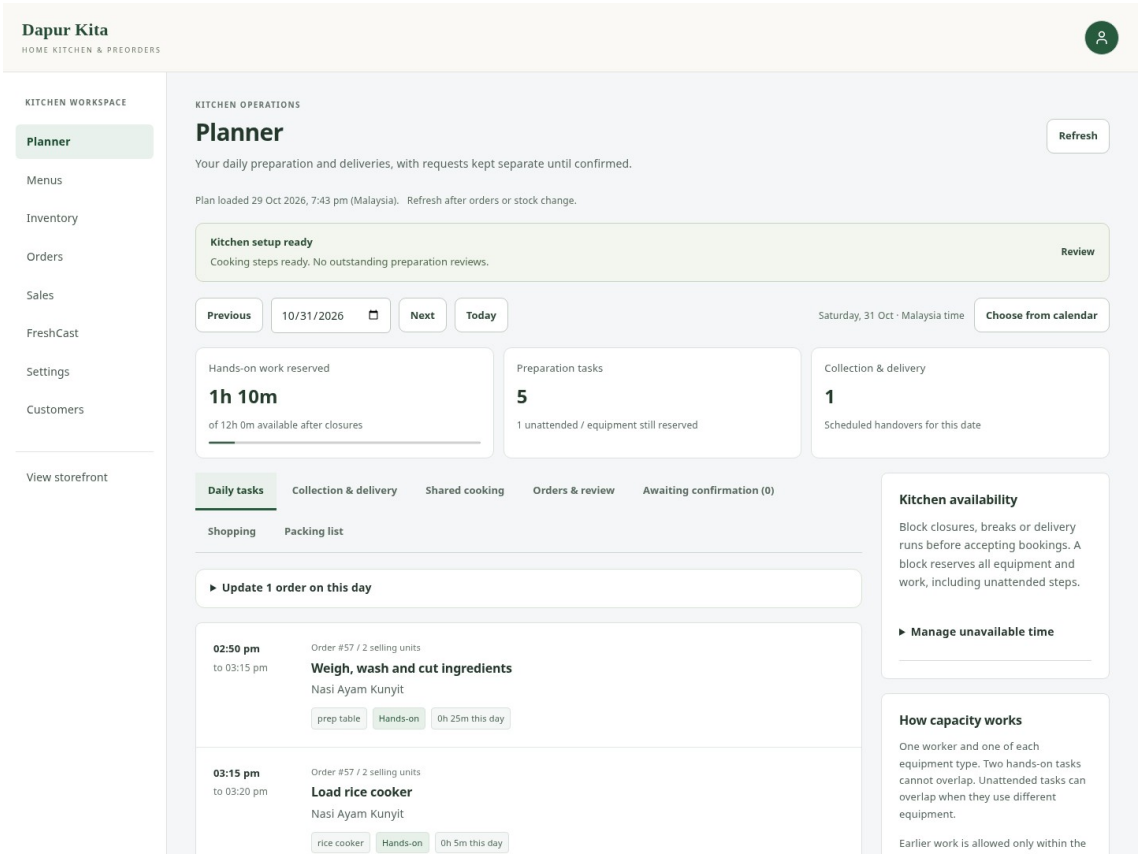


Figure 4.12: Planner with dated work, review queues and kitchen availability (local test date).

For a valid future delivery, Calculate plan derives each step's start and finish from accepted recipes and constraints. Preview shows a schedule before confirmation. Use current recipe and preparation settings is explicit because a recipe edit should not silently alter the accepted ingredients. A later delivery can be confirmed together with its new valid plan. A signed preview expires after ten minutes and rejects changes to the order or kitchen availability.

Preparation can span multiple days when its settings permit it. The scheduler distinguishes permitted gaps between steps, allowed overnight unattended fridge/rest work, independent repeated batches and explicitly splittable attended work. Splittable work is limited to supported preparation/packing/no-equipment tasks with a positive

maximum pause and a permitted pause-storage resource. Cooking and unattended steps cannot simply be interrupted to make a schedule fit. Documented recipe handling limits, not merely a larger day count, determine what is valid.

Kitchen hours apply to preparation. Delivery and collection have their own configured hours. Therefore work after a delivery closing time can prepare a later order if the menu's preparation span and finishing rules allow it. A delivery after kitchen closing can also be feasible if cooking completes within kitchen hours and the accepted travel/finish rules fit. These are independent windows; neither authorises arbitrary unsafe holding. A zero early-finish allowance uses a flexible same-dispatch-day finishing window rather than forcing every menu to finish at exactly the same minute.

Kitchen availability can block closures, breaks or delivery runs. A block reserves the whole kitchen, including otherwise unattended resources. The owner should record their own driving time if it removes the sole worker from cooking. The manual arrangement path is an advanced override with a reason and review: a continuous whole-kitchen reservation can include overnight gaps and does not retain a detailed automatic step plan. It is not equivalent to an automatically verified safe process.

4.4.4 SHARED COOKING, PACKING AND GROUPED HANDOVER

Shared cooking explicitly combines compatible confirmed, future, pending single-menu orders. The initial implementation requires identical product identity, accepted recipe/preparation/selling settings, delivery instant and dispatch buffer. A preview compares reservations for 2–10 orders. Confirmation locks the kitchen and saves a common reservation reference, counted once in calendar/availability views. It does not merge customers, payments, accepted ingredient deductions or delivery records.

Calendar export provides individual preparation tasks and deliveries; it should be re-exported after a plan changes and is not a live calendar subscription.

The daily handover view groups confirmed work by Malaysia date/hour and fulfilment. Packing lists retain each customer's accepted quantities and instructions. Up to 20 eligible orders can be selected for food-ready, dispatch or delivered/collected confirmation. The server checks actual eligibility and edit versions and applies the group atomically. An error leaves the selected group unchanged. Ready deducts ingredients once; dispatch can queue opted-in updates; delivered means an actual handover confirmed by the owner.

This reduces repeated status administration for a small operator without pretending to know events the system cannot observe. Preparing and dispatch are optional intermediate steps. A timer may identify overdue follow-up, but it cannot prove that a rider collected the food or that a buyer received it. Collection cannot be marked out for delivery. Payment remains independent of every physical stage.

4.4.5 COURIER AND CUSTOMER COORDINATION

The owner books and pays/arranges the rider through the courier's normal service, then can record carrier name, reference, link and contact information against the website order. Appropriate recipient details and a WhatsApp contact draft reduce transcription effort. Customer instructions specify the agreed contact/handover arrangement. A customer may still be unavailable or a rider may be early/late; email and saved details assist coordination but do not guarantee reception or authorise leaving perishable food unattended.

There is no connected GrabExpress booking/tracking API and no verified webhook that automatically updates website delivery status. This is an explicit scope boundary, not an invisible integration that should be claimed in the report. The existing design can support a future documented adapter, but API access, regional testing and provider events would need independent implementation and verification. The current website remains usable without that extension.

4.4.6 SALES, COSTS AND RECORDED CONTRIBUTION

Sales views show defined order/sales cohorts, menu quantities, trends and recommendation engagement. The source/date basis matters: the contribution view groups by order placement date, while demand forecasts use delivery history with labelled fallbacks. Menu line revenue, discounted order food revenue and delivery fees are not interchangeable. Filters and CSV exports disclose what range and number of records they include.

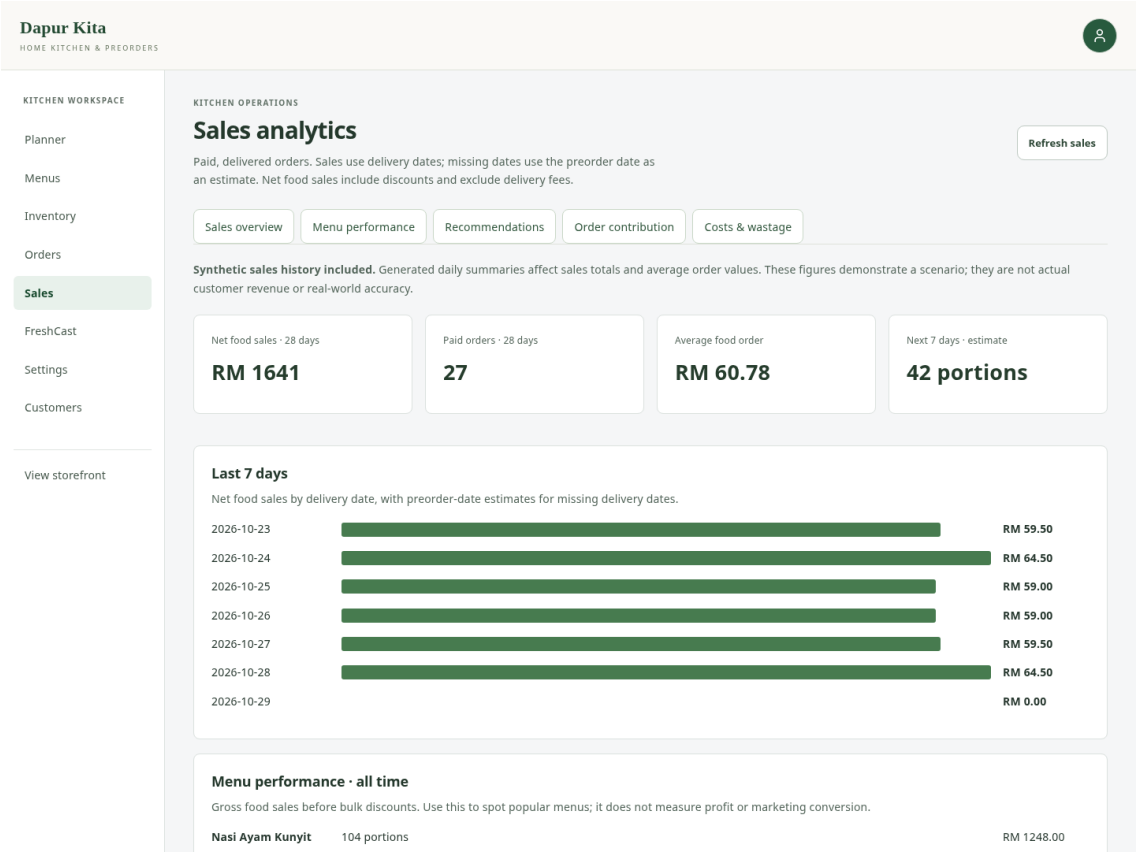


Figure 4.13: Owner analytics with distinct sales and evidence-based report tabs (local test data).

Current recipe estimates multiply ingredient requirements by estimated unit costs and add packaging. Price/bulk-offer scenarios compare a hypothetical price, discount, ingredient increase and additional entered costs. Recorded order contribution instead requires accepted recipe quantities, actual consumption matched to those quantities, known batch purchase costs, accepted packaging and an eligible realised outcome. Synthetic sales summaries intentionally lack real cooking/consumption transactions, so they may provide demand history while contribution remains unavailable.

A missing contribution is therefore not an arithmetic failure or RM0. It means the evidence needed to compute it is absent or incomplete. A completed, calculated zero or negative result is retained. At the current implementation boundary, food contribution excludes delivery income/cost, payment fees, labour and overhead unless a specific scenario includes entered assumptions. It is not net profit or audited financial accounting.

The system also records operating expenses and actual waste with request IDs, quantity bounds and voiding/correction workflows where supported. Disposal is a stock-changing event; a FreshCast forecast is not. A cancelled order does not automatically refund money or restore already consumed ingredients.

4.4.7 SETTINGS AND OPERATIONAL CHECKS

Settings provides kitchen hours, public identity/story, contact details, collection/delivery choices, fees, payment instructions, setup checks and email diagnostics. Technical diagnostics and recovery controls use progressive disclosure so routine owner tasks are not dominated by environment-variable names. Forms distinguish short delivery-area choices from the narrative service-area description and the public collection address.

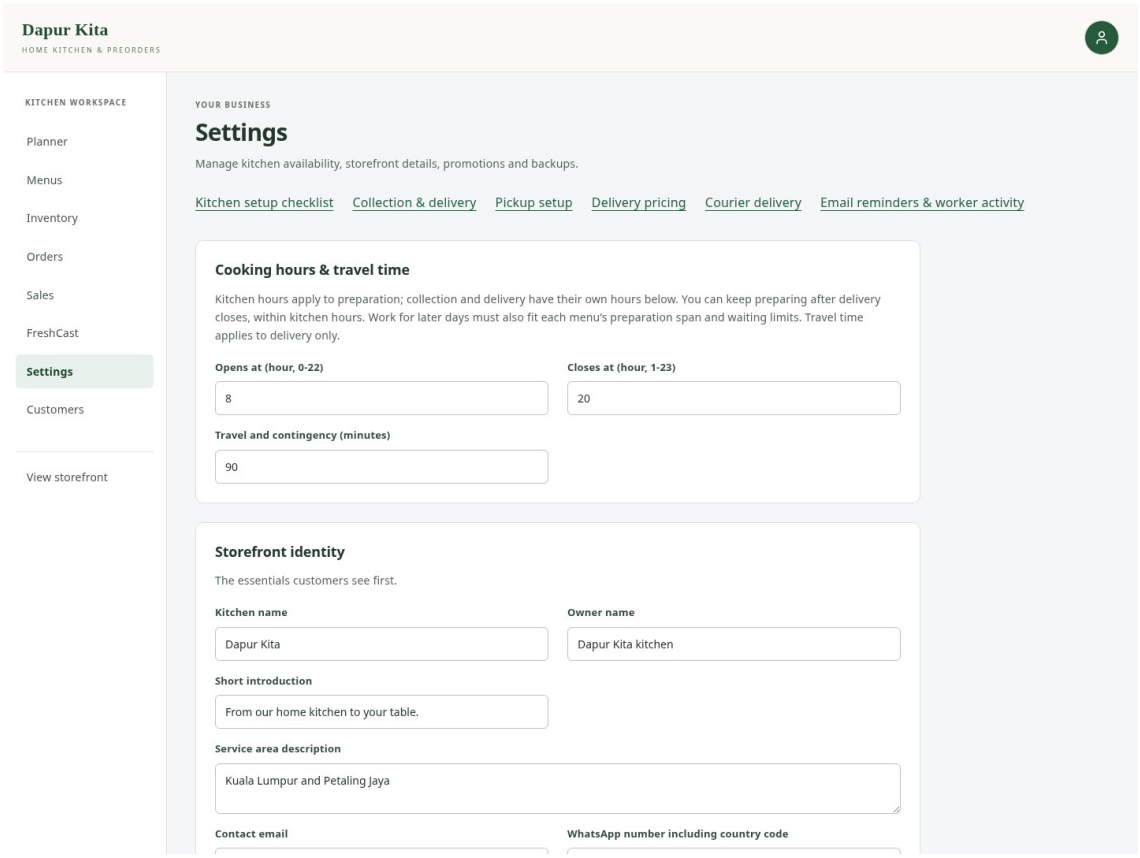


Figure 4.14: Owner Settings and focused configuration sections (local test settings).

During save, dependent controls are disabled to avoid editing values that an eventual response would replace. A failed save keeps the draft and offers another save rather than a destructive reload. An initial read failure offers a read retry. Existing order schedules and accepted fees remain snapshots rather than being retrospectively replaced by new defaults.

Setup checks describe recorded configuration such as recipes, cooking steps and costs; they do not certify food safety, production readiness or email inbox receipt. Backup controls link source/model recovery separately from private kitchen data. Changing a UI label does not itself change old database rows, and a Git push does not prove the production deployment is Ready.

4.5 KEY CALCULATIONS AND TRANSACTION RULES

4.5.1 EFFECTIVE DEADLINE AND REMAINING WINDOW

For manufacture mode, the base deadline is manufacture date plus documented shelf-life days. For receipt mode, it is received date plus documented storage-life days. For printed mode, it is the entered printed deadline. Applicable original printed, opened-plus-days and thawed-plus-days limits can only shorten the effective deadline. In notation, E = minimum of the valid applicable deadlines. Uncertain handling or a known storage breach holds the lot independently of E.

For an assessable non-empty received lot, the displayed remaining-window indicator is bounded between 0 and 100: $R = 100 \times (E - \text{today}) / (E - S)$, where S is manufacture date for manufacture mode and receipt date otherwise. The code reports unknown when an appropriate denominator or required evidence is missing. Expired/held batches contribute zero to the aggregate; unknowns are separately counted. The aggregate is an equal-weight mean over assessed batches, not a quantity-weighted measure of physical freshness. Stock dated today can still be eligible through that calendar day even though its remaining-window percentage is zero.

This implementation meets the record-monitoring aspect of Objective 2 without claiming a laboratory result. It does not detect temperature excursions, pathogens or food safety from appearance. Calendar-day guidance cannot be silently substituted for a short hourly holding rule. Owners remain responsible for applying reliable ingredient-specific guidance and actual conditions.

4.5.2 CURRENT PORTIONS, FUTURE CAPACITY AND FEFO

For a current recipe with eligible quantities Q_i and per-selling-unit requirements r_i, current possible units are the floor of the smallest Q_i/r_i ratio. Missing or invalid recipes do not create fictitious stock. Stock eligibility excludes non-positive, held, already-expired and future-receipt lots. This is an ingredient quantity calculation, separate from a date’s labour/equipment capacity or future procurement.

At food-ready confirmation, required quantity for ingredient i is accepted quantity per unit times ordered units. The server locks and consumes usable batches in earliest-expiry order, records per-batch consumption and cost evidence, and sets inventory_deducted. A shortage rolls the transaction back. Planning allocations and scenarios are read-only; they must not decrement batches in advance or deduct the same lot for two hypothetical requirements within one calculation.

4.5.3 PREPARATION WORK AND RESOURCE FIT

Batch count is ceiling(ordered selling units / accepted batch size). In combined mode, each step's duration is base duration plus (batches − 1) times its additional-batch duration. Independent mode represents repeated runs rather than simply inflating one uninterrupted cooking step. Explicit task order, resource, hands-on flag, permitted waits/pauses and overnight permission define the schedule.

Total step time is not elapsed window or worker load. Unattended steps can overlap other resources, while one worker cannot perform two hands-on tasks together. The conservative greedy backward search may reject a feasible arrangement a human could find; it is not a proof of optimal scheduling. Existing bookings remain fixed unless an authorised reviewed amendment changes them.

For the starter banana-cake settings, ten boxes imply three batches. The six step durations total 540 minutes, with 180 hands-on minutes. The 8:00–20:00 kitchen window is 720 minutes, so the example warning is not simply nine hours exceeding twelve. The daily selling limit is eight boxes. The preview now begins at a quantity within that cap and numerically explains an explicitly larger sample. Reducing its preview quantity does not change the menu's daily limit or book a real slot.

4.5.4 PRICES, PAYMENT AND ACTUAL CONTRIBUTION

Let F be the sum of accepted unit prices times quantities, D the applicable food discount and H the accepted delivery fee. The payable amount is F − D + H. Order.total_amount is discounted food value; shipping_fee is stored separately. Client totals are not trusted. PaymentEvent records external/manual evidence, including completed, pending and failed outcomes. A unique request ID and provider reference checks support safe replay.

For a complete realised delivered-paid order, recorded food contribution is discounted food revenue minus actual batch ingredient cost minus accepted packaging estimate. Cooked cancellations and refunds have their defined revenue treatment. Missing consumption, missing costs or unpaid delivery can make the value unavailable. Current recipe scenarios cannot reconstruct those historical facts.

4.6 FRESHCAST AND FORECASTING IMPLEMENTATION

4.6.1 THREE DATA SOURCES AND THEIR PURPOSES

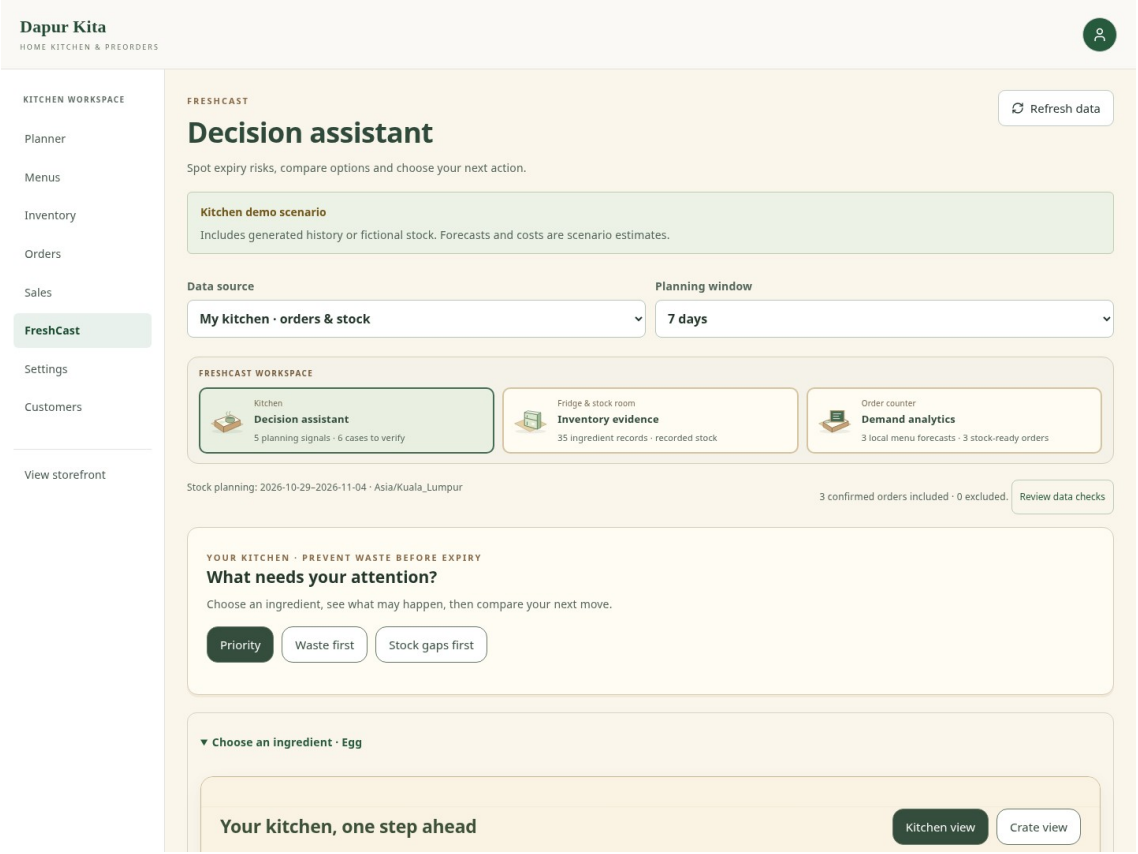


Figure 4.15: Kitchen FreshCast decision assistant with source and evidence labels (local scenario).

Table 4.3: Three data sources and their purposes.

Source	Demand and inventory basis	Appropriate interpretation
My kitchen	Database orders, accepted recipes and dated ingredient batches; local menu forecasts	Kitchen-specific decision support, with generated-history warnings where applicable
Genpact	External weekly centre/meal data and saved CatBoost artifact; explicit simulated operational CSVs	External ML demonstration, not a trained predictor for the kitchen's named dishes
Fixed illustrative walkthrough	Frontend fixtures with predefined quantities and actions	Model-free explanation/tutorial; no database or failed-API fallback

The interfaces share a decision structure but retain source-specific controls and chart units. Genpact uses centres, meal IDs and weeks; local kitchen uses dates, named menus, native ingredient units and accepted bookings. A fixed sample is useful for explaining the journey without external dependencies, but it must remain labelled and optional. Its inputs do not validate predictive accuracy.

4.6.2 LOCAL MENU AND AGGREGATE FORECASTS

The current per-menu decision path in kitchen_decisions.py uses paid, delivered historical portions, delivery dates and current menu identity checks. Historical renamed menus are excluded rather than assigning their past sales to a different recipe. Missing delivery dates use explicitly identified reporting fallbacks. Absent days mean zero recorded sales, not verified zero latent demand. Generated history remains marked as generated evidence.

Candidate daily methods are a trailing 28-day mean, trailing seven-day mean and same-weekday average. With at least 42 recorded calendar days, the last 28 days are split into 14 validation and 14 test days. Methods are selected using validation MAE; the held-out test is reported separately. Forward planning recursively uses predicted values inside a future window. A separate full-window diagnostic avoids describing rolling one-day predictions as demonstrated multi-day accuracy. Sparse history uses a limited baseline rather than inventing a confidence interval.

Known future commitments are subtracted from total forecast demand before speculative needs are added. Plannable extra portions are constrained by current recipes, offered dates, cutoffs, lead notice and daily limits. Forecast portions become ingredient quantities through recipes and conservative dated stock allocation. Low/high sensitivity cases vary demand by error/limited-history assumptions; they are not statistical confidence intervals. Detailed tracing connects menu portions to ingredient need, batch use, uncovered need and unused eligible stock at expiry.

A separate aggregate weekly kitchen_forecast.py comparison can fit ridge autoregression using lag1, lag2 and rolling-four-week features, alongside last-week and moving-average baselines. Its chronological train/validation/test split and refit are distinct from per-menu decision forecasting. The simpler sales-summary estimator in forecast.py is another reporting baseline. Consequently, the report does not state that every kitchen prediction is CatBoost or that no kitchen routine ever fits coefficients. The deployed per-menu decision logic is a local statistical forecasting process; it does not require the external Genpact training dataset.

4.6.3 GENPACT CATBOOST MODEL

The saved external CatBoost model (Prokhorenkova et al., 2018) uses 456,548 historical rows from 77 centres. Source tables join weekly meal demand with meal category/cuisine and centre metadata. Features include centre/meal identity, prices,

promotion flags, area, earlier demand lags, a shifted four-observation mean and the interval since the last observation. Shifted features use only earlier observations at each target week.

The training script uses CatBoostRegressor with 200 iterations, depth 6, learning rate 0.08, RMSE loss and seed 42, fitting log(1 + demand). Predictions are transformed back with expm1 and bounded below by zero. Training uses eligible weeks through 135; the saved rolling one-step holdout covers weeks 136–145. The artifact was restored and independently re-evaluated for the 29 October recovery evidence; no retraining was performed for this report. Its reproduced WAPE is discussed in Chapter 5 and is not a kitchen-customer accuracy result.

The operational recipes, batches and supplier inputs for the Genpact view are explicitly simulated. Model meal IDs are not matched to Product IDs merely because both are integers. Changing a promotion flag produces an associational what-if, not measured campaign uplift. Unsupported operational centres fail explicitly instead of silently receiving invented inventory.

4.6.4 EXPIRY RISK, ACTIONS AND FOLLOW-UP

An expiring batch’s projected unused quantity equals eligible starting stock minus confirmed allocation minus additional forecast use before its deadline, bounded below by zero. A batch expiring later carries its remaining stock beyond the current window. Dated shortages and expiry surpluses can coexist: sufficient total stock does not mean it arrives before the demand or survives until it is needed. Zero projected waste can therefore accompany an uncovered demand gap.

FreshCast offers reviewable action comparisons, including preparation/menu review, monitoring and a hypothetical purchase with arrival, expiry, quantity and optional cost. A comparison uses a common baseline and preserves source labels. It does not actually buy stock, publish promotions, change orders or promise savings. Signed decision snapshots and browser-held follow-up notes help relate later outcomes to an earlier scenario; browser storage can be cleared, and recorded outcomes are not causal proof that a recommendation saved money or prevented waste.

4.7 PAYMENTS AND ASSISTED ORDERING

PayPal Sandbox follows an order/capture interface (PayPal, n.d.) and supports saved provider order creation, buyer approval, server capture verification, order-bound payment records and recoverable status checking. The interface uses PayPal as the

primary label with an explicit test-payment note. Webhook verification and owner checks support reconciliation when configured; a browser approval flag cannot create a completed payment. Live-money merchant activation and live refund reconciliation are separate deployment/business work.

Order-bound escrow uses Sepolia only when the server configuration is complete. Its invoice binds a random reference, contract, buyer/seller/arbiter, MYR record, configured test-wei amount, dispatch deadline and minimum confirmations. The server verifies funding independently over JSON-RPC and writes one ledger receipt or an observed refund event. It does not hold private keys. The separate wallet tool and generated demo-credit balance remain demonstrations, not funded bank accounts. A public-chain production escrow service would require independent contract/security and legal review.

Kitchen Help provides approved topic answers in supported languages and can use an optional bounded AI classifier. The Plan an order assistant extracts limited fields, presents proposals and uses authenticated basket/checkout tools. It does not gain arbitrary SQL or unrestricted authority to mark paid, invent availability or silently place a customer order. Fixed approved answers and bounded actions reduce hallucinated policy or payment instructions; external model failure has a local recovery path.

4.8 EMAIL REMINDERS, RECOVERY AND DEPLOYMENT

One authenticated worker serves the owner and all opted-in customers. The owner’s preparation/delivery reminders cover eligible open events within 24 hours, including overdue events up to 24 hours. Customer advance reminders become eligible within two hours before the agreed time or arrival-window start and require confirmed preparation plus consent. Dispatch and agreed-time updates are separate choices. At most three emails are attempted per call, with room for owner work; the worker prioritises the defined customer-update queues.

The GitHub Actions workflow declares a five-minute schedule and is enabled by a repository variable with a private worker secret matching the server. A five-minute poll does not send the same reminder every five minutes: successful event receipts suppress routine repeats. A schedule can be delayed; a successful manual run does not prove automatic cadence. SMTP acceptance is not inbox receipt, and a crash between SMTP acceptance and saving its receipt can still produce a duplicate retry. Settings

exposes current run/queue diagnostics and an owner test-email action separately from a normal status refresh.

Website code, the Genpact runtime archive and private kitchen data require different backups. GitHub source does not contain the persistent database or automatically include a Release asset. The saved model archive has a recorded SHA-256 for recovery; uploaded menu photos are part of the private PostgreSQL backup. The release build checks for pending migrations rather than automatically mutating production schema. Credentials and backups containing customer information do not belong in the public repository.

4.9 INTERFACE AND ACCESSIBILITY PRINCIPLES

The visual references supplied during development informed compact food cards, prominent but limited primary actions, readable grouping, sticky basket access and clear progress. They were used as design inspiration rather than copied screens or proof of usability. Nielsen’s visibility, consistency, error prevention and recovery principles support showing the actual basket quantity, distinguishing waiting requests from bookings, retaining failed-save drafts and preventing duplicate submission (Nielsen, 1994).

Progressive disclosure groups optional contact, recipe and technical details without hiding the next required action. Semantic form labels, keyboard focus and native dialogs support access. Automated Axe checks used WCAG A/AA-related rules in selected desktop, tablet and phone states; this is partial verification against accessibility concerns, not complete WCAG conformance certification (W3C, 2023). Table detail and graph units remain explicit where simplification would lose evidence.

Success popups close after four seconds, information after six and errors after ten; hover/focus can pause them. Persistent inline form errors are retained where needed for correction. A cart success starts after its refresh, and a successful write with failed read uses truthful recovery. This prevents notifications from disappearing while an action still appears busy and avoids forcing the owner to dismiss every routine confirmation.

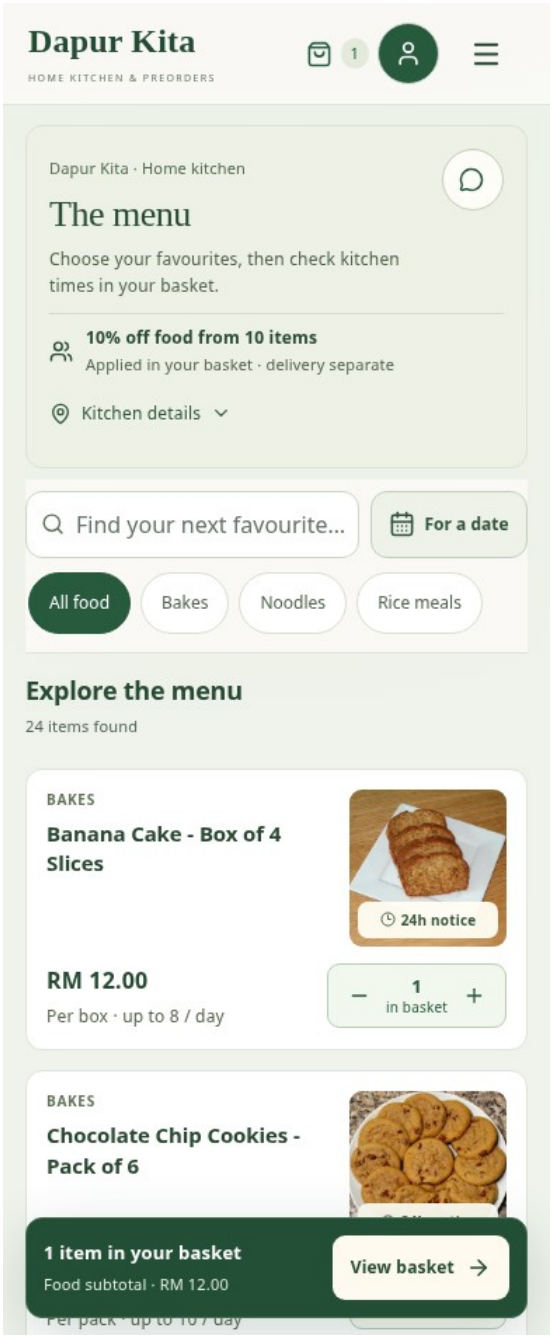


Figure 4.16: Phone menu with compact navigation, food cards and basket access (390-pixel local viewport).

4.10 CONCLUSION

The implemented interfaces support the five objectives through a coherent chain: branded discovery, constrained suggestions, authoritative checkout, accepted order evidence, reviewable preparation, dated stock and actual fulfilment. Supporting payment, reminder and forecasting features preserve their external boundaries. The following chapter evaluates these behaviours using repeatable tests and explains which practical outcomes still need validation with intended users.

CHAPTER 5

TESTING AND RESULTS

5.1 INTRODUCTION

This chapter evaluates the implemented system against the original five objectives. It separates functional verification, database concurrency checks, interface inspection, external-model evaluation and validation with intended users. The university briefing highlights testing that is verifiable, handles exceptions and security concerns, and relates findings to the objectives. Accordingly, a passing screen or build is not used as evidence of customer satisfaction, real food safety, commercial conversion or on-time courier delivery.

The latest verification was executed for source revision ab1a976 on 29 October 2026. Tests used disposable local databases, in-memory email or mocked provider responses. Current screenshots were captured with isolated test accounts and data. No production customer order, stock quantity, payment or email subscription was changed for this report. Earlier dated browser and external-model checks are identified as retained evidence rather than newly invented field observations.

5.2 TESTING APPROACH AND ENVIRONMENT

The approach combines white-box unit tests for calculations, black-box API behaviour, integration/retry tests, PostgreSQL concurrency tests and browser-based interface checks. Positive cases are paired with rejected inputs, stale state, duplicate requests and interrupted responses. The same assertion can serve more than one objective, but it is counted once in its suite. Model metrics are reported separately from software test counts.

Table 5.1: Testing approach and environment.

Environment or evidence	Configuration / scope	What it establishes
Application baseline	Git ab1a976; React/Django code and current migrations	Reproducible source identity, not production deployment status
Full backend regression	In-memory SQLite; dotenv disabled; in-memory mail; restored original model paths	Application calculations, permissions and response contracts
Concurrency/SQL checks	Temporary PostgreSQL 16; loopback-only TLS connection; disposable database	Actual row locks, races, transactional effects and schema-qualified correction SQL

Environment or evidence	Configuration / scope	What it establishes
Frontend verification	Node utility/rendering suites, ESLint and Vite production build	Tested response/state logic, component output and compilability
Browser evidence	Chromium; actual local bundle; selected 1440/768/390/320-pixel states	Tested interaction/recovery, selected automated accessibility and overflow checks
External-model evidence	Original checksum-verified CatBoost artifact and its historical holdout	Reproduced external-data prediction error, not local business accuracy
User evaluation	Protocol prepared; no supplied participant observations	An executable future validation method, not completed UAT

The regression manifest records each command, exit code and log SHA-256. The full backend run discovered 829 tests. It executed 806 successfully in SQLite and skipped 23 requiring PostgreSQL. Those same 23 were then executed separately against PostgreSQL and all passed. Thus all 829 discovered application tests were exercised across the two database environments; this does not mean that all 829 were repeated on PostgreSQL. The restored-model tests ran rather than being skipped for missing artifacts.

The first temporary PostgreSQL setup lacked the TLS support required by the application connection settings, so it failed before tests executed. The fixture was rebuilt with local TLS and the full 23-check selection then passed. The application’s production TLS requirement was not weakened to obtain a passing result. This setup correction is distinguished from a product defect or a production database failure.

5.3 DECISION TABLES

5.3.1 CHECKOUT AND REQUEST CONFIRMATION

Table 5.2: Checkout and request confirmation.

Rule	Sale/date rules valid	Automatic preparation fits	Customer requests review	Expected outcome
C1	Yes	Yes	No	Recheck and save confirmed task plan with explicit available payment choice
C2	Yes	No, permitted planning exception	Yes	Save unpaid deferred request; reserve no tasks

Rule	Sale/date rules valid	Automatic preparation fits	Customer requests review	Expected outcome
C3	No: past date, closed cutoff or disallowed day	Any	Any	Reject; do not create a valid booking by calling it a request
C4	Yes at quote, changed before placement	No longer matches	Any	Recompute or reject stale quote; do not trust old browser values
C5	Yes	Yes	No; repeated same checkout UUID	Return existing valid result without another order/debit
C6	Yes	Yes	Delivery with cash selected	Reject cash; cash is restricted to collection

5.3.2 BATCH ELIGIBILITY AND DEADLINES

Table 5.3: Batch eligibility and deadlines.

Rule	Received by allocation date	Effective date not passed	Held	Evidence	Expected allocation result
I1	Yes	Yes	No	Valid required input	Eligible positive stock; use FEFO ordering
I2	No	Any	No	Any	Exclude future receipt
I3	Yes	No	No	Any	Exclude expired batch
I4	Yes	Yes	Yes	Any	Exclude held batch independently of date
I5	Yes	Yes	No	Calculation mode lacks source/duration	Reject incomplete save; do not invent a life
I6	Yes	Several limits	No	Earlier opened/thawed /printed limit	Earliest applicable deadline controls

A displayed remaining-window percentage is not an additional safety decision rule. In particular, expiry-day stock can have zero remaining-window percentage while the calendar-day deadline has not yet passed. Uncertain handling remains a hold regardless of the percentage.

5.3.3 ACTUAL FOOD READINESS AND GROUPED PROGRESS

Table 5.4: Actual food readiness and grouped progress.

Rule	Valid accepted plan / transition	Version current	Enough eligible stock	Already deducted	Expected outcome
F1	Yes	Yes	Yes	No	Consume accepted ingredients once; log batches; mark ready
F2	Yes	Yes	Any	Yes	Repeated readiness does not deduct stock again
F3	No	Any	Any	No	Reject; confirm preparation before actual readiness
F4	Yes	No	Any	Any	Reject stale selection; retain current state
F5	Yes	Yes	No	No	Roll back all selected orders and deductions
F6	Collection order	Yes	Not relevant to dispatch	Any	Reject out-for-delivery; confirm collection after actual handover

5.3.4 PAYMENT, REMINDER AND BUDGET EVIDENCE

Table 5.5: Payment, reminder and budget evidence.

Condition	Expected behaviour
PayPal browser says approved but verified amount/currency does not match	No completed ledger receipt or paid status

Condition	Expected behaviour
Sepolia hash is submitted but confirmations are insufficient	Retain recoverable pending evidence; remain unpaid
Repeated valid provider receipt or payment-choice request	One durable result; no duplicate charge/receipt caused by the application
Customer has only a phone number	Contact route remains available; no SMTP reminder to a phone number
Customer consent absent, email changed or plan unconfirmed	Exclude advance-reminder event
Successful reminder receipt already exists	Suppress routine repeat for that event; do not send every polling interval
Budget absent	No price restriction; comparison control inactive
Complete candidate total equals budget	Within; Above is strictly greater
Mixed basket target three with one current item	Add only two items to reach three total, subject to constraints
One-menu alternative replaces an existing basket	Require explicit replacement confirmation and current basket fingerprint

5.4 FUNCTIONAL AND SECURITY TEST CASES

The following cases summarise executed automated tests. Each source name is a locator in backend/myapp or the named frontend utility, not a signature from an independent human tester. Logs include the individual test methods and outcomes. Test fixtures are intentionally synthetic and cannot substantiate user adoption or measured sales effects.

Table 5.6: Executed functional cases BT-01–BT-06.

Case / objective	Test action	Expected and observed	Executed source locator
BT-01 Obj. 1 Whole basket quantity	Request target three with two different menus.	Three total items; never three of each.	test_basket_guide.BasketGuideTests.test_three_means_three_total_mixed_items_not_three_of_each_menu PASS
BT-02 Obj. 1,5 Budget includes delivery	Search with food within budget but delivery exceeding it.	Filter using complete order quote.	test_basket_guide.BasketGuideTests.test_whole_order_budget_includes_existing_basket_and_delivery PASS

Case / objective	Test action	Expected and observed	Executed source locator
BT-03 Obj. 5 Same-menu alternatives	Search same-menu mode while another menu is in basket.	Independent options; explicit replacement required.	test_basket_guide.BasketGuideTests.test_same_menu_offers_alternatives_and_requires_explicit_replacement PASS
BT-04 Obj. 5 No-budget search	Omit optional budget in same-menu mode.	Alternatives returned without price restriction.	test_basket_guide.BasketGuideTests.test_same_menu_without_budget_returns_alternatives_without_price_restriction PASS
BT-05 Obj. 5 Price partition	Compare within/above before limiting three results.	Requested price relation applied before truncation.	test_basket_guide.BasketGuideTests.test_budget_ranges_are_partitioned_before_three_result_limit PASS
BT-06 Obj. 1,5 Suggested addition retry	Apply returned option twice.	One atomic addition and no reservation.	test_basket_guide.BasketGuideTests.test_success_is_atomic_idempotent_and_reserves_no_kitchen_or_stock PASS

Table 5.7: Executed functional cases BT-07–BT-12.

Case / objective	Test action	Expected and observed	Executed source locator
BT-07 Obj. 1 Date-only suggestion	Add dated option without choosing an hourly slot.	Day rechecked, no hour reserved.	test_basket_guide.BasketGuideTests.test_date_only_add_checks_day_without_selecting_or_reserving_an_hour PASS
BT-08 Obj. 1 Separate hours	Plan handover after kitchen closes.	Cooking still inside kitchen hours.	test_adaptive_planning.AdaptivePlanningTests.test_delivery_can_follow_kitchen_closing_without_late_cooking PASS
BT-09 Obj. 1 Later-day preparation	Cook after delivery closes for a later allowed order.	Fit preparation and explicit earlier-finish bounds.	test_adaptive_planning.AdaptivePlanningTests.test_preparation_can_continue_after_delivery_closes_for_an_allowed_later_order PASS
BT-10 Obj. 1 Review request	Submit a short-notice eligible request.	Unpaid with no reserved tasks.	test_adaptive_planning.AdaptivePlanningTests.test_short_notice_request_is_unpaid_and_reserves_no_tasks PASS

Case / objective	Test action	Expected and observed	Executed source locator
BT-11 Obj. 1 Invalid dates	Submit invalid/unoffered time via request path.	Request cannot bypass sale/date rules.	test_adaptive_planning.AdaptivePlanningTests.test_invalid_or_unoffered_times_still_cannot_be_submitted PASS
BT-12 Obj. 1 Delivery cash bypass	Forge delivery cash and paid flag.	Rejected; basket retained.	test_order_payment_choice.OrderPaymentChoiceTests.test_delivery_rejects_cash_even_with_forged_paid_status_without_losing_basket PASS

Table 5.8: Executed functional cases BT-13–BT-18.

Case / objective	Test action	Expected and observed	Executed source locator
BT-13 Obj. 1 Collection cash	Use valid cash at collection.	Unpaid collection; forged paid state ignored.	test_order_payment_choice.OrderPaymentChoiceTests.test_cash_at_collection_still_works_and_ignores_forged_paid_status PASS
BT-14 Obj. 1 Deferred payment	Create request needing review.	No instructions, debit or provider charge.	test_order_payment_choice.OrderPaymentChoiceTests.test_review_request_is_deferred_without_instructions_debit_or_provider_call PASS
BT-15 Obj. 1 Payment choice replay	Repeat selection with same request ID.	One choice; no charge or changed total.	test_order_payment_choice.OrderPaymentChoiceTests.test_choice_is_once_only_retry_safe_and_does_not_charge_or_change_total PASS
BT-16 Obj. 1 Private payment endpoint	Use other customer/staff identity or omit CSRF.	Ownership/CSRF enforced.	test_order_payment_choice.OrderPaymentChoiceTests.test_own_order_and_csrf_required_including_staff PASS
BT-17 Obj. 1,3 Private bounded history	Read current/history/all for different accounts.	Only owned counts/rows; bounded response.	test_order_history.OrderHistoryTests.test_default_is_bounded_and_counts_never_include_other_customers PASS
BT-18 Obj. 1,3 Stable cursor	Insert a newer order while paging older rows.	No duplicates or missing older records.	test_order_history.OrderHistoryTests.test_cursor_survives_new_order_without_duplicates_or_missing_older_rows PASS

Table 5.9: Executed functional cases BT-19–BT-24.

Case / objective	Test action	Expected and observed	Executed source locator
BT-19 Obj. 4 Large-history queries	Increase history to 1,000 fixture rows.	Ten-row response; bounded data-query count.	test_order_history.OrderHistoryTests.test_response_and_query_count_are_bounded_with_large_history PASS
BT-20 Obj. 2 Optional ingredient guidance	Patch legacy material without guidance.	No compulsory or accidental guidance deletion.	test_storage_guidance.StorageGuidanceTests.test_optional_and_legacy_patch_do_not_require_or_clear_guidance PASS
BT-21 Obj. 2 Guidance is explicit	Update reference after an old batch exists.	No automatic application or recalculation.	test_storage_guidance.StorageGuidanceTests.test_guidance_is_not_automatically_applied_or_retroactive PASS
BT-22 Obj. 2 Guidance access	Read/write guidance as customer/anonymous.	Staff boundary enforced.	test_storage_guidance.StorageGuidanceTests.test_customer_and_anonymous_cannot_read_or_change_guidance PASS
BT-23 Obj. 1 Reminder exclusions	Use unconfirmed, closed, past or inactive order.	No eligible customer advance event.	test_customer_reminders.CustomerReminderTests.test_unconfirmed_unreserved_closed_past_and_inactive_orders_are_excluded PASS
BT-24 Obj. 1 Reminder consent change	Opt out or change account email.	Old-recipient send suppressed.	test_customer_reminders.CustomerReminderTests.test_opt_out_and_changed_email_suppress_old_recipient_until_enabled_again PASS

Table 5.10: Executed functional cases BT-25–BT-30.

Case / objective	Test action	Expected and observed	Executed source locator
BT-25 Obj. 1 Reminder send failure	Fail SMTP and retry later.	Bounded attempts; failed event stays retryable.	test_customer_reminders.CustomerReminderTests.test_failed_sends_are_bounded_and_retry_later PASS
BT-26 Obj. 1 Shared mail budget	Queue owner and customer reminders together.	At most three attempts with owner capacity.	test_customer_reminders.CustomerReminderTests.test_shared_budget_preserves_owner_capacity_and_caps_three_attempts PASS

Case / objective	Test action	Expected and observed	Executed source locator
BT-27 Obj. 2 Concurrent stock overdraw	Competing menus consume shared batches in PostgreSQL.	Cannot overdraw; consistent lock order.	test_postgres_operations.PostgreSQLOperationsTests.test_opposite_menu_order_locks_stock_consistently_and_cannot_overdraw PASS
BT-28 Obj. 2 Stale stock edit	Consume concurrently while old stock form is open.	Old version cannot restore consumed quantity.	test_postgres_operations.PostgreSQLOperationsTests.test_stale_stock_edit_cannot_restore_concurrently_consumed_quantity PASS
BT-29 Obj. 1 Concurrent checkout replay	Repeat same checkout identity concurrently.	One order and one debit.	test_postgres_commitments.PostgreSQLCommitmentTests.test_same_checkout_reference_concurrently_creates_one_order_and_debit PASS
BT-30 Obj. 1 Concurrent last capacity	Two customers request final daily portion.	Cannot reserve the same final capacity twice.	test_postgres_commitments.PostgreSQLCommitmentTests.test_two_buyers_cannot_reserve_the_last_portion_twice PASS

Table 5.11: Executed functional cases BT-31–BT-36.

Case / objective	Test action	Expected and observed	Executed source locator
BT-31 Obj. 1 Concurrent shared plan	Confirm same common reservation concurrently.	One reservation and replay-safe result.	test_store_extensions.SharedCookingConcurrencyTests.test_simultaneous_confirmation_saves_one_shared_reservation_once PASS
BT-32 Obj. 2 Historical sample correction	Run targeted storage correction twice.	Preserve quantities/audit; repeat is a no-op.	test_sample_stock_storage.SampleStorageSQLTests.test_exact_targets_preserve_values_audit_before_after_and_repeat_is_noop PASS
BT-33 Obj. 2 Earlier deadline	Supply printed, opened and thawed evidence.	Earliest applicable limit preserved.	test_costs_freshness.CostsAndFreshnessTests.test_earliest_deadline_and_original_preserved PASS
BT-34 Obj. 2 Evidence anchor	Use manufacture and receipt calculation modes.	Different starts calculate appropriate deadlines.	test_expiry_evidence.ExpiryEvidenceTests.test_manufacture_and_storage_have_different_anchors PASS

Case / objective	Test action	Expected and observed	Executed source locator
BT-35 Obj. 2 Handling hold	Use uncertain handling with a future deadline.	Held despite future date.	test_expiry_evidence.ExpiryEvidenceTests.test_uncertain_handling_is_held_even_with_a_future_computed_deadline PASS
BT-36 Obj. 2 Accepted FEFO	Allocate accepted ingredients against dated stock.	No stock reuse; accepted recipe retained.	test_local_planning.LocalPlanningTests.test_fefo_does_not_reuse_stock_and_honours_accepted_recipe PASS

Table 5.12: Executed functional cases BT-37–BT-42.

Case / objective	Test action	Expected and observed	Executed source locator
BT-37 Obj. 1 Multi-day pause	Plan opted-in large preparation over two days.	Explicit sessions and pause reservation fit.	test_multiday_planning.MultiDayPlanningTests.test_large_opted_in_preparation_spans_two_days_with_reserved_pause PASS
BT-38 Obj. 1 Cooking continuity	Request oversized uninterrupted cooking.	Cooking not silently split.	test_multiday_planning.MultiDayPlanningTests.test_large_uninterrupted_cooking_is_not_silently_split PASS
BT-39 Obj. 4 Unknown contribution	Omit an actual cost.	Missing is not zero profit.	test_commitments.CommitmentTests.test_unknown_cost_is_not_zero_profit PASS
BT-40 Obj. 3 Photo upload	Upload a large valid image as staff and non-staff.	Sanitisation/resize and access boundary enforced.	test_store_extensions.StoreExtensionsTests.test_upload_resizes_large_photos_and_staff_boundary PASS
BT-41 Obj. 1 PayPal capture	Repeat verified capture.	One bound completed receipt.	test_paypal.PayPalTests.test_capture_verifies_server_response_and_is_idempotent PASS
BT-42 Obj. 1 Escrow confirmations	Use insufficient confirmation/RPC failure.	No paid state without verification.	test_escrow_orders.EscrowOrderTests.test_insufficient_confirmations_and_rpc_failure_never_mark_paid PASS

5.5 REGRESSION RESULTS

Table 5.13: Regression results.

Verification group	Observed result	Interpretation / limit
Full Django application discovery	829 discovered; 806 passed in SQLite; 23 skipped there	The skip reasons require PostgreSQL features
Deferred PostgreSQL checks	23 passed, zero skips, temporary PostgreSQL 16	The skipped application checks were actually exercised separately
Release, archive, backup and entrypoint checks	34 passed	Packaging/recovery controls and SPA entrypoint; not live Vercel health
Frontend core, UX, checkout and Planner suites	All commands passed	Tested state, calculations and rendering contracts
Recommendation, predictive and shopping suites	All commands passed	Source/budget/session boundaries and response handling
PayPal, escrow and image suites	All commands passed	Local verification/recovery and image-processing behaviour
ESLint	Passed	Static checks; does not prove absence of security defects
Vite production build	Passed	Bundle compiled; does not measure live loading time
Production providers / real customers	Not exercised in this run	No inferred SMTP inbox, live capture, courier or UAT result

The current pass results supersede the 29 September snapshot in the earlier Chapters 4–6. That historical draft reported 87 backend tests, frontend lint errors and an unresolved escrow test. It would be inaccurate to carry those outcomes forward as the present release status. Equally, passing the larger suite is not proof that every possible defect has been removed. Exact commands, identities and logs accompany this document’s evidence package, and Appendix F maps the principal source files.

5.6 INTERFACE, RECOVERY AND PERFORMANCE FINDINGS

5.6.1 RESPONSIVE AND ACCESSIBILITY CHECKS

Retained browser checks for the latest five-page review exercised menu, basket, checkout, Settings and My orders at desktop, tablet and phone widths. Their assertions checked selected WCAG A/AA-related Axe rules, document overflow, error states and actual production-bundle interactions. The Settings review covered delayed saves and

failed-save draft retention; the history review covered filters, repeated pages, a failed append, a failed refresh and stale delayed responses. The latest notification review included a five-second delayed basket refresh, repeated quantity notices, removal and a confirmed write followed by a failed read.

In those documented states, the automated checks reported no violations of the selected rules or horizontal document overflow. These are constrained results: keyboard-only usability, screen-reader comprehension and complete WCAG conformance are not established by Axe alone. The screenshots newly captured for this report illustrate the current implementation and were visually inspected; capturing an image is not an additional human participant test.

5.6.2 DEFECTS AND RETESTED CORRECTIONS

Table 5.14: Defects and retested corrections.

Concrete trigger	Earlier behaviour	Current correction and verification
Slow basket GET after a successful Add	Toast interval could run while the action still appeared busy	Success starts after refresh; browser delay test passes
Quantity increase/removal	No success popup for those paths	Distinct timed notices; repeated actions tested
Successful basket write but failed read	Ambiguous recovery could encourage another Add	Saved-change information; one persisted addition verified
Ten-box banana-cake preview with eight-box daily cap	Default sample immediately warned without explaining the numeric mismatch	Default respects cap; explicit over-limit sample and preview-only correction tested
Edit Settings while save is pending	Later response could overwrite newer draft edits	Related controls disabled during save; failure retains draft
Long customer history	Full history fetched despite ten initially displayed cards	Ten-row API paging, private counts and stable cursor
One-menu finder with an existing basket	Existing item could dominate alternatives	Independent alternatives and explicit replacement confirmation tested
Delivery checkout with hidden cash selection	Could imply unsuitable courier cash collection	Server rejects delivery cash; collection cash remains available

These corrections address visibility, consistency, error prevention and recovery rather than merely changing colours. The observed behaviour supports Nielsen’s principles applied in Chapter 4. It does not quantify reduced owner workload or customer abandonment, because timed user sessions and field data were not supplied.

5.6.3 BOUNDED WORK AND QUERY EVIDENCE

The order-history regression increases a customer's fixture history to 1,000 records while keeping a page at ten records and the data-query count at no more than three in the controlled API fixture. It verifies owner-scoped counts, stable older-ID pages when new orders arrive, and rejection of malformed or repeated parameters. The query count excludes the cost of normal production authentication middleware and does not establish a production latency target.

Other bounded interfaces include menu pagination, at most 30 whole-basket capacity checks in the finder, at most 20 grouped progress updates and a three-email attempt budget per worker call. Uploaded media is bounded and re-encoded, and optional payment/forecast components are separated from ordinary catalogue browsing. The current build also retains a main-chunk size advisory above 500 kB after minification. Build success does not remove that optimisation opportunity. These limits control work but are not a load test. No live Core Web Vitals, peak concurrent-user capacity, database capacity forecast or guaranteed email throughput was measured for this report.

5.6.4 SECURITY AND CONCURRENCY FINDINGS

Access tests distinguish public menu data, a customer's own basket/orders and staff business records. CSRF is required for state-changing session requests. Forged price, paid flags, foreign order IDs, stale versions and invalid provider evidence do not gain authority. Photo uploads validate decoded content rather than trusting an extension or MIME string. Financial CSV text is protected against spreadsheet-formula interpretation. Test provider endpoints reject evidence that does not bind to the accepted order.

The 23 PostgreSQL cases exercise simultaneous final-capacity checkout, repeated checkout IDs, recipe/plan confirmations, wallet concurrency, grouped basket replacement, account-email identity, reminder worker overlap, deletion locks and stock edits/consumption. These tests establish the selected race outcomes under actual row locking. They are not an external penetration test, a review of all deployed Supabase RLS policies or an audit of the public Sepolia contract deployment. Runtime secrets, scheduler activation, SMTP configuration and live provider identities require operational verification.

5.7 FORECAST EVALUATION AND DATA LIMITS

5.7.1 EXTERNAL GENPACT MODEL RECOVERY

The retained 29 October model-recovery evidence restored the original runtime archive, verified SHA-256 697f8e10ee6b2c1b794ee38ec81067477870315fb7c51a1280684e7f0601361a and reloaded its saved model. Independent predictions on the original 32,821-row holdout reproduced the stored rounded metrics. This was artifact recovery and re-evaluation, not new training or an independent new test set.

Table 5.15: External genpact model recovery.

Measure	Reproduced value	Evaluation context
Historical source size	456,548 rows, 77 centres	External centre/meal dataset
Holdout	Weeks 136–145; 32,821 rows	Earlier weeks used for training
CatBoost WAPE	28.23494%	Rolling one-step-ahead predictions
Previous-observation baseline WAPE	34.80762%	Same holdout and demand target
Absolute WAPE difference	Approximately 6.57 percentage points lower	Error comparison, not sales uplift
API checks in the retained recovery run	17 passed, zero skips	Actual restored artifact used; no production DB

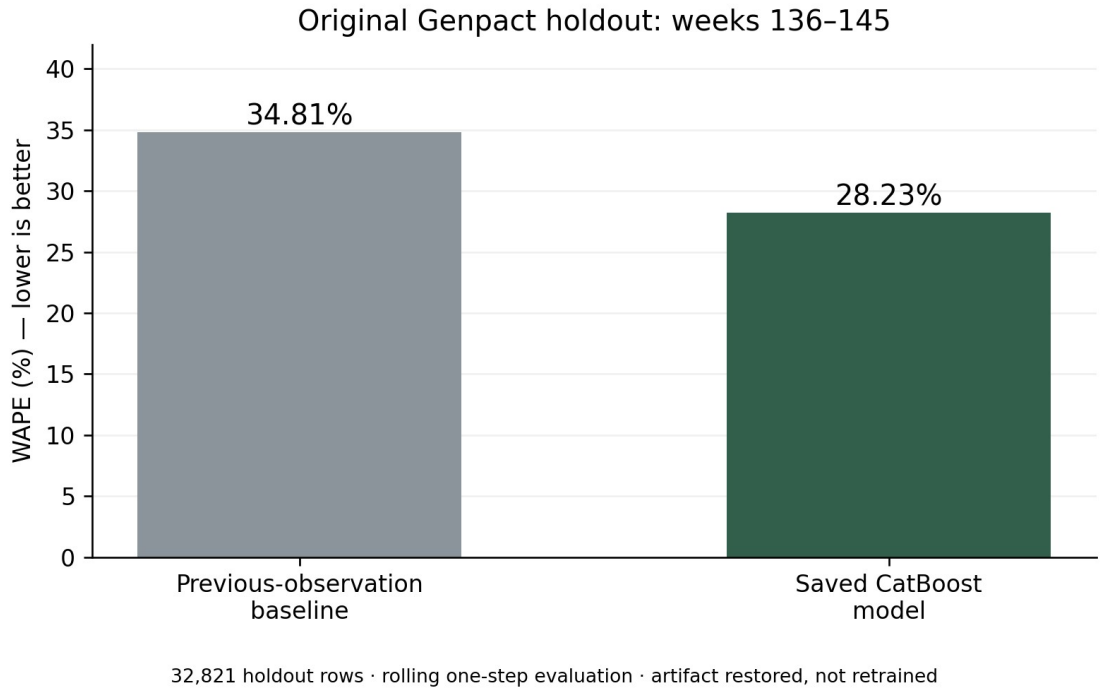


Figure 5.1: Reproduced Genpact rolling one-step WAPE on the original holdout; lower is better.

WAPE is 100 × sum of absolute prediction errors divided by sum of absolute actual demand. A 28.23% WAPE is not 71.77% customer accuracy or a guarantee about each meal. The holdout uses actual earlier observations as they become available; multi-week recursive performance and promotion causality are different questions. The external model's lower error supports this benchmark comparison, while its transfer to a small kitchen with different named menus is not established.

5.7.2 KITCHEN FORECASTS AND SYNTHETIC HISTORY

The kitchen path can translate eligible local demand into recipe needs without a Genpact model. Its menu-level baseline selection and held-out calculations are deterministic and tested for chronological separation. However, much of the showcase history is generated, and zero recorded sales on absent days may reflect missing recording rather than true no-demand days. Renamed-menu exclusions, date fallbacks, paid/delivered eligibility and order-coverage warnings are necessary for interpreting the output.

Changes to preparation dates, order status, recipe identity or stock records can change included bookings, costs and future allocations. They do not by themselves create more representative historical demand. The normal baseline calculation reads current eligible history; the saved Genpact model does not need retraining merely because a planner task is corrected. Fitting a more complex local model is justified only

after collecting consistent representative history and comparing it chronologically with simple baselines.

A prior offline aggregate local-booking experiment did not outperform its four-week baseline and is not the live predictor. This is a useful negative finding: connecting a database and adding machine learning does not guarantee better prediction. No new kitchen accuracy percentage, waste reduction, realised savings or marketing uplift is claimed from generated orders or forecasts in this report.

5.8 VALIDATION AGAINST THE OBJECTIVES

Table 5.16: Validation against the objectives.

Original objective	Implemented and verified evidence	What remains to validate
1. Flexible date-based preordering and scheduling	Whole-basket quotes, menu dates/cutoffs, independent hours, accepted plans, guarded multiday work and replanning	Task success and acceptable booking effort for actual owners/customers
2. Freshness by shelf life and storage duration	Three evidence modes, reusable guidance, effective deadlines, holds, FEFO and traceable consumption	Correct application of guidance in practice; no physical-safety claim
3. Personalised single-vendor storefront	Kitchen identity/story, menu/category/photo management and a competitor-free storefront	Owner branding comprehension and usefulness in a real deployment
4. Sales analytics for decisions	Defined sales/cost cohorts, contribution unknowns, expenses/waste, exports and source-labelled forecasts	Operator interpretation and improved decisions using representative data
5. Recommendations reducing fatigue and improving conversion	Explainable immediate cards, optional mixed/same-menu finder, budget/capacity checks and telemetry	Decision-time/fatigue comparison and genuine conversion outcomes

The matrix supports implemented feature coverage and verified behaviour. It does not rewrite Objective 5 to avoid its outcome claim. That outcome remains an evaluation question. Observational recommendation events can describe engagement, but repeat visits, menu price, availability and exposure differences can confound causal conversion comparisons. A simulated completed checkout is task completion, not a commercial sale.

5.9 USER ACCEPTANCE AND FIELD-EVALUATION PROTOCOL

5.9.1 STATUS, PARTICIPANTS AND ETHICS

No genuine respondent dataset, completed evaluation form, signed independent test sheet or field timing observations were supplied for this report. Accordingly, this section specifies the prepared evaluation rather than claiming it happened. The senior report's participants, percentages, signatures and charts are not reused. The supervisor should agree recruitment size, consent, ethical requirements and the final rubric before any session. The supplied briefing does not independently prescribe a universal sample size.

Recruit participants representing both home-food operators and customers. Use participant codes instead of names in analysis and test orders instead of real financial transfers. Record role, device, viewport, source revision and relevant experience. Explain that date-based freshness is a planning record and test payments move no real funds. Permit withdrawal and minimise unnecessary personal data. Keep consent and any private contact details separate from anonymised observations.

5.9.2 TASKS LINKED TO OBJECTIVES

Table 5.17: Tasks linked to objectives.

Task	Participant role	Action and success criterion	Objective
U1	Customer	Find an available menu, select a future handover and save a valid test preorder	1
U2	Customer	Recover from an unavailable date without guessing several rejected times	1
U3	Owner	Record unlabelled chilled stock using a matching documented reference; identify its deadline and hold	2
U4	Owner	Explain why a held/expired batch cannot cover an accepted order	2

Task	Participant role	Action and success criterion	Objective
U5	Owner	Update kitchen story/contact and a menu image, then locate the public result	3
U6	Customer	Identify the single kitchen and its collection/delivery/contact conditions	3
U7	Owner	Explain the difference between unknown contribution, a cost estimate and actual food contribution	4
U8	Owner	Interpret a shortage alongside an expiring surplus and choose a justified review action	4
U9	Customer	Choose a valid three-item basket within a stated whole-order budget using ordinary browsing	5
U10	Customer	Complete an equivalent choice using recommendations and describe the reasons/total	5
U11	Owner	Confirm several eligible ready orders and identify a deliberately ineligible member	1, 4

Record independent/assisted/failed completion, elapsed time, errors, requests for help and whether the participant can explain the outcome. Do not count a task completed by the facilitator as independent success. Interview notes should distinguish interface confusion from deliberately inconsistent fixture settings.

5.9.3 RECOMMENDATION COMPARISON AND SURVEY

Use equivalent menu-choice tasks with the same date, quantity and budget constraints. Counterbalance whether each participant starts with browsing or recommendations and rotate the food sets to reduce learning effects. Measure time to a valid selection and perceived decision difficulty using the same scale. Report medians and dispersion, completion counts with denominators and common qualitative

problems. A small convenience sample can suggest improvements but does not establish population-wide efficacy.

The short survey in Appendix E maps each question to the objectives. A five-point agreement scale can be used consistently, with a separate not-applicable option. Ask about successful tasks and observed difficulty rather than asking whether the participant agrees the project is successful. Retain anonymised raw data and calculation rules before generating charts. If real conversion is studied later, predefine exposure, purchase window, consent/privacy handling and comparison conditions; do not label unrelated page visits as attributed purchases.

5.9.4 ACCEPTANCE, DEFECT HANDLING AND REPORTING

Agree acceptance criteria before collection. At minimum, no unauthorised cross-account disclosure, duplicate charge, stock overdraw or false delivered state is acceptable. Owner/customer tasks must have defined valid outcomes and understandable recovery. For every observed problem, record severity, reproduction steps, source revision, correction and retest result. Report assisted and failed tasks as well as successes. An independent reviewer can sign an actual execution record after testing; this report does not insert a fabricated sign-off.

The briefing's consultation-log, originality/similarity-report and submission instructions require the student's genuine documents. A completed software test log does not replace supervisor meetings or an institutional similarity report. These items are recorded as author completion requirements outside the main technical evidence.

5.10 DISCUSSION IN RELATION TO THE LITERATURE

Chapter 2 identifies the operational gap between informal ordering and more complex industrial food-monitoring systems. The implementation responds with accepted digital orders, explicit kitchen constraints and low-cost date evidence rather than requiring sensors or industrial routing infrastructure. This design is consistent with the relevance of food quality and service expectations discussed by Tan and Kim (2021) and the home-food safety perspective of Pallianysamy et al. (2025), while it cannot infer microbiological safety from those studies or from its own dates.

The distinction from advanced supply-chain work is practical scope. The application records and reasons about owner-entered evidence; it does not implement the physical monitoring examined by Tsang et al. (2019) or certify that a refrigerator maintained a temperature. It also separates operational planning from stochastic demand

modelling and does not claim to reproduce the inventory-routing results discussed by Violi et al. (2024).

The menu finder addresses choice effort through a small explainable set, total-order prices and explicit recovery. That is a design rationale and a testable hypothesis, not a new experimentally demonstrated contribution to decision-fatigue literature. The external model benchmark supports one demand-error comparison only. The project’s contribution is the implemented integration of these lightweight workflows and traceable constraints for a single kitchen; practical business benefits remain hypotheses for the planned validation.

5.11 CONCLUSION

The current source passed its application, concurrency, release and frontend checks within the stated local environments. The strongest evidence concerns authoritative business rules, transaction/retry safety, dated stock, clear state separation and bounded interfaces. Model-recovery evidence is reproducible but external to the kitchen’s real demand. User acceptance, decision-fatigue reduction, conversion improvement and long-term food/operational outcomes require the genuine observations specified above. Chapter 6 draws conclusions within these limits.

CHAPTER 6

CONCLUSION AND SUGGESTIONS

6.1 INTRODUCTION

This final chapter reviews the project against its unchanged objectives, discusses development challenges, advantages and limitations, and prioritises future improvements. Dapur Kita is a web-based system for one home-food business rather than a marketplace or a universal solution for every seller. Chapters 3 and 4 describe its implemented design, while Chapter 5 supplies repeatable software-verification evidence and an honest boundary for user evaluation.

The project integrates a branded storefront, flexible preordering, recipe-aware planning, dated ingredient inventory, analytics and recommendations. Supporting test payments, reminder emails, courier coordination and FreshCast expand the demonstration without replacing the original academic purpose. References and appendices follow this chapter; they are supporting matter rather than additional main chapters.

6.2 ACHIEVEMENT OF THE PROJECT OBJECTIVES

6.2.1 FLEXIBLE DATE-BASED PREORDERING AND SCHEDULING

The system implements future date/time selection, per-menu offered dates and weekdays, order cutoffs, advance notice, daily quantity limits and independent preparation/delivery/collection hours. Whole-basket feasibility checks consider one worker, equipment, closures and accepted commitments. Calculated and manual-reviewed plans, bounded searches for later times and guarded multi-day work support flexibility without automatically overriding handling limits.

Confirmed plans and unpaid requests are explicit states. Snapshots retain accepted recipes, preparation and fulfilment terms, while signed previews and transactional confirmation support changes. Grouped progress and shared cooking reduce repeated administration in compatible cases. These functions demonstrate the first objective’s implementation. The remaining evaluation is whether intended operators and customers can use them efficiently under representative workload; software tests alone cannot establish that outcome.

6.2.2 INGREDIENT SHELF LIFE AND STORAGE-DURATION MONITORING

The inventory module supports printed dates, manufacture-plus-life and receipt-plus-life evidence, optional reusable guidance, earlier opened/thawed limits, handling holds and remaining-window indicators. Received, expired, future and held stock are treated explicitly. FEFO and recorded consumption connect the monitoring feature to actual cooking and traceability rather than a random freshness label.

The second objective is implemented as record-based monitoring. Its contribution is a structured operational aid, not a sensor or food-safety certification. Unlabelled wet-market food still requires reliable guidance matching its actual storage and state. The owner must not equate a date or green indicator with an assurance that food is safe.

6.2.3 PERSONALISED SINGLE-VENDOR STOREFRONT

The owner can maintain kitchen identity, story, service/contact information, menus, categories, prices and photographs, including local image uploads. The public catalogue presents that kitchen’s products and does not introduce a competitor feed. Optional size groups, selling units and fixed-content descriptions represent different home-food offers clearly.

These features fulfil the implementation scope of the third objective. Whether they measurably strengthen a specific seller’s brand should be assessed through owner and customer feedback. The current project is a configurable single-kitchen installation, not a hosted multi-vendor management platform with tenant isolation.

6.2.4 SALES ANALYTICS AND DATA-DRIVEN DECISIONS

The owner can inspect defined sales/menu trends, current recipe estimates, actual contribution where evidence is complete, expenses, waste and exports. FreshCast compares future recipe need with dated stock and explains unused expiry stock and shortages. Distinct data sources and missing-cost labels prevent estimates from being silently presented as realised profit or physical waste.

The fourth objective is supported by implemented calculations and tested report behaviour. Data quality remains a condition: generated history, missing batch costs and absent delivery dates limit interpretation. Representative real records and task-based owner evaluation are needed before claiming better financial or purchasing outcomes.

6.2.5 MENU RECOMMENDATIONS, DECISION FATIGUE AND CONVERSION

The system implements two complementary recommendation surfaces: immediate explainable menu cards and an optional date/quantity/budget finder. The finder permits mixed or one-menu alternatives, checks the total order and whole-basket capacity, and retains honest empty/recovery states. Session-bound telemetry supports later engagement analysis.

The fifth objective's recommendation system is implemented, but its intended reduction in decision fatigue and improvement in conversion are not yet empirically established by supplied participant data. The objective is deliberately retained unchanged. The planned counterbalanced task comparison and, where appropriate, a real conversion study should determine whether the mechanism produces those effects. A passing recommendation test or generated purchase cannot answer that behavioural question.

6.3 DEVELOPMENT CHALLENGES AND RESPONSES

6.3.1 DISTINGUISHING PLANNING FROM ACTUAL EVENTS

A major challenge was avoiding one status or date implying several different facts. A saved request is not accepted preparation; food readiness is not payment; dispatch is not delivery; projected expiry loss is not recorded waste. The response was to separate snapshots, financial events, inventory consumption and owner-confirmed progress, with contextual queues and recovery actions. This adds data concepts but makes the records more defensible.

6.3.2 FLEXIBILITY WITHOUT UNDOCUMENTED HANDLING ASSUMPTIONS

Some early combinations could not fit kitchen hours or required every menu to finish at the same dispatch instant. The scheduler was refined to allow a flexible same-day finish window, documented earlier work, independent batches and supported pause/resume tasks. However, making every task splittable or expanding an early-finish period without evidence would create a false capacity promise. The system therefore exposes constraints and reviewed alternatives rather than guaranteeing that any quantity fits any requested time.

6.3.3 HISTORICAL INTEGRITY AND CHANGING MENUS

Menus, ingredients, prices and delivery defaults change during development and business operation. The application now keeps accepted names/prices, ingredient quantities, preparation terms, address and fulfilment settings. Historical renamed menus are excluded from inappropriate local forecast mapping. Archive/deletion policies and explicit current-settings adoption protect relevant commitments. This supports traceability without inventing cooking costs for synthetic history.

6.3.4 EXTERNAL PAYMENTS, EMAIL AND RECOVERY

External systems can approve an operation while the browser loses its response. Checkout UUIDs, order-bound provider identities, transaction hashes, ledger events and retry rules preserve recoverable evidence. SMTP receipts reduce routine duplicate reminders, although inbox delivery and the acceptance/receipt crash window remain external limits. Sandbox and testnet environments permit demonstration while retaining clear disclosures. These challenges require more than displaying a success alert.

6.3.5 DATA QUALITY AND PREDICTIVE CLAIMS

The availability of a trained Genpact model did not make it a trained predictor for the kitchen’s different dishes. The system separates external IDs and simulated operations from database recipes and bookings. Local statistical baselines and coverage warnings remain useful when genuine history is sparse. The unsuccessful offline local candidate reinforced the need to compare models with baselines rather than assuming additional complexity creates accuracy.

6.3.6 OWNER WORKLOAD AND MOBILE USABILITY

Repeated status actions, long forms and technical diagnostic text can burden a sole operator. Progressive disclosure, focused daily queues, optional intermediate stages, grouped handover, compact cards, reliable timed notifications and retained drafts respond to that problem. Desktop and phone checks found and corrected concrete interface issues, but representative owner sessions are still needed to evaluate whether daily use is manageable.

6.4 ADVANTAGES OF THE IMPLEMENTED SYSTEM

The first advantage is integration around one kitchen’s accepted order. The same order connects the customer’s chosen food and handover with accepted recipes, time reservations, actual stock use, payment evidence and later reporting. That is more

structured than relying on an unlinked conversation and a separate spreadsheet, while the owner can still use WhatsApp and an external courier where needed.

A second advantage is explicit evidence. Unknown costs remain unknown, a review request stays unconfirmed, scenario stock is not physically deducted, and forecasts carry their source. This supports explainability for both a project demonstration and future business use. It also gives a supervisor traceable examples of negative and retry cases rather than only screenshots of successful pages.

A third advantage is configurable local operation. Selling units, daily limits, specific dates, cutoffs, collection hours, fees and fixed contents can represent meals, bakes, small batches and occasional offers. Seasonal shipping enquiries are handled as conversations rather than forced into a local delivery promise. The system is therefore suitable for a useful range of home-food profiles without claiming universal coverage.

A fourth advantage is recovery and maintainability. The code is versioned, migrations and source/model backups are separate from private data, bounded APIs control work, and tests exercise actual concurrency boundaries. A lost browser response or failed settings save has an explicit recovery route. These are practical foundations, although continued maintenance and deployment checks remain necessary.

6.5 LIMITATIONS

Table 6.1: Limitations.

Area	Present limitation	Practical implication
Scheduling	Conservative bounded greedy search; one worker and one unit per resource	Some feasible manual arrangements may not be discovered; larger kitchens need another resource model
Multi-day food work	Depends on documented waits, pauses and finish allowances	A larger span alone does not justify safe holding or interruption
Ingredients	Plans allocate advisory stock; procurement is not an integrated supplier purchasing system	Future bookings can need purchases; actual cooking can still be blocked by a shortage
Freshness	Owner-entered dates and conditions; no continuous physical measurement	Incorrect evidence or storage breaches can invalidate the record’s practical usefulness
Finance	Contribution is narrower than net profit; missing actual costs remain unknown	Generated history and current estimates cannot reconstruct historical accounting

Area	Present limitation	Practical implication
Payments	PayPal Sandbox and Sepolia prototype; manual transfer needs external confirmation	No claim of production-money merchant readiness or audited escrow security
Courier	Manual booking/tracking and owner-confirmed events	No automatic GrabExpress ETA or verified delivery event
Email	Polling, consent, SMTP and provider queues; at most three attempts per call	Delays or missed inbox receipt remain possible; workload may require capacity review
Recommendations	Heuristic ranking and bounded candidate search	Not exhaustive, not an allergy assessment and not proven to improve real conversion
Forecasts	Sparse/generated local history; external Genpact scope	Demonstration metrics do not establish local customer accuracy or realised waste reduction
Validation	Automated/local evidence and a prepared user protocol	Real owner/customer usability and business outcomes remain unmeasured

The application also depends on connectivity, correct deployment configuration and private backup discipline. Local tests cannot certify live database access policies, email scheduler cadence or the current production alias. Automated accessibility checks cover selected states rather than every assistive-technology journey. These limits should be retained in a final report instead of being removed to make the system appear complete.

6.6 PRIORITISED FUTURE DEVELOPMENT

6.6.1 VALIDATION WITH INTENDED USERS

The highest priority is to execute the objective-linked owner/customer protocol in Chapter 5, retain anonymised raw observations and correct/retest discovered problems. Measure selection time, comprehension and task success before claiming reduced effort or conversion improvement. A pilot with representative home-food operators should examine whether their actual batch, cutoff, pickup and handover practices match the configuration model.

6.6.2 REPRESENTATIVE DATA AND FINANCIAL COMPLETENESS

Collect consistently dated real completed sales and stable menu identities with consent and suitable privacy controls. Record actual consumption and batch costs where

contribution is required. For forecasting, compare baselines and any learned candidate with chronological validation and untouched tests across meaningful periods. Do not replace real demand with synthetic records to improve a metric. More complete supplier/procurement and financial reconciliation workflows should precede claims of full business accounting.

6.6.3 OPERATIONAL RELIABILITY AND MONITORING

Review reminder throughput against real volume, monitor queue age and failed sends, and verify the chosen scheduler rather than enabling duplicate schedules. Provider delivery/bounce information, bounded job recovery and operational alerting can improve observability. Retain consent and current-recipient checks. Stronger operational monitoring should also cover failed payments, stale plans, backups and release migration checks.

6.6.4 MORE GENERAL CAPACITY MODELLING

A future scheduler could support multiple workers, equipment quantities, delivery-route reservations and more systematic search, while preserving accepted commitments and stock evidence. Shared cooking could expand beyond the current strict compatible single-menu case after its physical and accounting implications are defined. Any optimisation should be evaluated against a clear baseline and real recipes; an algorithmic fit must not override food-handling limits.

6.6.5 OPTIONAL INTEGRATIONS AND SECURITY REVIEW

A courier adapter remains optional and is not required for the present FYP scope. If later justified by business need and approved provider access, it should use authenticated, deduplicated provider events and preserve manual exception handling. Automated WhatsApp messaging similarly requires official platform arrangements and consent; ordinary draft links should remain usable without it. Live payments or broader escrow require provider approval, secure operational setup and independent review before real funds are introduced.

6.7 MAINTENANCE AND HANDOVER

Maintain separate copies of source code, the original model archive and private PostgreSQL data, including uploaded photographs. Verify recovery in an isolated environment and retain archive hashes. Apply schema changes through a documented maintenance process before releasing code requiring them. Do not solve a failed

deployment by disabling its migration guard or placing database credentials in public source.

For a demonstration, start with internally consistent menu notice, capacity, task, date and stock settings. Clearly identify generated history and test payments. After changing orders, stock or settings, refresh relevant planning views. Confirm actual readiness and handover rather than allowing the passage of time to claim physical completion. A short operating guide and source map are provided in the appendices for repeatable handover.

6.8 CONCLUSION

Dapur Kita implements the five intended feature areas in one single-vendor web application: flexible preordering, record-based ingredient freshness, personalised branding, analytics and menu recommendations. The current verification supports its stated calculations, access boundaries, retry behaviour and selected interface flows. It also demonstrates how accepted order evidence can connect planning to actual stock consumption and later reporting.

The project should be presented as an implemented and tested academic system with explicit operating limits. It has not yet demonstrated real-world conversion uplift, reduced waste, guaranteed punctual delivery or physical food safety. Completing the proposed user evaluation and collecting representative operational evidence are the next steps for judging those effects. Within those boundaries, the work provides a coherent foundation for a home-food operator and a traceable basis for further study.

REFERENCES

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. Agilemanifesto.org

Bernama. (2022, April 7). Malaysia's e-commerce income can breach RM1 trillion again this year - MDEC. https://www.bernama.com/en/business/news.php?id=2069812

Department of Statistics Malaysia. (2025, November 26). Malaysia digital economy (#276): Contribution of ICT to GDP in 2024. https://www.dosm.gov.my/uploads/release-content/file_20260327194302.pdf

Django Software Foundation. (2026). Meet Django. Djangoproject.com

Eu, E. Z. R., & Sameeha, M. J. (2021). Consumers' perceptions of healthy food availability in online food delivery applications and its association with food choices among public university students in Malaysia. Frontiers in Nutrition, 8, 674427. https://doi.org/10.3389/fnut.2021.674427

Food poisoning turns fatal. (2024, June 12). The Star. https://www.thestar.com.my/news/nation/2024/06/12/food-poisoning-turns-fatal

foodpanda. (2012). Backgrounder: foodpanda APAC. https://www.foodpanda.com/wp-content/uploads/2021/01/EN-Backgrounder-foodpanda-APAC.pdf

FoodSafety.gov. (n.d.). Cold food storage chart. https://www.foodsafety.gov/food-safety-charts/cold-food-storage-charts

Gogliettino, M., Balestrieri, M., Ambrosio, R. L., Anastasio, A., Smaldone, G., Proroga, Y. T. R., Moretta, R., Rea, I., De Stefano, L., Agrillo, B., & Palmieri, G. (2020). Extending the shelf- life of meat and dairy products via PET-modified packaging activated with the antimicrobial peptide MTP1. Frontiers in microbiology, 10, 2963. https://www.frontiersin.org/journals/microbiology/articles/10.3389/fmicb.2019.02963/full

Google. (2026). Firebase documentation. Google Cloud. https://firebase.google.com/docs

Gou, Z., Gamage, W., Lau, S. S. Y., & Lau, S. S. Y. (2018). An investigation of thermal comfort and adaptive behaviors in naturally ventilated residential buildings In tropical climates: A pilot study. Buildings, 8(1), 1-17. https://doi.org/10.3390/buildings8010005

Grab. (2018, May 28). Grab launches GrabFood beta and unveils vision to be an everyday app. Grab Press Centre. https://www.grab.com/sg/press/business/grab-launches-grabfood-beta-and-unveils-vision-to-be-an-everyday-app/

Hansen, C. (2026). Preorder and freshness: Dapur Kita [Computer software and implementation evidence, revision ab1a976]. GitHub. https://github.com/CHplus2/preorder-freshness/tree/ab1a976

Islam, A. K. M. Z., & Ferworn, A. (2020). A comparison between agile and traditional software development methodologies. Global Journal of Computer Science and Technology, 20(2), 7-42. https://globaljournals.org/GJCST_Volume20/2-A-Comparison-between-Agile.pdf

Kandoli, L. (2022). Effect of food shelf life on food quality. Journal of Advances in Humanities and Social Sciences, 8(1), 38-42. http://www.tafpublications.com/gip_content/paper/jahss-8.1.5.pdf

Leau, Y. B., Loo, W. K., Tham, W. Y., & Tan, S. F. (2012). Software development life cycle AGILE vs traditional approaches. In International Conference on Information and Network Technology (Vol. 37, No. 1, pp. 162-167). https://ku-fpg.github.io/files/agile-traditional.pdf

Lok, Y. H., Liou, Z. Y., Teoh, K. B., Lim, J. T., Lim, K. E., Lim, P. E., & Kee, D. M. H. (2024). Influencing Factors on Customer Behavioral Intentions to Use a Food Delivery App: A Study of GrabFood in Malaysia. Advances in Global Economics and Business Journal, 5(1), 1-14. https://www.agebj.org/index.php/agebj/article/view/92

Nielsen, J. (1994, April 24). 10 usability heuristics for user interface design. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

OpenJS Foundation. (2026). Express documentation (Version 4.21). https://expressjs.com/

Pallianysamy, S., Abdul-Mutalib, N. A., Abidin, U. F. U. Z., Rashid, N. K. M., & Ahmad, N. H. (2025). Home-based Foods in Malaysia: A Food Safety Perspective. Pertanika Journal of Tropical Agricultural Science, 48(3). http://pertanika2.upm.edu.my/resources/files/Pertanika%20PAPERS/JTAS%20Vol.%204 8%20(3)%20May.%202025/01%20JTAS-3151-2024.pdf

PayPal. (n.d.). Orders API v2. PayPal Developer. https://developer.paypal.com/docs/api/orders/v2/

Peres, F. A. P., Bondarczuk, B. A., Gomes, L. D. C., Jardim, L. D. C., Corrêa, R. G. D. F., & Baierle, I. C. (2025). Advances in food quality management driven by Industry 4.0: A systematic review-based framework. Foods, 14(14), 2429. https://www.mdpi.com/2304-8158/14/14/2429

PostgreSQL Global Development Group. (2026). PostgreSQL 18.4 documentation. https://www.postgresql.org/docs/

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. Advances in Neural Information Processing Systems, 31. https://arxiv.org/abs/1706.09516

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26(August), 1-9.

Sixty-five Sungai Petani primary school pupils suffer food poisoning. (2024, May 21). The Star. https://www.thestar.com.my/news/nation/2024/05/21/65-sungai-petani-primary-school-pupils-suffer-food-poisoning

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

Statista. (2024). Online food delivery-Malaysia. https://www.statista.com/outlook/emo/online-food-delivery/malaysia

Susanto, D. A., Suef, M., Karningsih, P. D., & Prasetya, B. (2026). Quality Management System Model for Food SMEs. Sustainability, 18(2), 890. https://www.mdpi.com/2071-1050/18/2/890

Tan, H., & Kim, V. W. E. (2021). Examining the factors that influence consumer satisfaction with online food delivery in Klang Valley, Malaysia. Journal of Management Theory and Practice, 2(2). https://doi.org/10.37231/jmtp.2021.2.2.115

Teo, S. C., Liew, T. W., & Lim, H. Y. (2024). Factors influencing consumers' continuance purchase intention of local food via online food delivery services. Cogent Business & Management, 11(1). https://doi.org/10.1080/23311975.2024.2316919

Tsang, Y. P., Choy, K. L., Wu, C. H., Ho, G. T. S., & Lam, H. Y. (2019, August). An internet of things (IoT)-based shelf life management system in perishable food e-commerce businesses. In 2019 Portland International Conference on Management of Engineering and Technology (PICMET) (pp. 1-8). IEEE. https://ieeexplore.ieee.org/abstract/document/8893684

Violi, A., De Maio, A., & Fattoruso, G. (2024). Inventory management and delivery of perishable products with stochastic demands and risks consideration. Procedia Computer Science, 232, 2941-2949. https://www.sciencedirect.com/science/article/pii/S1877050924002886

World Wide Web Consortium (W3C). (2023, October 5). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

Zhou, X., Tang, J., Jacobs, T. L., & Saguy, I. S. (2025). Transforming food supply chains through digital tracking and monitoring technologies. Trends in Food Science & Technology, 163, 105142. https://doi.org/10.1016/j.tifs.2025.105142

APPENDIX A — SUPERVISOR CONSULTATION RECORD

The student’s completed institutional supervisor consultation log is to be attached separately. The supplied briefing indicates eight consultations. No attendance, meeting date, supervisor approval or signature is generated by this technical revision.

APPENDIX B — ORIGINALITY / SIMILARITY RECORD

Attach the actual institution-approved similarity/originality report and required submission forms after reviewing this report. No similarity score or institutional approval is claimed here. All figures, external guidance and reused proposal literature must remain attributed in the final submission.

APPENDIX C — DECLARATION OF INTEGRITY

Unsigned completion copy. Use the current institutional declaration form if its wording differs. The author must review and complete it before submission.

I hereby declare that the submitted work together with this form is my original work. The following are further details of this declaration which I approve.

All references and sources relevant to this submission (from prints or the Internet) are inspirations to my written idea and creativity as well as my understanding of used references and sources.

I did not copy any other people’s work, be it from amongst the people I know in real life or from random sources on the Internet. Similarities are expected for common facts but I understand that my worded works should not be too similar to other people’s works at an acceptable amount set by the college.

If I fail to satisfy my pledge, I am responsible for any errors or negligence that might occur in this submission and I accept the final decision made by the lecturer and the university college. I understand the risk of non-compliance to this pledge may result in a reduction of marks or failure.

Required detail	Author completion
Signed by	_____
Student’s name	CHRISTOPHER HANSEN
Student ID	2402081406
Current course code	_____
Current course lecturer	_____
Document title	IT PROJECT 2 REPORT — CHAPTERS 1–6
Date	_____

APPENDIX D — APPLICATION DATA DICTIONARY

This inventory was generated from the Django model metadata at revision ab1a976 without querying the production database. It lists the 26 application models and relevant field shape. Built-in User/authentication/session/migration tables are not duplicated. NULL means a nullable database field; blank text and nullable values are distinct. JSON fields store structured accepted terms or configuration, not implied relational links. Serializer/view validation adds rules beyond database field types.

MenuPhoto — myapp_menuphoto

Column	Django field type	Key / shape
digest	CharField	PK, max 64
content	BinaryField	Required by model shape
width	PositiveIntegerField	Required by model shape
height	PositiveIntegerField	Required by model shape
created_at	DateTimeField	Required by model shape

Category — myapp_category

Column	Django field type	Key / shape
id	BigAutoField	PK
name	CharField	UNIQUE, max 100
created_at	DateTimeField	Required by model shape

Product — myapp_product

Column	Django field type	Key / shape
id	BigAutoField	PK
selling_details	JSONField	Required by model shape
delivery_weekdays	JSONField	Required by model shape
delivery_dates	JSONField	Required by model shape
order_cutoff_time	TimeField	NULL
order_cutoff_days_before	PositiveSmallIntegerField	Required by model shape

Column	Django field type	Key / shape
selling_status	CharField	max 10
packaging_cost	DecimalField	NULL, 10 digits / 2 dp
lead_hours	PositiveIntegerField	Required by model shape
preparation_minutes	PositiveIntegerField	Required by model shape
preparation_tasks	JSONField	Required by model shape
max_preparation_days	PositiveIntegerField	Required by model shape
batch_size	PositiveIntegerField	Required by model shape
additional_batch_minutes	PositiveIntegerField	Required by model shape
packing_minutes_per_portion	PositiveIntegerField	Required by model shape
max_early_minutes	PositiveIntegerField	Required by model shape
daily_capacity	PositiveIntegerField	Required by model shape
social_url	CharField	max 200
name	CharField	max 200
description	TextField	Required by model shape
ai_summary	TextField	Required by model shape
category_id	ForeignKey	NULL, → Category; SET_NULL
price	DecimalField	10 digits / 2 dp
image_url	CharField	max 200
created_at	DateTimeField	Required by model shape
updated_at	DateTimeField	Required by model shape

RawMaterial — myapp_rawmaterial

Column	Django field type	Key / shape
id	BigAutoField	PK
estimated_unit_cost	DecimalField	NULL, 14 digits / 6 dp
storage_guidance	JSONField	Required by model shape
name	CharField	UNIQUE, max 100
unit	CharField	max 10

Column	Django field type	Key / shape
created_at	DateTimeField	Required by model shape

ProductIngredient — myapp_productingredient

Column	Django field type	Key / shape
id	BigAutoField	PK
product_id	ForeignKey	→ Product; CASCADE
raw_material_id	ForeignKey	→ RawMaterial; CASCADE
quantity_required	DecimalField	10 digits / 3 dp

Database constraints: [['product', 'raw_material']]

InventoryItem — myapp_inventoryitem

Column	Django field type	Key / shape
id	BigAutoField	PK
supplier	CharField	max 160
source_type	CharField	max 20
label_date_type	CharField	max 20
original_expiry_date	DateField	NULL
opened_date	DateField	NULL
after_open_days	PositiveIntegerField	NULL
thawed_date	DateField	NULL
after_thaw_days	PositiveIntegerField	NULL
handling_history	CharField	max 20
handling_note	CharField	max 500
unit_cost	DecimalField	NULL, 14 digits / 6 dp
expiry_basis	CharField	max 20
manufactured_date	DateField	NULL
shelf_life_days	PositiveIntegerField	NULL
storage_type	CharField	max 20

Column	Django field type	Key / shape
guidance_note	CharField	max 300
quarantined	BooleanField	Required by model shape
raw_material_id	ForeignKey	→ RawMaterial; PROTECT
quantity	DecimalField	12 digits / 3 dp
batch_code	CharField	max 100
storage_location	CharField	max 200
received_date	DateField	Required by model shape
expiry_date	DateField	Required by model shape
created_at	DateTimeField	Required by model shape
updated_at	DateTimeField	Required by model shape

Wallet — myapp_wallet

Column	Django field type	Key / shape
id	BigAutoField	PK
user_id	OneToOneField	UNIQUE, → User; CASCADE
balance	DecimalField	18 digits / 10 dp
wallet_address	CharField	UNIQUE, max 255
is_external	BooleanField	Required by model shape
created_at	DateTimeField	Required by model shape
updated_at	DateTimeField	Required by model shape

WalletTransaction — myapp_wallettransaction

Column	Django field type	Key / shape
id	BigAutoField	PK
wallet_id	ForeignKey	→ Wallet; CASCADE
amount	DecimalField	18 digits / 10 dp
type	CharField	max 20

Column	Django field type	Key / shape
reference	CharField	max 255
created_at	DateTimeField	Required by model shape

Address — myapp_address

Column	Django field type	Key / shape
id	BigAutoField	PK
recipient_name	CharField	max 150
user_id	ForeignKey	→ User; CASCADE
line1	CharField	max 255
line2	CharField	max 255
city	CharField	max 100
state	CharField	max 100
postal_code	CharField	max 20
country	CharField	max 100
phone	CharField	max 20
is_default	BooleanField	Required by model shape

Database constraints: <UniqueConstraint: fields=('user',) name='one_default_address_per_user' condition=(AND: ('is_default', True))>

CartItem — myapp_cartitem

Column	Django field type	Key / shape
id	BigAutoField	PK
user_id	ForeignKey	→ User; CASCADE
product_id	ForeignKey	→ Product; CASCADE
quantity	PositiveIntegerField	Required by model shape
added_at	DateTimeField	Required by model shape

Database constraints: [['user', 'product']]

Order — myapp_order

Column	Django field type	Key / shape
id	BigAutoField	PK
fulfilment_snapshot	JSONField	Required by model shape
checkout_request_id	UUIDField	UNIQUE, NULL, max 32
payment_method	CharField	max 20
payment_instructions	JSONField	Required by model shape
preparation_end_at	DateTimeField	NULL
preparation_plan	JSONField	Required by model shape
delivery_address	JSONField	Required by model shape
delivery_at	DateTimeField	NULL
preparation_at	DateTimeField	NULL
delivery_method	CharField	max 20
inventory_deducted	BooleanField	Required by model shape
user_id	ForeignKey	→ User; PROTECT
address_id	ForeignKey	NULL, → Address; SET_NULL
total_amount	DecimalField	10 digits / 2 dp
discount_amount	DecimalField	10 digits / 2 dp
shipping_fee	DecimalField	10 digits / 2 dp
status	CharField	max 20
payment_status	CharField	max 20
created_at	DateTimeField	Required by model shape
updated_at	DateTimeField	Required by model shape

OrderItem — myapp_orderitem

Column	Django field type	Key / shape
id	BigAutoField	PK
recipe_source	CharField	max 30
preparation_snapshot	JSONField	Required by model shape

Column	Django field type	Key / shape
packaging_unit_cost	DecimalField	NULL, 10 digits / 2 dp
order_id	ForeignKey	→ Order; CASCADE
product_id	ForeignKey	NULL, → Product; SET_NULL
product_name	CharField	max 200
unit_price	DecimalField	10 digits / 2 dp
quantity	PositiveIntegerField	Required by model shape
subtotal	DecimalField	10 digits / 2 dp

InventoryLog — myapp_inventorylog

Column	Django field type	Key / shape
id	BigAutoField	PK
inventory_item_id	ForeignKey	NULL, → InventoryItem; SET_NULL
change	DecimalField	12 digits / 3 dp
reason	CharField	max 255
reference	CharField	max 255
admin_id	ForeignKey	NULL, → User; SET_NULL
created_at	DateTimeField	Required by model shape

Storefront — myapp_storefront

Column	Django field type	Key / shape
id	BigAutoField	PK
delivery_enabled	BooleanField	Required by model shape
pickup_enabled	BooleanField	Required by model shape
pickup_location	CharField	max 250
pickup_instructions	TextField	max 1000
delivery_open_hour	PositiveSmallIntegerField	Required by model shape
delivery_close_hour	PositiveSmallIntegerField	Required by model shape

Column	Django field type	Key / shape
pickup_open_hour	PositiveSmallIntegerField	Required by model shape
pickup_close_hour	PositiveSmallIntegerField	Required by model shape
delivery_areas	JSONField	Required by model shape
delivery_fee_myr	DecimalField	8 digits / 2 dp
free_delivery_minimum_myr	DecimalField	NULL, 8 digits / 2 dp
manual_payment_enabled	BooleanField	Required by model shape
bank_transfer_instructions	TextField	max 1000
duitnow_qr_url	CharField	max 200
kitchen_open_hour	PositiveSmallIntegerField	Required by model shape
kitchen_close_hour	PositiveSmallIntegerField	Required by model shape
founder_name	CharField	max 100
founder_intro	CharField	max 250
founding_story	TextField	Required by model shape
food_philosophy	TextField	Required by model shape
signature_food	CharField	max 200
service_area	CharField	max 200
whatsapp_number	CharField	max 20
founder_image_url	CharField	max 200
name	CharField	max 100
tagline	CharField	max 200
story	TextField	Required by model shape
social_url	CharField	max 200
contact_email	CharField	max 254
delivery_buffer_minutes	PositiveIntegerField	Required by model shape
bulk_minimum	PositiveIntegerField	Required by model shape
bulk_discount_percent	PositiveIntegerField	Required by model shape

Review — myapp_review

Column	Django field type	Key / shape
id	BigAutoField	PK
product_id	ForeignKey	→ Product; CASCADE
user_id	ForeignKey	→ User; CASCADE
rating	PositiveSmallIntegerField	Required by model shape
comment	CharField	max 1000
created_at	DateTimeField	Required by model shape

Database constraints: [['product', 'user']]

OrderReminder — myapp_orderreminder

Column	Django field type	Key / shape
id	BigAutoField	PK
order_id	ForeignKey	→ Order; CASCADE
event	CharField	max 20
sent_at	DateTimeField	Required by model shape

Database constraints: [['order', 'event']]

KitchenBlock — myapp_kitchenblock

Column	Django field type	Key / shape
id	BigAutoField	PK
start_at	DateTimeField	Required by model shape
end_at	DateTimeField	Required by model shape
reason	CharField	max 160

Database constraints: <CheckConstraint: condition=(AND: ('end_at__gt', F(start_at))) name='kitchen_block_end_after_start'>

OperatingExpense — myapp_operatingexpense

Column	Django field type	Key / shape
id	BigAutoField	PK

Column	Django field type	Key / shape
request_id	UUIDField	UNIQUE, max 32
voided	BooleanField	Required by model shape
date	DateField	Required by model shape
category	CharField	max 20
amount	DecimalField	12 digits / 2 dp
note	CharField	max 300
created_at	DateTimeField	Required by model shape

WasteRecord — myapp_wasterecord

Column	Django field type	Key / shape
id	BigAutoField	PK
request_id	UUIDField	UNIQUE, max 32
inventory_item_id	ForeignKey	→ InventoryItem; PROTECT
quantity	DecimalField	12 digits / 3 dp
estimated_cost	DecimalField	NULL, 14 digits / 2 dp
reason	CharField	max 300
created_at	DateTimeField	Required by model shape

OrderIngredient — myapp_orderingredient

Column	Django field type	Key / shape
id	BigAutoField	PK
order_item_id	ForeignKey	→ OrderItem; CASCADE
raw_material_id	ForeignKey	→ RawMaterial; PROTECT
material_name	CharField	max 100
unit	CharField	max 10
quantity_per_portion	DecimalField	10 digits / 3 dp

IngredientConsumption — myapp_ingredientconsumption

Column	Django field type	Key / shape
id	BigAutoField	PK
order_item_id	ForeignKey	→ OrderItem; PROTECT
inventory_item_id	ForeignKey	→ InventoryItem; PROTECT
material_name	CharField	max 100
batch_code	CharField	max 100
unit	CharField	max 10
quantity	DecimalField	12 digits / 3 dp
unit_cost	DecimalField	NULL, 14 digits / 6 dp
recorded_expiry	DateField	Required by model shape
created_at	DateTimeField	Required by model shape

PaymentEvent — myapp_paymentevent

Column	Django field type	Key / shape
id	BigAutoField	PK
order_id	ForeignKey	→ Order; PROTECT
request_id	UUIDField	UNIQUE, max 32
kind	CharField	max 10
outcome	CharField	max 12
amount	DecimalField	12 digits / 2 dp
method	CharField	max 20
reference	CharField	max 200
note	CharField	max 500
source	CharField	max 30
resolves_id	OneToOneField	UNIQUE, NULL, → PaymentEvent; PROTECT
actor_id	ForeignKey	NULL, → User; SET_NULL
created_at	DateTimeField	Required by model shape

AuthAttempt — myapp_authattempt

Column	Django field type	Key / shape
id	BigAutoField	PK
key	CharField	max 64
created_at	DateTimeField	Required by model shape

OrderAmendment — myapp_orderamendment

Column	Django field type	Key / shape
id	BigAutoField	PK
order_id	ForeignKey	→ Order; PROTECT
actor_id	ForeignKey	NULL, → User; SET_NULL
before	JSONField	Required by model shape
after	JSONField	Required by model shape
created_at	DateTimeField	Required by model shape

RecommendationSession — myapp_recommendationsession

Column	Django field type	Key / shape
id	UUIDField	PK, max 32
owner_key	CharField	max 64
variant	CharField	max 20
context	JSONField	Required by model shape
products	JSONField	Required by model shape
created_at	DateTimeField	Required by model shape

RecommendationEvent — myapp_recommendationevent

Column	Django field type	Key / shape
id	BigAutoField	PK

Column	Django field type	Key / shape
session_id	ForeignKey	→ RecommendationSession; CASCADE
event	CharField	max 20
product_id	ForeignKey	NULL, → Product; SET_NULL
order_id	OneToOneField	UNIQUE, NULL, → Order; SET_NULL
request_id	UUIDField	UNIQUE, max 32
created_at	DateTimeField	Required by model shape

APPENDIX E — USER-EVALUATION INSTRUMENT

Prepared instrument; not an executed survey. Link an online form or paper instrument to these questions only after supervisor agreement and consent. Use the same five-point agreement scale (1 strongly disagree to 5 strongly agree), with Not applicable recorded separately. Preserve task outcomes and times in addition to opinion ratings. Do not publish respondent identity or raw addresses.

ID / objective	Customer or owner question
Q1 · Objective 1	I could identify a suitable future handover time without repeatedly guessing rejected times.
Q2 · Objective 1	The difference between an awaiting-confirmation request and a confirmed preorder was clear.
Q3 · Objective 2	I could record an ingredient batch using the correct date/evidence mode.
Q4 · Objective 2	I understood why a held or expired batch could not be used, and that a date indicator is not proof of food safety.
Q5 · Objective 3	I could identify this kitchen and understand its service/contact details without competitor distraction.
Q6 · Objective 3	The kitchen’s story, images and menu information supported the identity the owner wanted to present.
Q7 · Objective 4	I could interpret the report’s date range, total and unavailable-cost labels.
Q8 · Objective 4	The stock/demand explanation helped me identify a justified next action.
Q9 · Objective 5	I could find a suitable food choice using the suggestions.
Q10 · Objective 5	I understood the total item count and whole-order budget in a suggested option.
Q11 · Objective 5	Compared with the equivalent browsing task, choosing with suggestions required less mental effort.
Q12 · General	The interface gave understandable feedback and a useful recovery when something could not be completed.

Open questions: Which task was hardest, and why? Was any status, price, date or instruction misleading? Which feature would you keep, simplify or remove? For

owners, describe a real ordering/fulfilment case that the system did not represent. For customers, explain what would stop you completing a preorder.

Observation column	**Required entry**
Participant	Anonymous code, role, relevant experience; consent held separately
Context	Source commit, date, device/browser and viewport
Condition	Browsing or recommendation first; equivalent task set
Task	U1–U11 and actual assigned constraints
Timing	Start/end and elapsed seconds
Outcome	Independent / assisted / failed; valid result reference
Problems	Errors, help requests and observed confusion
Feedback	Optional anonymised comments; distinguish observation from interpretation

APPENDIX F — IMPLEMENTATION AND EVIDENCE MAP

The following source locators identify the implementation discussed in the report. Some older repository handoff files describe earlier revisions; current code and the dated final checks take precedence. The accompanying evidence bundle retains this report's isolated command logs, manifest, diagram sources, schema extraction and anonymised local screenshots.

Area	Source locator
Customer navigation and sessions	frontend/src/App.jsx; contexts/AuthProvider.jsx; backend/myapp/views/auth.py
Menu and immediate discovery	components/MenuFoodCard.jsx; components/MenuRecommendations.jsx; views/discovery.py; services/recommendation.py
Current whole-order finder	components/GuidedMenu.jsx; components/GuideBasketResults.jsx; views/basket_guide.py; services/basket_plans.py
Basket and checkout	contexts/CartProvider.jsx; pages/customer/CheckoutPage.jsx; views/cart.py; views/orders.py
Preparation and multiday	services/scheduling.py; services/work_sessions.py; views/kitchen.py; views/shared_batches.py
Actual progress	services/order_progress.py; views/order_progress.py; views/orders.py::deduct_order_inventory
Inventory and references	models.py; serializers.py; views/inventory.py; services/freshness.py
Menu photo sanitisation	views/menu_photos.py; MenuPhoto model
Sales, pricing and contribution	views/planning.py; views/costs.py; views/pricing.py; views/outcomes.py
Local forecasting / decisions	services/kitchen_decisions.py; services/kitchen_forecast.py; services/local_planning.py
External forecasting	backend/predictive_ai/forecast_core.py; train_model.py; service.py; evaluate_model.py
PayPal and deferred choice	views/paypal.py; views/paypal_notifications.py; views/order_payment_choice.py
Order-bound escrow	services/escrow.py; views/escrow.py; escrow/PreorderEscrow.sol
Customer history	views/orders.py::OrderList; contexts/OrderProvider.jsx; utils/orderHistory.js
Email and scheduling	services/reminders.py; views/reminders.py; .github/workflows/delivery-reminders.yml
Courier/contact	views/courier.py; views/handover.py; docs/courier-integration-readiness.md

Area	Source locator
Evidence and checks	scripts/verify_fyp.py; docs/CUSTOMER-FLOW-REVIEW-2026-10-29.md; docs/evidence/20261029-genpact-restore/

Reproduction environment: Python 3.13 for the application, installed frontend Node tooling, disposable SQLite/PostgreSQL and the original externally restored model. Tests must override any production DATABASE_URL, disable dotenv and use an in-memory email backend. Do not run tests against the persistent kitchen database. The run manifest records exact commands without production credentials.

APPENDIX G — SHORT OPERATING GUIDE

G.1 Owner setup

Save kitchen identity/contact, preparation hours and enabled collection/delivery settings. A collection location must be accompanied by enabled collection and valid hours. Configure fees and clear public handover instructions. Set each menu's selling unit, category, price, recipe, notice, dates/cutoff, capacity and actual steps.

G.2 Receiving ingredients

Create/select the material and native unit. Record the actual batch quantity, location and receipt. Choose the evidence mode; review documented guidance and conditions before copying it. Record earlier opened/thawed limits and hold uncertain stock. Check the effective deadline, then save.

G.3 Accepting and preparing orders

Review the daily queue. Automatically scheduled pending orders already have a reservation. For an unconfirmed request, agree a future time and calculate/review/confirm its plan before starting. Use accepted settings unless current changes are explicitly adopted. Block closures or your own delivery time.

G.4 Actual readiness and handover

Select eligible orders and confirm food actually ready; stock is consumed once. Record the courier externally and save its reference/contact. Confirm actual rider handover or customer collection/delivery. Intermediate progress is optional. Do not mark delivered merely because a clock time has passed.

G.5 Payment and reminders

Keep payment records separate from physical status. Cash is collection-only; manual transfer needs external evidence; Sandbox/Sepolia are test facilities. Customers choose reminders per order to their account email. Configure only one authenticated schedule and inspect run/queue status; a refresh does not send email.

G.6 Reading analytics and FreshCast

Inspect the date basis and missing-value labels. Current estimates are not historical contribution. In FreshCast, select the intended source, inspect exclusions and

distinguish shortages from unused dated stock. A scenario does not buy, consume or waste ingredients. Retain provenance for generated evidence.

G.7 Recovery

After a lost response, refresh the saved order/basket before creating a replacement. Retain the same supported request identity when retrying. On a stale-plan conflict, calculate a fresh preview. Keep source, original model archive and private database backups separately and test their restoration.